CeTTa:

A Direct C Runtime for the MeTTa Family

Programming, execution architecture and application measurements

Zar Goertzel
Oruži

5 November 2026
Tutorial and historical results: 27 October and earlier

Abstract

CeTTa is a direct C runtime for the MeTTa family, with PeTTa as its primary dialect and a distinct Hyperon Experimental-compatible lane. It combines symbolic rewriting and nondeterministic search with mutable spaces, indexed relational queries and native execution of admitted program fragments. The ordinary evaluator and the specialized machines share explicit call, answer, ownership and fallback contracts; data returned by one route must not be executed accidentally by another. A reusable C ABI and Python package expose engines, structured atoms, spaces, retained answers, streams, callbacks and numerical buffers.

This paper teaches the languages with executable examples, describes the runtime and its specification and proof boundaries, and measures the November preview against reference implementations, checking every timed run's answers. On nine application tasks whose MM2 programs come from MORK's own benchmarks and tutorials, ordinary PeTTa programs on CeTTa compute the same complete result sets as standalone MORK, and CeTTa's whole process is faster than MORK's execution alone on every task with either of MORK's two builds, by 2.4 to 460 times against the faster one. On 36 graph-analytics, query and search workloads written for MORK's experimental leapfrog join, CeTTa is faster on 34 and within 2% on the other two, at a geometric mean of 0.55 of MORK's time. CeTTa's PeTTa lane prints the same output as SWI-PeTTa on 161 of the 164 PeTTa examples, with the three differences explained, and is faster on all timed examples but one; its HE lane is faster than Hyperon Experimental on all 17 shared programs that both complete, by a geometric mean of 36, and completes three on which Hyperon Experimental aborts. The nuPLN inductive learner runs five times faster than on SWI-PeTTa. The ai-mathematician's premise selection matches its Python original's results and runs slightly faster, measured on an earlier binary because the preview fails that stage. Runs that fail or do not finish are reported as such, never as times, and earlier measurements keep their dates. The reflective rho calculus, Prime research lines, translation results and historical experiments retain their stated scopes and dates; their proofs and tests do not amount to a proof of the whole C implementation.

1 Introduction

MeTTa [1] is a reflective, typed, nondeterministic language designed as the core language of the OpenCog Hyperon framework. A MeTTa program is a space of atoms; equations among those atoms say how expressions rewrite; a query may have many answers, and the answers form a bag. Three families of implementation exist:

HE Hyperon Experimental, in Rust, the reference implementation with Python bindings and a large standard library (version 0.2.10 is the oracle used here).

PeTTa MeTTa compiled to Prolog clauses and run on SWI-Prolog, a dialect of its own with Prolog's search and indexing.

CeTTa This work: a direct C runtime, now at version 1.5.0-dev on its development lines, that runs the HE dialect, the PeTTa dialect (its default) and MeTTa Prime from one binary.

The dialects share a surface syntax and differ in meaning. HE leaves an expression it cannot rewrite as data; PeTTa can fail when a defined relation has no matching clause, while an unknown head remains uninterpreted; Prime evaluates by need, so an argument the body never demands never runs. A program written for one is not, in general, a program for another, and CeTTa treats each as a named semantic product with its own oracle: upstream HE for the HE lane, SWI-PeTTa for the PeTTa lane, and the Prime metatheory for Prime.

Reading the dates. The tutorial listings and the line-by-line development account retain their October snapshots. In particular, their references to staged work and open repairs describe those dates, not the current preview's release checklist. Section 3.3 describes the Linux embedding preview as of 5 November, and Appendix A gives fresh runtime measurements with binary and source identities. A usable preview is distinct from exhaustive qualification of every dialect, platform and resource-failure path.

What this paper is for. Earlier versions of this paper described CeTTa through its evidence. This version is first a guide. Section 2 teaches programming in CeTTa with listings run on all three lanes; Section 3 describes how the runtime is built; Section 4 how its semantics is kept honest; Section 5 what each line of development is for and what it has measured; Sections 6, 7 and 8 the Prime lines, the interaction basis for spaces and channels, and the Prime language. The remaining sections keep the paper's earlier results, each with its date: the verified HE–PeTTa translation, the reflective rho calculus, the benchmark ladder and its studies, the performance analysis, the dialect landscape, the observer-factored account of nondeterminism and state, the dependent-telescope extension, and the generated readers. The appendices record how every listing was run, and keep the LeaTTa cross-check, the abstract-machine design record and the distilled principles.

Contributions.

1. A tutorial of CeTTa's MeTTa in which every listing was run on a named binary, on the HE, PeTTa and Prime lanes, with the outputs quoted and the lanes' differences stated and explained: inert data against failure, argument types that decide evaluation, call by need, module destinations, the editable standard library with its `std:` originals, `transform`, and channels.

2. A specification-driven semantics: explicit profiles; result-bag conformance against upstream HE and SWI-PeTTa; a reflective modal layer over the one-step relation; a small-step rule table of ten live rule classes whose removal tests prove the evaluator depends on them, five of them certified in Lean.

3. A history of every line of development with its purpose and measured results, among them: on a 96-cell panel against SWI-PeTTa 1.0.6 (23 October 2026), CeTTa's PeTTa lane was clearly faster on 78 of 92 comparable cells and clearly slower on 8; on each of the five programs measured on 26 October, the native `transform` and its MeTTa library ran faster than SWI-PeTTa running the same library; space drains went from quadratic to linear.

4. Answer-checked measurements of the November preview (Appendix A): against standalone MORK, in its default build and with its leapfrog join, on MORK's own MM2 programs and on graph-analytics, query and search workloads; against SWI-PeTTa on all 164 PeTTa examples and on an inductive learner; against Hyperon Experimental on shared HE programs; and against the Python ai-mathematician's premise selection. Every timed run's answers are checked, and runs that fail or do not finish are reported as such.

5. The interaction basis as the direction for spaces and channels, with what is proved in Lean, what the Prime draft runs, and what the native runtime still lacks.

6. The earlier results, kept and dated: the HE↔PeTTa translation development (thirteen Lean modules with no `sorry` in their sources); the reflective rho calculus with an axiom-clean Lean characterization of evaluation at communication and a funded, receipt-producing cost profile; the cross-runtime benchmark ladder; the tail-safe evaluation-arena collector; the counted-PathMap backend and its faithful-backend obligations; generated readers; MeTTa Zero.

Conventions. A *lane* is a language as CeTTa runs it, selected by `--lang`. Every listing names its lanes, and Appendix B names the binaries by the first twelve hexadecimal digits of their SHA-256 digests. Statements about Prime carry their kind: [draft] for the behaviour of the C draft, [candidate] for a property of a candidate design, [direction] for a design direction with its reasons, [decided] for a recorded decision. Dates are of 2026. "Instructions" are user-mode instructions counted by `perf`.

2 Programming in CeTTa

This section teaches the language by running it. Every listing below was run on the binaries named in Appendix B, and every output panel is copied from that run. The same source is run on three *lanes*: CeTTa's HE lane (`--lang he`), its PeTTa lane (`--lang petta`, the default) and the Prime lane (`--lang prime`). The lanes are distinct dialects that share a surface syntax. Where a listing behaves differently on two lanes, the text says so and says why; nothing in this section assumes that a program written for one lane runs the same on another.

2.1 Running a program

One executable, `cetta`, hosts every language. The `--lang` option selects it:

```
cetta program.metta                    # PeTTa 1.0.6, the default
cetta --lang petta@1.1 program.metta   # PeTTa with static typing
cetta --lang he program.metta          # CeTTa's fork of Hyperon Experimental 0.2.10
cetta --lang he --profile extended program.metta
cetta --lang prime program.metta       # MeTTa Prime
cetta --lang rhocalc program.rho       # the reflective rho calculus
```

The binary also carries an MM2 front end, the MeTTa Zero kernel and its relatives, and a native HOL Light proof-stream checker; `cetta --list-languages` prints the inventory and `cetta --help` the options. The binary built from `main` (1.4.0-dev) defaults to the HE lane; PeTTa became the default on 17 October, in the 1.5 line.

A program is a sequence of atoms. An atom is a symbol (`red`), a variable (`$x`), a grounded value (a number or a string) or an expression, a parenthesized sequence of atoms. A top-level atom is added to the program's own space, `&self`; a top-level `!(...)` is a query, evaluated in place, whose answers are printed. The lanes print differently. HE and Prime print the answers of one query as

one bracketed list, `[a, b]`, and an effect that returns the unit value prints as `[()]`. PeTTa prints one line per answer and `true` for an effect. Comments run from `;` to the end of the line. The option `--pretty-vars` prints fresh variables by their written names; the listings use it.

2.2 Equations and evaluation

An equation `(= lhs rhs)` says that an atom matching `lhs` may be rewritten to `rhs`. Evaluating a call looks up the equations whose left side matches it, instantiates their right sides and evaluates those in turn. Grounded operations such as `+` and `*` compute directly.

Listing 1: Equations and evaluation; all three lanes

```
; Equations are atoms of the program space; a call rewrites by them.
(= (double $x) (* 2 $x))
(= (len ()) 0)
(= (len (Cons $h $t)) (+ 1 (len $t)))
(= (fact $n) (if (== $n 0) 1 (* $n (fact (- $n 1)))))
!(double 21)
!(len (Cons a (Cons b (Cons c ()))))
!(fact 20)
; a call no equation matches
!(len nothing)
```

HE lane

```
[42]
[3]
[2432902008176640000]
[(len nothing)]
```

PeTTa lane

```
42
3
2432902008176640000
```

Prime lane

```
[42]
[3]
[2432902008176640000]
[(len nothing)]
```

The three lanes agree on every computed answer. They disagree on the last query, and the disagreement is a real difference between the dialects. On the HE and Prime lanes an expression that no equation rewrites is its own normal form: `(len nothing)` stays as data. This is MeTTa's rule that what cannot be interpreted is left uninterpreted. On the PeTTa lane a function is compiled to Prolog clauses, and a call that no clause of a *defined* function accepts fails: it has no answer. SWI-PeTTa, the reference implementation of the dialect, gives the same output on this listing and on the six PeTTa-lane listings of Sections 2.3–2.7, up to the spelling of fresh variables (`$V0` against `$_0`).

2.3 Several answers, and the bag they form

A call may match several equations. Each match is a separate answer, and a query prints them all. `superpose` makes the answers of a list, and `collapse` does the reverse: it evaluates its argument completely and returns one expression holding every answer.

Listing 2: Several answers, and the bag they form

```
; Several equations for one call give several answers.
(= (color) red)
(= (color) green)
(= (color) blue)
!(color)
; collapse turns the answers into one expression: the bag, as a value
!(collapse (color))
!(let $x (superpose (1 2 3)) (* $x $x))
; duplicates are real occurrences
```

```
!(collapse (superpose (a b a)))
; an order-insensitive view of a bag
!(let $bag (collapse (superpose (c a b))) (sort-atom $bag))
; a head with no equations at all is data
!(undefined-head 1)
```

HE lane

```
[red, green, blue]
[(red green blue)]
[1, 4, 9]
[(a b a)]
[(a b c)]
[(undefined-head 1)]
```

PeTTa lane

```
red
green
blue
(red green blue)
1
4
9
(a b a)
(a b c)
(undefined-head 1)
```

Prime lane

```
[red, green, blue]
[(red green blue)]
[1, 4, 9]
[(a b a)]
[(a b c)]
[(undefined-head 1)]
```

The collection of answers is a *bag*: a multiset, in which duplicates are real occurrences (`(a b a)` keeps both copies of `a`) and the order in which an engine happens to find the answers carries no meaning. A printed result is necessarily a sequence, but programs and tests should not depend on it; the conformance gates of Section 4.3 compare bags. When a program wants a canonical sequence, it sorts the bag explicitly, as the fifth query does. A program written for one engine may still exploit that engine's documented order; it then says so by being engine-specific.

The fifth query binds the bag with `let` before sorting it, and this matters on the HE lane. There, a declared argument type of `Expression` or `Atom` tells the evaluator *not* to evaluate that argument, and `sort-atom` is declared `(-> Expression Expression)`:

Listing 3: Argument types decide evaluation on the HE lane

```
; sort-atom is declared (-> Expression Expression) on the HE lane
!(get-type sort-atom)
!(sort-atom (c a b))
; written directly, the argument is not evaluated first
!(sort-atom (collapse (superpose (c a b))))
; bound with let first, the bag is sorted
!(let $bag (collapse (superpose (c a b))) (sort-atom $bag))
```

HE lane

```
[(-> Expression Expression)]
[(a b c)]
[(c a b)]
[(a b c)]
```

PeTTa lane

```
%Undefined%
(a b c)
(a b c)
(a b c)
```

Prime lane

```
[(-> Expression Expression)]
[(a b c)]
[(a b c)]
[(a b c)]
```

Written directly, `(sort-atom (collapse ...))` sorts the *unevaluated* expression `(collapse (superpose (c a b)))` and only then evaluates the result, so the bag comes out unsorted. On the PeTTa and Prime lanes `sort-atom` evaluates its argument first, although Prime declares the same type. On the HE lane the type of a function decides which of its arguments are evaluated, and `let` is the portable way to force a value before a call. A user-defined function with a parameter typed `Atom` receives that argument unevaluated on the HE and PeTTa lanes alike (SWI-PeTTa agrees), but not on the Prime lane (Section 8).

2.4 Demand and call-time choice

The HE and PeTTa lanes are eager: arguments are evaluated before a function's equations are tried. Prime evaluates by need: an argument is evaluated only when the chosen equation demands it, and then at most once.

Listing 4: Demand and call-time choice

```
(= (const $x $y) $x)
(= (pair $x) ($x $x))
; is an argument the body never needs evaluated?
!(const 5 (println! "argument evaluated"))
; one choice, used twice
!(pair (superpose (1 2)))
```

HE lane

```
argument evaluated
[5]
[(1 1), (2 2)]
```

PeTTa lane

```
"argument evaluated"
5
(1 1)
(2 2)
```

Prime lane

```
[5]
[(1 1), (2 2)]
```

`const` never uses its second argument, so on the Prime lane the `println!` never runs; on the eager lanes it runs before `const` is entered. The second query shows *call-time choice*: the argument `(superpose (1 2))` is one choice, shared by both uses of `$x`, so the answers are `(1 1)` and `(2 2)`, never `(1 2)`. The eager lanes give the same answers for a different reason: they choose before the call. Under call by need the sharing is the semantics: a delayed argument is one *need cell*, and every use of it sees the same value (Section 8).

2.5 Spaces: `match`, `add-atom`, `remove-atom`

A *space* is a store of atoms. `new-space` makes one and `bind!` gives it a name; `add-atom` and `remove-atom` change it; `(match space pattern template)` answers one instance of the template for each stored atom that matches the pattern. A comma pattern `(, p1 p2)` is a conjunction: both patterns must match, with shared variables bound alike. The program space is a space like any other, and its equations are atoms in it.

Listing 5: Spaces, queries and updates

```
!(bind! &kb (new-space))
!(add-atom &kb (parent tom bob))
!(add-atom &kb (parent bob ann))
!(add-atom &kb (parent bob jim))
; match: one answer per matching occurrence
!(match &kb (parent bob $c) $c)
; a conjunctive query
!(match &kb (, (parent $g $p) (parent $p $c)) (grandparent $g $c))
(= (children $p) (collapse (match &kb (parent $p $c) $c)))
!(children bob)
!(remove-atom &kb (parent bob jim))
!(children bob)
; the program space is a space too: equations are atoms in it
(= (double $x) (* 2 $x))
!(match &self (= (double $x) $body) $body)
```

HE lane

```
[()]
[()]
[()]
[()]
[ann, jim]
[(grandparent tom ann), (
    grandparent tom jim)]
[(ann jim)]
[()]
[(ann)]
[(* 2 $x)]
```

PeTTa lane

```
true
true
true
ann
jim
(grandparent tom ann)
(grandparent tom jim)
(ann jim)
true
(ann)
(* 2 $V0)
```

Prime lane

```
[()]
[()]
[()]
[()]
[ann, jim]
[(grandparent tom ann), (
    grandparent tom jim)]
[(bob ann jim)]
[()]
[(bob ann)]
[(* 2 $x)]
```

The lanes agree, with one exception that is a defect of the current Prime draft, not a property of Prime. On the Prime lane, `(children bob)` answers `(bob ann jim)`: inside `match`, the pattern variable `$p` is treated as a fresh variable even though the call has bound it to `bob`, so the query matches every parent. The same probe gives the same answer on both draft binaries of Appendix B. The deviation was found by the interaction study of Section 7 and is listed among the draft's open repairs in Section 8.4 [draft]; the interaction form `transform` respects outer bindings.

2.6 Types

A type declaration `(: atom type)` is itself an atom of the space. Arrow types `(-> A B C)` type functions. What a lane does with a declaration differs more than anything else in this section.

Listing 6: Declared types and an ill-typed call

```
(: Nat Type)
(: Z Nat)
(: S (-> Nat Nat))
(: add (-> Nat Nat Nat))
(= (add Z $y) $y)
(= (add (S $x) $y) (S (add $x $y)))
!(add (S Z) (S (S Z)))
!(get-type (S Z))
!(get-type add)
; an argument of the wrong type
!(add (S Z) "two")
```

HE lane

```
[(S (S (S Z)))]
[Nat]
[(-> Nat Nat Nat)]
[(Error (add (S Z) "two") (
    BadArgType 2 Nat String))]
```

PeTTa lane

```
(S (S (S Z)))
Nat
(-> Nat Nat Nat)
```

Prime lane

```
[(S (S (S Z)))]
[Nat]
[(-> Nat Nat Nat)]
[(Error (add (S Z) "two") (
    BadArgType 2 Nat String))]
```

The HE and Prime lanes check the call and answer an error value, `(Error ...(BadArgType 2 Nat String))`, which is ordinary data that a program can match. The default PeTTa lane checks argument types when the call runs and makes an ill-typed call fail, so the last query has no answer; SWI-PeTTa agrees. The typed PeTTa 1.1 lane checks the whole program before running any of it and rejects it:

PeTTa 1.1 lane

```
PeTTa type error: Type mismatch: call arguments conflict with inferred type for add
```

Two further typed products exist on the PeTTa side, the `typecheck-v2` and `typecheck-v3` profiles of Section 4.2. Prime's typing is its own and never defers to HE's type engine [decided]; Section 8

describes its faces.

2.7 Modules and imports

`import!` loads a file of atoms. With `&self` as its destination the atoms join the program space. A module name is resolved relative to the importing file. Libraries shipped with the binary are named `(library name)` on the PeTTa lane, and by their module name on the HE and Prime lanes (`!(import! &self lts:he)`, `!(import! &self interaction)`).

Listing 7: Importing a module file

```
; load a module file into the program space
!(import! &self lib/geometry)
!(square 7)
!(area (rect 3 4))
```

HE lane

```
[()]
[49]
[12]
```

PeTTa lane

```
true
49
12
```

Prime lane

```
[()]
[49]
[12]
```

A module can also be loaded into a space of its own and queried there. On the HE and Prime lanes the destination must then be `&self` or a name not yet bound; `import!` creates the space. Importing into a space bound earlier with `bind!` is refused with an error value. The PeTTa lane accepts either form.

Listing 8: Importing a module into a space of its own

```
; load a module into a fresh space of its own, then query it
!(import! &geo lib/geometry)
!(match &geo (= (square $x) $body) $body)
```

HE lane

```
[()]
[(* $x $x)]
```

PeTTa lane

```
true
(* $V0 $V0)
```

Prime lane

```
[()]
[(* $x $x)]
```

2.8 The standard library, and redefining it

Every lane ships a standard library: list and expression operations (`car-atom`, `cdr-atom`, `size-atom`, `sort-atom`, `foldl-atom`), numeric and string operations, space operations, and control (`if`, `case`, `let`, `let*`, `collapse`, `superpose`). It is precompiled into the binary, so a program starts without parsing it: a one-line query completes in under ten milliseconds at about 14 MB of peak memory on the HE and PeTTa lanes (single runs).

What happens when a program defines an equation for a library name differs by lane. On the HE lane the grounded operation wins and the program's equation is never consulted. On the PeTTa lane, as in SWI-PeTTa, the library function is a static Prolog procedure, and the program's equation is refused with a permission error:

Listing 9: Adding an equation for a builtin on the HE and PeTTa lanes

```
(= (size-atom $x) many)
!(collapse (size-atom (a b c)))
!(std:size-atom (a b c))
```

HE lane

```
[(3)]
[(std:size-atom (a b c))]
```

PeTTa lane

```
error: (Error (permission_error modify
    static_procedure (/ size-atom 2)) (context
    (: system (/ assertz 2)) $_))
```

The Prime draft makes the standard definitions ordinary, editable occurrences of the program space and keeps the originals under the `std:` prefix [draft], since 26 October 2026. An added equation is one more alternative; removing the standard rows removes their behaviour; `(std:definitions name)` returns the original declarations and equations as data, and re-adding them restores the original. A qualified call such as `std:size-atom` always reaches the original implementation:

Listing 10: Editing a standard definition on the Prime lane

```
; Standard definitions are ordinary, editable occurrences of the program
; space; std: names keep the originals.
!(size-atom (a b c))
!(std:size-atom (a b c))
!(collapse (std:definitions size-atom))
; add an alternative: a second answer for every call
(= (size-atom $x) many)
!(collapse (size-atom (a b c)))
!(std:size-atom (a b c))
; remove the standard rows: only the program's equation is left
!(let $d (std:definitions size-atom) (remove-atom &self $d))
!(collapse (size-atom (a b c)))
; restore them
!(let $d (std:definitions size-atom) (add-atom &self $d))
!(collapse (size-atom (a b c)))
```

Prime lane

```
[3]
[3]
[((: size-atom (-> Expression Number)) (= (size-atom $argument0) (std:size-atom $argument0)))]
[(3 many)]
[3]
[(), ()]
[(many)]
[(), ()]
[(many 3)]
```

After the restoration the bag is `(many 3)`: the program's equation now precedes the restored rows, and occurrence order is observable in the printed sequence, though not in the bag. The native implementation of a standard operation is used only while the definitions it stands for are unchanged; an edited operation goes through ordinary equation evaluation.

2.9 `transform`: one atomic step over a space

`match` reads and `add-atom`/`remove-atom` write, one at a time. A program that must consume a job exactly once, or move money between two accounts, needs to read, consume and produce in one indivisible step. `transform` is that step. Its sources are patterns, each marked by what the step does with the matched occurrence:

- a plain pattern or `(read p)` *keeps* it (reads it and leaves it);
- `(take p)` *takes* it (consumes it);

- `(absent p)`, `(unique p)` and `(all p P)` are *predicate queries*: they look at the whole space (no match, exactly one, every match) and dispose of nothing;
- `(every c)` fires every match of `c` in the same step;
- `(let P E)` evaluates `E` once the sources have bound its variables, and contributes no row when it has no value.

The template is instantiated once per fired row. With a destination space as fourth argument the instances are added there and the answer is how many; with none they are answered to the caller. On the PeTTa lane the operation is a library, `lib_transform`, written in MeTTa, and the binary fires it natively when the library is the unmodified one:

Listing 11: `transform` on the PeTTa lane

```
!(import! &self (library lib_transform))
!(bind! &s (new-space))
!(add-atom &s (job 1))
!(add-atom &s (job 2))
!(add-atom &s (rate 3))
; take one job and keep the rate, in one step
!(transform &s (, (rate $r) (take (job $n))) (priced $n $r))
; every: all remaining matches fire in this one commit
!(transform &s (every (take (job $n))) (done $n))
!(collapse (match &s (job $n) $n))
; absent: a predicate query; here a closure that adds each path once
!(bind! &g (new-space))
!(add-atom &g (edge a b))
!(add-atom &g (edge b c))
!(transform &g (every (, (edge $x $y) (absent (path $x $y)))) (path $x $y) &g)
!(transform &g (every (, (path $x $y) (edge $y $z) (absent (path $x $z)))) (path $x $z) &
    g)
!(transform &g (every (, (path $x $y) (edge $y $z) (absent (path $x $z)))) (path $x $z) &
    g)
!(sort-atom (collapse (match &g (path $x $y) ($x $y))))
```

PeTTa lane

```
true
true
true
true
(priced 1 3)
(done 2)
()
true
true
2
1
0
((a b) (a c) (b c))
```

PeTTa lane, authored firing

```
true
true
true
true
(priced 1 3)
(done 2)
()
true
true
2
1
0
((a b) (a c) (b c))
```

SWI-PeTTa

```
true
true
true
true
(priced 1 3)
(done 2)
()
true
true
2
1
0
((a b) (a c) (b c))
```

The first step keeps the rate and takes job 1; the `every` step takes every remaining job; the closure adds each path once, guarded by `absent`, and the third round adds nothing. The native firing, the authored firing in the library (forced with an environment switch), and SWI-PeTTa running the same library file all leave the same answers and the same space. Section 5.2 describes how the native realization was built and measured.

2.10 Processes and channels

One interaction form on the Prime lane. On the Prime lane the draft implements the interaction form of Section 7 as a native kernel with a MeTTa library, imported as the module `interaction` [draft]. A step reads, keeps, takes and makes in one atomic *commit*, across any number of spaces. *Regions* made by `new-region` carry a profile (space, channel, mailbox, log or board) that fixes which of the four parts each holder may perform; a commit outside its holder's rights is not formed. `await` parks a step until it can fire and fires it once; `whenever` installs it as a rule that fires on every enabling; Rholang's receipts (`for` with `<-`, `<=`, `«-`) are aliases of these.

Listing 12: One interaction form on the Prime lane: commits, regions, waiting

```
!(import! &self interaction)
!(bind! &s (new-space))
!(add-atom &s (job 1))
!(add-atom &s (rate 3))
; one commit: keep the rate, take the job, make the result
!(transform &s (, (read (rate $r)) (take (job $n))) (priced $n $r))
; a row never reads and takes the same occurrence
!(add-atom &s Token)
!(transform &s (, (read Token) (take Token)) committed)
!(add-atom &s Token)
!(transform &s (, (read Token) (take Token)) committed)
; regions: a channel has no predicate queries, for anyone
!(let $c (new-region channel) (rights-of $c))
!(let $c (new-region channel) (transform $c (absent (msg $x)) empty))
; waiting: await parks until the commit can fire, then fires once
!(bind! &q (new-space))
!(bind! &out (new-space))
!(await (transform &q (take (job $x)) (add-atom &out (got $x))))
!(transform &q () (, (add-atom &q (job 1)) (add-atom &q (job 2))))
!(collapse (match &out (got $x) $x))
; whenever: a rule that fires on every enabling
!(whenever (transform &q (take (job $x)) (add-atom &out (served $x))))
!(transform &q () (add-atom &q (job 3)))
!(sort-atom (collapse (match &out (served $x) $x)))
; Rholang's receipt, as an alias of await over a take
!(bind! &c2 (new-space))
!(for ((msg $m) <- &c2) (add-atom &out (heard $m)))
!(transform &c2 () (add-atom &c2 (msg hello)))
!(collapse (match &out (heard $m) $m))
```

Prime lane

```
[()]
[()]
[()]
[()]
[(priced 1 3)]
[()]
[()]
[committed]
[(receive peek send)]
[(Error transform (not-formed (missing-right observe (Sealed region))))]
[()]
[()]
[()]
[()]
[(1)]
[()]
[()]
[(2 3)]
[()]
[()]
[()]
[(hello)]
```

The first commit keeps the rate and takes the job. The two `(read Token)` `(take Token)` steps show *identification*: one row may not keep and take the same occurrence, so with one token nothing commits (the step answers `()`), and with two it does. A channel region has no predicate queries, so `absent` on it is not formed, whatever the holder. The `await` fires once, on job 1; the rule installed by `whenever` serves the remaining job 2 and then job 3 as it arrives; and the receipt written with `for` is woken by the send. These are the semantics of the interaction study, run on the draft binary; the trusted part is a small kernel, and the meaning is stated beside it in the library.

The reflective rho calculus. `--lang rhocalc` runs the reflective higher-order rho calculus of Section 10. A process sends on a name with `x!(P)` and receives with `for ($y <- x) {Q}`; `@{P}` quotes a process into a name and `*$y` runs the process a name quotes. The one reduction, COMM, consumes a send and a receive on the same name. When the message is itself code and the receiver runs it, a communication is an evaluation step:

Listing 13: Evaluation at COMM in the strict-core rho calculus

```
// Eval at COMM: running code you were sent, with no execution primitive.
//
// The rho-calculus has no "run" instruction.  A quoted process is inert
// data -- until it is communicated.  The runner idiom makes execution cost
// exactly one handshake:
//
//     run!(P) | for ($y <- run) { *$y }   --COMM-->   P
//
// The send carries the program P as data (the engine quotes it on the
// wire); the receiver drops (runs) whatever arrives.  Channels are quoted
// processes, so run = @{0} and out = @{@{0}!(0)} are two distinct names.
//
// Here P = out!(0) is itself a send that completes a second handshake with
// a receiver already waiting on out.  The whole program settles at 0 only
// if the delivered code actually ran: first COMM delivers P, the drop
```

```
// activates it, and the second COMM consumes it.
//
// Run: ./cetta --lang rhocalc examples/rho/pure/eval_at_comm.rho
// Expected: 0

@{0}!(@{@{0}!(0)}!(0)) |
for ($y <- @{0}) {*$y} |
for ($r <- @{@{0}!(0)}) {0}
```

```
--lang rhocalc
```

```
0
```

The program settles at `0` only if the delivered code ran: the first COMM delivers the send `@{@{0}!(0)}!(0)`, the drop activates it, and the second COMM consumes it. Without the last receiver the activated send is left over:

Listing 14: The same program without the last receiver

```
@{0}!(@{@{0}!(0)}!(0)) |
for ($y <- @{0}) {*$y}
```

```
--lang rhocalc
```

```
@{@{0}!(0)}!(0)
```

2.11 What the listings show about the lanes

Behaviour	HE lane	PeTTa lane	Prime lane
call no equation of a defined head matches	inert data	no answer	inert data
parameter typed `Atom`	held	held	evaluated; `(Data Atom)` holds [draft]
unneeded argument	evaluated	evaluated	never evaluated
ill-typed call	error value	no answer (1.1: program rejected)	error value
`import!` into a bound space	refused	accepted	refused
equation for a library name	ignored	refused	one more alternative; `std:` keeps the original
outer variable inside a `match` pattern	bound	bound	fresh [draft], a defect
`transform`	absent	`lib_transform`, native firing	interaction kernel [draft]

Each row is a listing or a probe above, run on the named binaries. Wherever SWI-PeTTa was run, it behaves as the PeTTa column says; its output is identical up to the spelling of fresh variables, except that it reports the refused library equation as an error of its loader.

3 CeTTa Architecture

CeTTa is a direct evaluator, not a compiler or boot pipeline: every lane evaluates source atoms in place, and compiled fragments are optimizations that must give the evaluator's answers. The HE lane implements the six mutual functions of the HE evaluation specification (`EvalAtom`, `MettaCall`, `InterpretExpression`, `InterpretFunction`, `InterpretArgs`, `InterpretTuple`) as C functions

over shared arenas; the PeTTa lane runs a heap-based choice and continuation machine; the Prime lane adds need cells and an exact outcome carrier over the same term and space substrate.

3.1 Parts of the implementation

The runtime core implements atoms, symbols, bindings, arenas, term identity, spaces, matching, evaluation and retained execution state. Native modules provide parsers and language services, regular expressions, tensors, files, processes and proof checkers. Optional Rust bridges connect to PathMap and MORK; they are separate from the C/Python embedding runtime's dependencies. The PeTTa search machine, prepared fragments and shared query machinery are implementation routes through these services, not separate language meanings. The source tree and named build configurations identify which optional providers a particular executable contains.

3.2 Core design

Arenas. Atoms are allocated from arenas of 64 KB blocks by default, with optional block growth, recycling of retired blocks and a per-arena cache of canonical scalar atoms. There is no per-atom `malloc` or `free`; scratch is released in bulk at scope exit, per branch, and per iteration of a deterministic tail loop.

Interning. Symbols and terms intern into a term universe; equality on the hot path compares identifiers. The interning table uses a hash-derived odd stride, so every probe sequence visits the whole table (Appendix I).

Equation index. Head-symbol and structural indexes select candidate equations and facts; revision-aware compiled views reuse their discrimination work. Shared binding and substitution machinery avoids per-candidate variable copying.

Tail calls and fuel. A trampoline re-enters tail calls without growing the C stack. Fuel is unlimited by default; a stack overflow is caught on an alternate signal stack and reported.

Precompiled standard library. A stage-0 build compiles the standard library into a binary blob that the final binary links, so programs start without parsing it.

Compiled fragments. Several admitted fragments run outside the general evaluator, each with a conservative fallback. `PreparedPureProgram` admits deterministic, effect-free closed equation fragments: naive `fib 30` runs in 0.04 s on the pre-main HE lane, and the runtime counters show one admission and one commit of the prepared route. `MatchDecision` compiles clause discrimination under revision-pinned identity; `RuleMachineCoreV1` runs bounded admitted rule packages.

Named build products. Each test and probe binary is named by its full build configuration (sanitizers, statistics, bridge, Prolog and Python linkage), so differently configured products never overwrite one another.

3.3 Linux embedding preview: Python and the C ABI

The November preview exposes the existing evaluator through a versioned C header, `include/cetta_embed.h`, and a thin CPython extension. The standalone C library and the Python package share the embedding core. The Python package's runtime does not require SWI-Prolog, Rust or C++; the CLI's optional providers and bridge builds have their own dependencies.

From a source checkout, the ordinary installation and first call are:

```
python3 -m pip install .
python3 -m pip install '.[numpy]'     # optional numerical interface
```

```
from cetta import Engine

with Engine(language="petta") as engine:
    engine.run("(= (square $x) (* $x $x))")
    assert engine.call("square", 7).to_python() == [49]

with Engine(language="he") as engine:
    assert engine.eval("(+ 20 22)").to_python() == [42]
```

Whole-result calls return retained answer bags; queries also provide a closable stream and optional bounded physical stepping. Structured atoms keep symbols distinct from strings and preserve variable identity and duplicate answers. Independent engines have separate mutable runtime contexts. Python callbacks, output capture and interruption use host hooks; embedding does not initialize or finalize its host Python interpreter. Hosting the engine in Python is a separate surface from importing Python functions inside a MeTTa program; the latter retains its provider-specific compatibility requirements.

Dense and sparse numerical exchange retains buffer owners. Immutable native results can back read-only NumPy views; mutable inputs copy when necessary for stable values, and an incompatible explicit no-copy request is rejected. The opt-in `tensor:dot-bounds` and `tensor:matvec-bounds` operations return a value with an error enclosure under their admitted floating-point conditions. These are bounds on the specified linear arithmetic, not a certificate for an entire neural likelihood or for arbitrary `exp` and `log` implementations.

The C SDK installs with `make install-cabi PREFIX=...`; it includes the public header, shared and static libraries, resources and `cetta.pc`. See `docs/python_embedding.txt` and the ordinary Python notebook `examples/python/getting_started.ipynb` in the CeTTa repository. The current local package was checked on CPython 3.13; the earlier portable matrix covers CPython 3.11–3.15 on glibc and musl, on x86_64 and emulated AArch64, and is not a measurement of this exact November source snapshot. Physical AArch64 remains unverified. The engine's memory limit accounts for CeTTa-owned allocations, excluding GMP and borrowed buffers; an allocation failure inside GMP can still abort the host process.

3.4 Tests and gates

The test manifest records each fixture's language, profile and observation policy; dedicated targets cover the runtime boundaries, language references, space backends, proof checkers and host adapters. A readiness run on the measured preview state (5 November) recorded 524 passing, no failing and 78 skipped cases of `make test`; it also left two decisions about the PeTTa reference open (the version that decides whether a function added at run time is visible to code loaded before it, and which stable PeTTa commit is the model). Those are specific observations of the identified preview, not a claim that every optional mode or failure path has been qualified. Historical gate receipts below retain their original dates.

For the November preview, `make main-readiness` is a bounded candidate check with a default twenty-minute budget including builds. It runs the ordinary suite, selected PeTTa/HE and lifetime controls, and the Python tests, preserving passing checks for reuse on matching inputs. A timeout reports incomplete. Focused fixes use the affected target; the broad sanitizer, performance and

exhaustive campaigns remain explicit qualification lanes rather than being restarted after each small repair.

4 Specification-Driven Semantics: Profiles, Conformance, and the Small-Step Rule Table

CeTTa is organized around an explicit principle: *the semantics is a named object, not an emergent property of the implementation.* This section describes the profile family, the oracle-conformance campaign against upstream HE, the evaluator bug classes that campaign exposed, the reflective modal layer over the one-step reduction relation, and the first welded rule pack that makes the reduction rules themselves runtime data.

4.1 The Profile Family

Profiles are distinct semantic products, implemented as additive layers—never as visibility masks over one shared universe:

`he-compat` A literal mirror of Rust HE v0.2.10, including its quirks: integer division for `/`, the strict three-argument `if` form, Rust's library inventory behavior. Compatibility means matching upstream *behavior*, not reproducing its library set; library availability is out of scope for conformance.

`he` The principled line. Spec-faithful where the written MeTTa specification speaks; where HE's implementation and its specification diverge, the divergence is adjudicated and recorded per-case. Carries exact arbitrary-precision rationals, true division for `/`, floor division `//`, Euclidean `math:mod`, and the clean `math:` namespace over the Rust-heritage `*-math` ops. Extensions that accept more programs (e.g. two-argument `if`) are deliberate, recorded semantic choices.

`he-extended` `he` plus labeled CeTTa extensions: typed space kinds, `with-space-snapshot`, the `lts` reflection layer, MORK/PathMap-backed spaces.

`he-prime` The dependent-telescope extension (Section 15).

`petta/typecheck-v2` The PeTTa dialect with the native, Roman-compatible residual type checker enabled.

`petta/typecheck-v3` A native refinement of v2's success-typing core, with whole-block fallback to v2 for mechanisms outside the factored calculus. The ordinary `--lang petta` profile remains unchanged; typing is an explicit opt-in semantic product, not a hidden mode of evaluation.

`petta@1.1` The PeTTa 1.1 language, typed: the whole program is checked before any of it runs, and an ill-typed call rejects the program (Section 2.6). The default `--lang petta` is PeTTa 1.0.6.

Two rules keep this family honest. First, unknown symbols stay inert: MeTTa's philosophy is to leave uninterpreted what it does not know how to interpret, so no profile ever raises an "unsupported feature" error—a profile determines what is *interpreted*, never what is *legal*. Second, profile differences are explicit splits with named tests (e.g. `profile_if_compat_arity` pins that `he-compat` rejects two-argument `if` exactly as upstream does, while `he`/`he-extended` accept it as a recorded extension).

4.2 The PeTTa Typecheck Profile and Its Native Refinement

The operational profile. PeTTa's type checker is exposed as an explicit profile:

```
cetta --lang petta --profile typecheck-v2 --silent program.metta
cetta --lang petta --profile typecheck-v3 --silent program.metta
```

The profile performs *success typing*: it rejects a definition only when static evidence identifies a concrete incompatibility, while incomplete search and missing information remain distinct from a negative verdict. It is residual in the practical sense that the checker reasons about the values and clause-selection behavior that may remain after PeTTa reduction, rather than treating a type annotation as a nominal label. The compatibility implementation is qualified against a fixed Roman-checker snapshot, but that snapshot is a measuring instrument only; new implementation and migration work targets the current upstream language.

The typing profile is isolated in both directions. Running ordinary `--lang petta` does not activate its operations or pay its admission boundary, and a build may omit the v2 checker physically. Conversely, the typed profile obtains declarations and expression facts from the live program revision rather than copying a second user-visible type space. At the 2026-08-14 qualification snapshot, the operational v2 corpus contains 389 oracle-classified cases: 164 acceptances and 225 clean rejections, all in agreement.

From a large checker to a small theory. The native v3 work did not transliterate thousands of lines of checker code. Instead, each observed v2 decision was reduced to a minimal witness, grouped by mechanism, and factored into four independent axes. The resulting calculus was formalized first and only then mirrored into the executable LanguageDef.

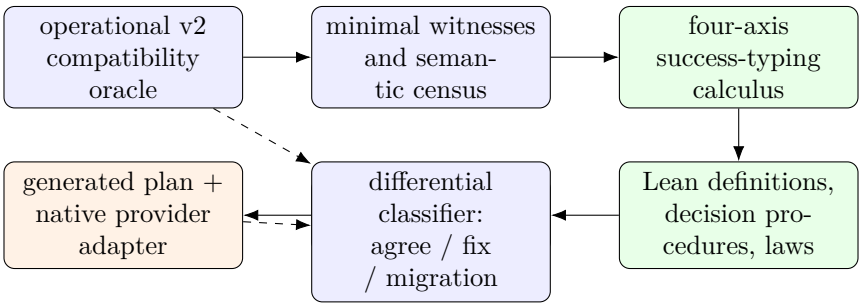


Figure 1: Distillation, not cloning. The compatibility checker supplies observations; the v3 implementation is generated from a factored calculus. V2 is never used as a v3 fact interface: the two meet only at a compatibility or differential boundary.

The four axes are:

Axis	Native object	Governing distinction
Shape compatibility	directional consistency on primitive, list, product, union, arrow, alias, and nominal-newtype shapes	an actual union is universal; a required union is existential; unknown is compatible ignorance, not a proof of equality
Result cardinality	the grade order det $\leq$ semidet $\leq$ nondet and grade sequencing	a deterministic promise cannot be discharged by a computation that may fail or branch; effect variables remain polymorphic
Stage evidence	held, evaluated, and compiled evidence, with an explicit grounding license	quoted code does not masquerade as its executed value; grounding requires compiled-stage evidence
Open-world knowledge	unknown, empty, or positively typed result evidence, plus selector facts	ignorance yields *undetermined*; exhaustion yields *incomplete*; only a replayable conflict yields *refuted*

The first row explains an initially surprising polarity law. If a value is known to be a `Number`, a requirement of `(| Number String)` is satisfied by one member. If the value itself may be either `Number` or `String`, a `Number`-only requirement is unsafe because *every* actual alternative must be admissible. Thus consistency is deliberately directional and, in the presence of unknown information, deliberately non-transitive.

A source-level example. The following selector is total on proper lists. The identity `let*` and its administrative alias must preserve both the `List Number` shape and the deterministic grade:

```
(: keep-number (-[det]-> Number Number))
(= (keep-number $x) $x)

(: select-list
   (-[$effect]-> (List Number)
                 (-[$effect]-> Number Number)
                 Number))
(= (select-list () $k) ($k 0))
(= (select-list (cons $head $tail) $k) ($k $head))

(: caller (-[det]-> Number))
(= (caller)
   (select-list
      (let* (($source (cons 1 ()))
             ($items  $source))
        $items)
      keep-number))
```

v3 accepts `caller`: lexical names elaborate to de Bruijn indices, alias resolution transports the original runtime evidence, and the selector consumes the positive proper-list fact. Replacing `(cons 1 ())` by `1` is the negative mirror: the alias still transports evidence exactly, so the call is rejected at the named shape boundary rather than accidentally acquiring a list fact. Replacing the producer by a nondeterministic superposition of Booleans similarly preserves shape but fails a deterministic cardinality promise. These paired examples ensure that "transport" means preservation, not widening.

Stage evidence supplies a second compact example:

```
(: Count (Newtype Number))

(: held-count (-[det]-> Count))
(= (held-count) (quote (brand Count 1)))      ; rejected: held code

(: evaluated-count (-[det]-> Count))
(= (evaluated-count)
   (eval (quote (brand Count 1))))            ; accepted: reactivated
```

Quotation preserves latent evidence but does not expose it as runtime typing evidence. Only the explicit `eval` transition reactivates it. Native v3 therefore rejects `held-count` and establishes `evaluated-count`; the latter is recorded as a named refinement and continues to follow v2 until that refinement is explicitly promoted. This is why the calculus can make the historical source-grounding defect unrepresentable: source shape alone cannot construct a grounding license.

Runtime authority and status. The executable checker sees program state through exactly five revision-keyed fact interfaces:

```
EnvDeclared               EnvDeclaredList
EnvNonNewtype             KnownExpressionEffect
KnownExpressionResultType
```

They derive facts from the live PeTTa program; they do not call v2 to manufacture v3 answers.

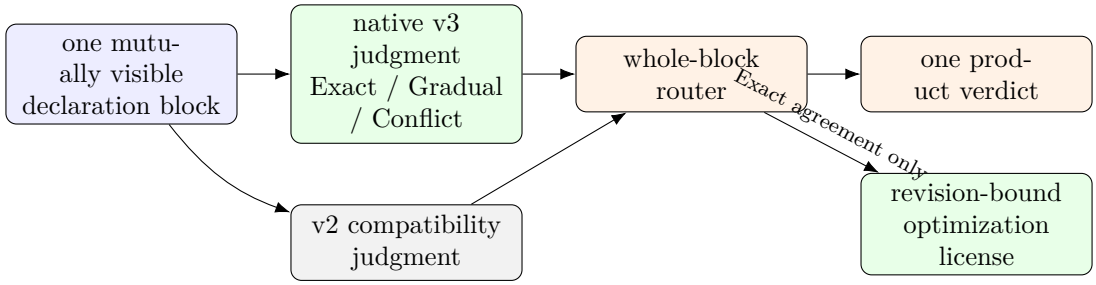


Figure 2: The KISS compatibility waist. Routing is per complete block, so a recursive definition group never mixes authorities. A gradual result, conflict, unpromoted refinement, or legacy route cannot mint an optimization license.

At the same 2026-08-14 snapshot, the Lean core contains the executable sound-and-complete consistency decision procedure, the cardinality and stage algebras, lexical evidence transport, products, aliases, rational-tree boundedness, and the Exact/Gradual/Conflict seam, with no holes. Its ordered LanguageDef mirror contains 149 rules (134 core and 15 seam rules) and passes three order/multiplicity mutation canaries. The isolated native harness passes 353/353 checks, including product canaries that distinguish exact agreement from rejection and gradual fallback. All 51 census-selected source candidates are executable end to end. In particular, v3 derives undeclared result/effect contracts from the revision-keyed live equation set, retains heterogeneous results as precise unions, and distinguishes proved concrete selection from unresolved open patterns.

The complete 389-case differential classifies 214 cases as native agreement, four as named refinements, and 171 as whole-block legacy-v2, with zero unclassified, incomplete, or faulted cases. Current-upstream PeTTaChainer qualification passes all three representative programs after small,

semantics-preserving migrations that replace effect-polymorphic traversal helpers with explicit typed recursion. The no-typecheck profile and v3 both match the PeTTa reference on those programs.

The optional v3 profile is not yet free. Three matched runs on the same materialized programs give the following medians; v3 includes both admission and ordinary execution:

Program	no typing	typecheck-v3	wall ratio	RSS ratio
`deductionrevision`	1.49 s	4.59 s	3.08×	2.41×
`flyingraven`	6.07 s	12.10 s	1.99×	1.59×
`raveninduction`	1.97 s	5.26 s	2.67×	1.98×

This is an integration baseline, not an optimization claim. Typed fast paths remain disabled until an exact, current license is consumed at an appropriate construction boundary. These denominators are intentionally separate: operational compatibility, theory/LanguageDef parity, native coverage, product migration, and execution cost are different claims.

4.3 Oracle Conformance

For `he-compat`, upstream HE v0.2.10 is the sole oracle, and the harness is a fail-by-default gate, not a green-manufacturing report. Comparison is by *result-bag equality*—multisets of parsed results, never byte-equality of output text—so formatting differences cannot masquerade as semantic agreement, and duplicate results (which are semantically real in MeTTa) are counted.

Current HE I/O semantic gate. The per-build oracle gate (`make test-he-compat-semantic-suite`) replays 23 curated HE I/O fixtures against the upstream HE v0.2.10 baseline. The July 2 receipt reports 23/23 CeTTa passes, 0 failures, and a passing baseline check. Six of those fixtures are core anchors into the Lean-side HE I/O conformance file. This is the current conformance claim used by the paper.

Archived catalog material. Earlier generated rule-capacity and broad-corpus catalogs remain useful triage artifacts, but they are not treated as current paper evidence unless their driver emits the claimed counts on the audited tree. In particular, this paper does not carry a 24-capacity maturity table or a broad-corpus count as a live conformance result.

Adjudicated divergences. Where CeTTa deliberately diverges from upstream, each case carries independent grounds. Three examples: upstream's post-0.2.10 `function`/`NoReturn` behavior contradicts its own written specification (a deliberate loop fix that left the document stale); upstream's `get-doc` errors on HE's own documentation example (an arity regression); and the user-visible result of top-level `eval` on a non-reducible term is genuinely under-determined by the specification—both engines deviate from the literal instruction-layer reading, in opposite directions.

Conformance is *testing*, and the specification is what gets *proven about* in Lean: a per-run check is never presented as a proof. Where the runtime does emit evidence (the Prime need trace, the commit traces of the interaction kernel, the receipts of the rho cost profile), an independent checker replays it; the runtime's own record is never the authority.

4.4 Plain PeTTa: Semantic Decisions

For plain PeTTa, SWI-PeTTa at a pinned revision is the oracle, and comparison is again by result bags. Where SWI-PeTTa is not consistent with itself, CeTTa follows the principle its consistent cases express, not each exception. Every deliberate departure is an executable entry of `tests/petta/swi_differences.metta` in the CeTTa repository: the query, SWI-PeTTa's answer, the reason and the date of the decision, with CeTTa's answers pinned and checked on both execution routes. The file records the known departures; finding an unknown one still takes running both engines.

Special forms reached at run time. A special form is syntax only where it is written. Reached at run time—as the value of a variable or expression head, as the application a `reduce` dispatches, or as a function passed to another—its arguments are evaluated first, and the application is data unless PeTTa or the program defines a function of that name, which is then applied to the values. This is SWI-PeTTa's `reduce/2` rule. Reading such a value as syntax, as CeTTa once did, let a specialization change a program's meaning once it learned which form a variable held.

Functions named like forms. A program may define equations under a special form's name. Where the form is written it keeps its meaning. Reached at run time, the name is the program's function, applied like any other: at the arity of an equation its equations run, with fewer arguments it waits for the rest, and with more it is over-applied. The form never runs in its place, and a call whose equations all fail fails rather than becoming data. Both engines agree. The two roles must stay distinct: a representation that erases the difference between the written form and the function cannot keep both meanings, and specializing a known name never turns a call into binding syntax (`PeTTa.BindingForms` and `BindingDispatch`, proved in Lean).

`let` and `let*` reached at run time, when the program defines no function of that name, are data, as `if`, `case` and `quote` are in both engines. The binding form is recognized only where it is written: an application reached at run time is not binding syntax, whatever its head is named. Evaluating the data later reads it as the syntax it spells, and so runs the binding. SWI-PeTTa registers both names as functions without defining one, so it returns a partial application, which `eval` leaves unchanged and which only a `let` equation with more parameters, defined later, completes. Decided 2026-09-26.

Errors. In plain PeTTa an error propagates to the nearest `catch`, which makes it an `(Error ...)` value; uncaught, it ends the run. (Prime carries a fault as an occurrence of the outcome bag instead, never ending the run; see principle B.11 of the Prime specification in the appendix.) An `(Error ...)` value that a function returns is ordinary data. A run-time dispatch recovers an error raised by the body it runs: that path fails, and every other alternative and every answer already given remain. SWI-PeTTa instead discards the body's remaining alternatives, so its answers depend on the search order. Errors in argument evaluation, in setting up the dispatch and in over-application are not the body's, and propagate. Specialization keeps the recovery of a call that was dynamic where it was written; SWI-PeTTa's specializer drops it (PeTTa issue 223). For pure, completely explored alternatives the recovery distributes over choice, $handle(P \cup Q) = handle(P) \cup handle(Q)$ on occurrence bags, which is proved in Lean (`PeTTa.DispatchErrorScope`); so is the check that lets a specialization call a callee directly when it raises nothing (`PeTTa.RaiseFree`). Decided 2026-09-26.

Over-application. An application with more arguments than any equation of its head takes raises a domain error naming the arities the head has, as in SWI-PeTTa: written, dispatched at run time, under `reduce`, `foldl-atom` or `collapse`. Inside the body a run-time dispatch runs, it is that body's error and the dispatch recovers it. CeTTa used to return the error as a value and continue, contrary to the rule above.

`eval` and `reduce` never evaluate a computed result again. `eval` evaluates its argument once; `reduce` evaluates the arguments of the application written in it and dispatches the head on their values, as SWI-PeTTa's `eval/2` and `reduce/2` do.

Imports invalidate specializations. A specialized function sees equations imported after it was specialized; SWI-PeTTa keeps the stale specialization. Decided 2026-09-26, as the meaning of the unspecialized program.

4.5 Evaluator Bug Classes the Campaign Exposed

Three bug classes were found and fixed, all instructive.

The Empty algebra. When overlapping equations yield a branch that reduces to `Empty`, two dual defects appeared: the surviving sibling branch was returned without being re-driven to applicative normal form (violating the spec's applicative order), and in other shapes `Empty` itself leaked into the visible result bag (the spec is explicit that `Empty` "is not returned among other results"). The fix treats the evaluator's outcome set as an algebra with `Empty` as its absorbing zero and re-drives surviving branches independently of their siblings. In modal terms (Section 4.7): the successor algebra must satisfy $\Diamond\bot = \bot$.

The one-step congruence bug. The reflective one-step layer reported `(+ 1 (+ 2 3))` as a *normal form*: the one-step relation lacked argument congruence (stepping a reducible argument when the head cannot fire), while the big-step evaluator had it. Because the one-step relation and the evaluator were separately hand-maintained C branch sets, the gap replicated identically across three development branches and was invisible to every test that only exercised head-reducible terms. The bug was diagnosed through the modal layer itself ($\Box\bot$ holding on a reducible term), fixed as deterministic left-to-right congruence matching the evaluator's applicative order—and is the direct motivation for the rule-pack weld in Section 4.8.

The switch/switch-minimal conflation. Upstream HE types `switch` as `(-> %Undefined% Expression %Undefined%)`—the scrutinee is *evaluated* before matching—while `switch-minimal` is `(-> Atom Expression Atom)`, matching the raw scrutinee structurally. CeTTa routed both spellings through one structural handler, so `(switch (+ 1 2) ((3 three) ($x $y) (got $x $y)))` returned `(got 1 2)` where upstream returns `three`. The defect was exposed not by testing but by the certification campaign (Section 4.9): modeling the rules in Lean forced a precise reading of the two upstream type contracts, and an oracle probe confirmed the divergence. The fix splits the rule into `HES_Switch` (evaluated scrutinee) and `HES_SwitchMinimal` (structural), with a distinguishing witness pair pinned in the test suite.

A representative conformance probe. The defect classes above are locked by gates rather than by a standing program suite (standing suites tax every future run; the classes are the durable artifact). One representative probe is worth stating because its *correct* answer is doubly instructive:

```
(= (same $x $x) hit)
!(same a a)   ; [hit]
!(same a b)   ; [(same a b)]  -- inert, NOT hit, NOT empty
```

A nonlinear pattern constrains both positions to be equal, so positional pattern compilation without an equality check answers `hit` — and an engine grounded in a failure-driven host answers with an empty bag — while the specified behaviour is that the unmatched call remains *uninterpreted*. CeTTa's eager and lazy lanes and upstream HE agree on this and on the sibling probes for grounded-argument strictness, duplicate multiplicity, view invalidation, and strict equality; engines with different declared semantics (PeTTa's Prolog grounding, set-carrier stores) simply answer as their own specifications say, which is what declared contracts are for.

4.6 Checker Bug Classes the Native Authority Flip Exposed

The differential discipline of Section 4.5 was built to catch *evaluator* defects. When the `prime` lane's conversion judgment was migrated from its legacy route—normalize both sides through the HE type-level normalizer, then compare the results syntactically—to a native $\lambda\Pi$ kernel that decides conversion directly for the closed, well-scoped fragment, the same discipline exposed a subtler class: *checker* false negatives. The migration kept the legacy route alive as a separately compiled test-only oracle (production has no toggle back) and compared the two over a pinned eleven-row corpus. At the corpus's first commit, on 24 August 2026, the routes agreed on six rows, including the refutation of the unequal well-typed operands `u0` and `(-> u0 u0)`. On two rows the legacy verdict was wrong, and on three it answered where no answer was licensed:

```
!(type:eq (-> (x : u0) u0) (-> (y : u0) u0))
; native: [True]     legacy: [False]
!(type:eq (sigma (x : u0) u0) (sigma (y : u0) u0))
; native: [True]     legacy: [False]
!(type:eq (idx 0) (idx 0))      ; an open, ill-typed operand
; native: unresolved (the query stays inert)     legacy: [True]
```

The first two pairs differ only in the name of a bound variable. The legacy route compared names where it should have compared binders, and so *refuted* a true conversion. The instructive part is not that a procedure was incomplete—that is ordinary—but that its incompleteness was silently promoted to refutation authority. An incomplete procedure can honestly answer "incomplete"; it cannot honestly answer "no."

Three consequences are locked into the gates. First, verdict equality with a buggy oracle cannot be the acceptance criterion for a checker migration. The current gate states the separation exactly: without the kernel only the built-in numeric constants are decided; the kernel alone decides the six universe and binder comparisons; ill-typed operands stay unresolved on both routes. The pinned expectation files keep the divergence as a permanent witness rather than a silent fix—the same adjudication policy Section 4.3 applies to upstream behavioral divergences, now applied to a typing judgment. Second, the repaired class must earn its authority: mutation canaries that corrupt the kernel's admission (the receipts and generations of checking and of synthesis) are killed by the suite, so the tests demonstrably constrain the code that produces the new verdicts. Third, the flip is class-exact and fail-open: outside the recognized fragment nothing is refuted, and exhausting the native budget reports incomplete rather than silently retrying the other authority—a verdict must never depend on which checker happened to have budget. The four files of the closed $\lambda\Pi$ corpus

(conversion, checking, synthesis and formation) give their expected outputs on the draft binary of Appendix B.

The same lesson closed a sibling defect at the interaction of typing and matching: a typed-applicability pruning path refuted candidate pairs that the positive matcher accepts (a space-class mismatch between a negative license and the positive semantics it claims to approximate). The rule that covers both incidents: a negative answer is a claim about all executions, and may only be issued by an authority whose completeness for that claim is established; everything weaker prunes nothing and abstains.

4.7 The Reflective Modal Layer

The library `lts` (generic, step-parameterized) and `lts:he` (bound to the HE evaluator's one-step relation) reflect the one-step reduction relation into MeTTa as data:

- `lts:he:transitions` emits the one-step successor set of an atom as quoted states (zero results on quiescent atoms); `lts:he:trace-2` enumerates two-step traces.
- `lts:he:is-normal-form` is exactly $\Box\bot$ (every successor satisfies falsity, i.e. no successor exists); `lts:he:is-can-step` is $\Diamond\top$.
- `lts:he:box` ϕ and `lts:he:diamond` ϕ generalize these to arbitrary unary predicates: $\Box\phi$ holds when every one-step successor satisfies ϕ (vacuously at normal forms), $\Diamond\phi$ when some successor does.

The De Morgan law $\Box\phi = \neg\Diamond\neg\phi$ is installed as a runtime self-test; it is the executable mirror of the Galois connection proven once and for all in the Mettapedia OSLF framework, where LanguageDef $\to$ RewriteSystem $\to$ ReductionSpan $\to (\Diamond, \Box)$ is derived generically and `langGalois` establishes the adjunction. Native behavioral types in the sense of Native Type Theory arise as predicates over exactly this reduction span; the runtime implementation and the Lean derivation describe the same object at two levels.

Multiplicity discipline: `transitions` preserves the result *bag* (duplicate successors are semantically real), while the modal predicates quotient to reachability. Evaluation and conformance compare bags; $\Diamond/\Box$ use support. Result sets are never deduplicated for speed.

4.8 The Small-Step Rule Table

The congruence bug demonstrated the structural disease: reduction rules as anonymous, duplicated C branches are invisible, unremovable, and untestable-by-absence. The cure is to make the rule set a *named, reflectable object* the evaluator provably depends on.

Three granularity strata are kept distinct. *HEFine* is the Lean `mettaHE` instruction machine (59 rules over interpreter states; the proof reference). *HE small-step* is the coarse user-visible one-step relation—one observable source rewrite, e.g. `(+ 1 (+ 2 3))` $\to$ `(+ 1 5)`—which is what the `lts` layer exposes. *HEBigStep* is full evaluation. The table encodes the small-step relation; its relationship to the fine machine is a collapse/simulation with explicit provenance links (each coarse rule names the fine rules that justify it), never a label identity.

The runtime object, `CettaLangDefPack`, carries: language, profile, and granularity identifiers; a schema version; a source digest over the canonical rule-descriptor text; the rule table; a disabled-rule mask populated from the `CETTA_LANGDEF_DISABLED_RULES` environment variable; and a *per-rule claim level* recording the strongest evidence behind each rule (Section 4.9). The table has seventeen

rows. Ten rule classes are live and gate the *only* implementations of their behavior in the one-step subsystem: grounded dispatch, leftmost expression congruence, equation match, `eval`, `chain`, `let`, `let*`, `case`, `switch`, and `switch-minimal` (the last two split per their distinct upstream contracts, Section 4.5). Seven rows are accounted but not gateable. Four are quiescence rows (quote, return, empty and error), since disabling them would make the semantics incoherent rather than smaller. Three record full-evaluator behaviour of the extended profile that is not a coarse one-step rule: a comma in a space position folds the pointwise bag meet of the spaces, a pipe their additive union, and a two-argument sourced `match` is a seed-threaded conjunction across named spaces. Each row carries a provenance string naming the fine-machine rules that justify it. The pack is reflected into MeTTa by `lts:he:step-rules`.

The load-bearing test is *anti-decorative*: a dedicated lane re-runs the witness suite with each live rule disabled and *requires* the witnesses to fail (the successor set must go empty; normal-form must flip)—baseline plus ten rule-removal failures confirmed. If the witnesses still passed with a rule removed, a duplicate hidden code path would exist—precisely the disease that produced the congruence bug. A disabled special form goes *inert* (its head stays uninterpreted; congruence still applies), per the principle that unknown symbols are data, never errors. The lane fails closed in both directions and runs on both the default and MORK-bridge builds.

Claim levels are deliberately *excluded* from the digest: the digest fingerprints rule *content* (names, profiles, provenance, schema), while claims are *evidence about* that content. Upgrading a rule's certification therefore must not—and verifiably does not—move the semantic object: the digest stays fixed across claim flips; it moves only when rule content changes. At the pre-main binary of Appendix B it is `fnv1a64:eda58fb31fa185e1`, schema 2. The pack is, by design, a hand-maintained object kept honest by the removal lane and the digest; we deliberately parked an earlier plan to generate specialized C from the rule source, because the weld's value lies in the named semantic object and its fail-closed tests, and code generation is machinery the certification program does not need.

4.9 Lean Certification of the Rule Table

Each per-rule claim takes one of three values, in increasing strength: *bag-tested-adequate* (the rule's behavior matches the legacy path and oracle bags on the witness corpus), *rule-sound-on-fragment* (a compiled, sorry-free Lean theorem proves the rule absorbed by the official declarative semantics on an explicitly characterized fragment), and *rule-sound* (the same, unconditionally). A claim flips only when the compiled theorem exists, so the reflective layer can never overclaim. The certification route is *absorption*: for each coarse rule, a theorem that whenever the rule relates $a \to a'$, the official declarative judgment of the Mettapedia HE formalization (`MettaCall`, `MinimalStep`, or `EvalAtom`) already relates the same atoms. Five of the ten live rules carry certification; the claims below are those the pre-main binary reports through `lts:he:step-rules`:

rule	claim	Lean witness
`HES_GroundedDispatch`	rule-sound-on-fragment	`mettaCall_absorbs_grounded_dispatch`
`HES_EquationMatch`	rule-sound	`mettaCall_absorbs_equation_match`
`HES_Eval`	rule-sound	`minimalStep_absorbs_eval`
`HES_Chain`	rule-sound-on-fragment	`minimalStep_absorbs_chain_source/_subst`
`HES_LeftmostExprCongruence`	rule-sound-on-fragment	`evalAtom_absorbs_congruence_frag`
`HES_Let`, `LetStar`, `Case`, `Switch`, `SwitchMinimal`	bag-tested-adequate	in progress

The certified fragment for the congruence and chain rules is characterized positively: untyped constructor-headed expressions with quiescent spectator arguments and a grounded-dispatch or

equation-match redex. Its supporting theory is a short ladder of compiled lemmas: a quiescence domain on which atoms officially self-evaluate, uniqueness of that self-evaluation, transport of official evaluation across a single changed argument, and absorption of inner fragment steps—about which the honest surprises accumulated (e.g. a bare `Error` *symbol* at tuple head becomes error-shaped, so the fragment must exclude it explicitly).

The five remaining rules are upstream *stdlib sugar*: their one-step behavior is the upstream definition of `let`, `let*`, `case`, `switch`, and `switch-minimal` as `unify/chain` programs. Certifying them required a *matcher bridge*: relating the public equation-query API to the official `matchAtoms/mergeBindings` semantics. The key insight from this development stage was that the real seam was *not* merely a direct one-way-to-two-way matcher comparison. The public HE query API had historically routed equation lookup through the simplified `simpleMatch` path, while the faithful two-sided matcher already existed separately. We therefore split the bridge into three explicit layers.

First, `DeclMatchSpec.lean` states the author's `match_atoms` semantics declaratively and proves the executable `matchAtoms` helper sound and complete against it (`matchAtoms_sound`, `matchAtoms_complete`). Second, `DeclMergeSpec.lean` does the same for `merge_bindings/add_var_binding/add_var_equality`, proving `mergeBindings_sound` and `mergeBindings_complete`. Third, `QueryInterfaceAgreement.lean` proves a bounded adequacy theorem for the *public* query interface: `queryEquations_ground_two_way_adequate` and its `queryEquationsAgainstVisible`-companion show that, on the ground-query fragment and under the explicit freshening adequacy hypothesis `EquationFresheningAdequate`, the repaired `matchAtoms/mergeBindings` API and the legacy `simpleMatch`-based scan agree in both directions up to replay fuel.

The earlier singleton-seed factorization theorem remains one ingredient inside this story, but it is no longer the whole story. The decisive insight is the G3/G3b split: equality-threading becomes observable only when the queried atom itself still contains variables, because that is the point at which `Bindings.resolve/Bindings.apply` must honor equalities rather than plain assignments. We therefore bank the ground-query adequacy result as the honest current certification API, and carry the variable-bearing query case as an explicit next obligation rather than silently pretending the legacy API was already faithful.

5 Development Lines and Integration

The November preview combines Opt's evaluator/query/storage work, NIK's checking infrastructure and the Python/C embedding interface. Its source identity and fresh application observations are recorded in the appendix. The dated histories below explain how those mechanisms arose; they are not an open-bug checklist for the preview or a claim that independent Prime research branches have all been release-qualified.

CeTTa is developed on several lines at once, each a git branch with its own worktree and its own purpose. This section gives each line's purpose, what landed and when, and what it measured. Dates are those of the commits. Numbers come from the measurement reports written as the work landed; each says what was measured, against what, and how often. "Instructions" are user-mode instructions counted by `perf`; "G" is 10^9 of them.

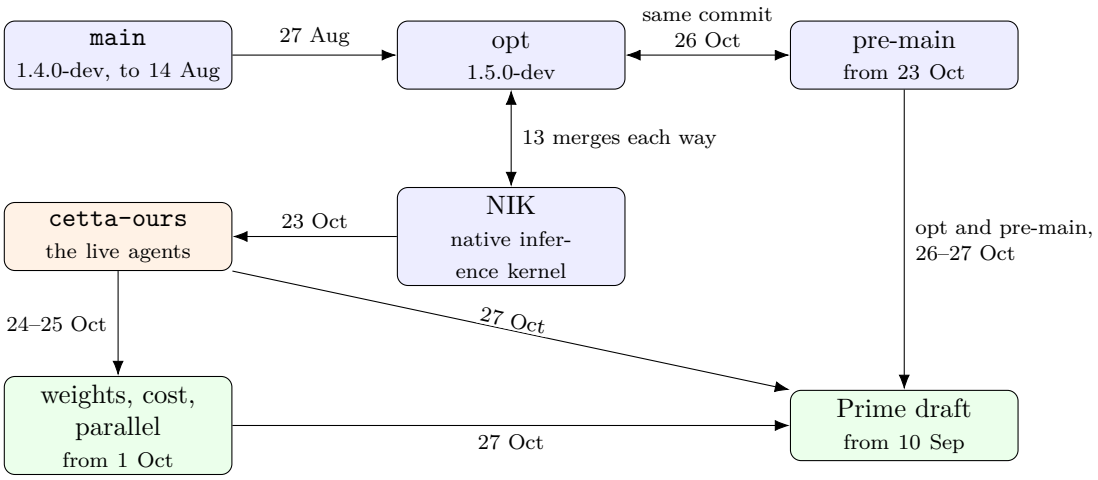


Figure 3: The lines and their main merges in October. An arrow points from the line merged to the line that merged it. The weights line also took the draft and pre-main on 24 and 25 October, and the draft had taken opt, NIK, the weights line and `cetta-ours` several times before (Section 6.1).

line	historical snapshot (27 Oct)	purpose
`main`	12da03b4, 14 Aug	the published, qualified baseline
opt	working tree based on feba78e3	the careful fixes: execution, query and memory architecture
pre-main	integration tree based on feba78e3	integration of the execution contracts; a candidate for `main`
NIK	2f168dca	native inference kernels, then native application libraries
Prime draft	working tree based on a0f1077a	Prime in C, in step with its metatheory
weights	750896ae	weights, cost accounting and parallel execution for Prime
`cetta-ours`	d436e11b	the engine of the live agents, kept running by quick fixes

A finding on `cetta-ours`, where quick fixes keep the agents running, is a lead for opt to reproduce and root-cause, never a change to copy. Opt's method, stated with its October goal, is to find the floor of a cost by specializing, ask how broad a region of programs the semantics admits, build the fast path for that whole region, and prove it admissible with executable laws.

5.1 `main`

`main` is the published baseline of the public repository. Its README describes it as a direct C runtime for MeTTa whose default lane is the HE evaluator, and as a working snapshot rather than a polished release. Its 215 commits run from the first evaluator (19 March 2026) through the storage substrate (term universe, MORK and PathMap backends, April), HE conformance and the rho calculus (June), the tail-safe collector (2 July), Prime, parsing and dependent search (July), to a main-readiness tranche in August: the native PeTTa corpus at 176/176, the counted-PathMap query route, `MatchDecision`, `typecheck-v2`, the Prime outcome carrier and MeTTa Zero. The last integration was declared qualified on 11 August, by a composed exhaustive run of 15 commands

taking 7,490 s, with the PeTTa corpus at 183/183 and `fib(999999)` giving the same SHA-256 digest on the HE, Prime and PeTTa lanes. The same tranche retired the ground-call memoization table, which earlier drafts of this paper described.

The published baseline described here is the August version. The November preview combines the committed Opt/NIK integration with the Python/C ABI and pending readiness repairs; its measured executable is identified in Appendix A. Historical branch distances are not a release-readiness criterion.

5.2 opt

The opt line (branch `cetta-shared-opt-audit-20260807`) is the integration base for the execution, query and memory architecture, and the line of careful fixes. Its September work is in Section 11; this section covers October.

What landed.

date	what
1 Oct	conformance checkpoint; an optional durable store; the open tier takes whole sequences
3 Oct	the typed tier: compiled PeTTa typed calls reuse exact guard facts
4–5 Oct	owned execution: explicit ownership, owned HE and PeTTa sequencing, canonical HE on owned continuation frames shared with PeTTa
7 Oct	directional matching (`=%` rules, the `pat` library)
10–11 Oct	the regional execution core: owned control and return storage, producer dependencies, ordered term-path indexes, counted conjunctions and cursors
13 Oct	tabled suffix forwarding, a head-chain store, removal by marks (a bitmap with a Fenwick tree, one contraction per batch)
14 Oct	the PeTTa default pinned to version 1.0.6, with version selection
16 Oct	resource lifetimes: results moved out of dying storage, ground matches from exact occurrences, reclaimable arena names, suspended calls that keep only what resuming needs
17 Oct	HOL Light, MM0 and Megalodon kernels; streaming files; `lib_re`; PeTTa becomes the default language
18 Oct	the prepared machine runs typed calls and MeTTa-defined library forms
19–20 Oct	64-bit variable identities; semi-naive saturation; one closure engine
20–23 Oct	PeTTa fast regions; JSON, process and tensor libraries merged from NIK
24–25 Oct	the evaluator tax: `not` and `case` in the prepared machine, closures specialized by use (4b74b879), lifting by shape, `let` destinations admitted by evidence
26 Oct	native `transform`, complete PeTTa lists built flat, late output, linear drains and the head index (2030a80f)
26–27 Oct	a spread hash for `unique-atom` (a93e4a6f); then, staged and not yet committed, bag-order reads, set spaces, a native query path, worst-case-optimal joins and joins over rows of bits

The ledger against SWI-PeTTa 1.0.6 (23 October). A fixed panel of 96 cells in 19 families, timed as whole processes, three-run medians, against the faster of two SWI-PeTTa references (version 1.0.6 upstream and the local reference checkout on SWI-Prolog 10.1.9). All 96 matched their declared observations. Of the 92 comparable cells, CeTTa was clearly faster on 78, SWI-PeTTa on 8, and 6 overlapped. Representative cells:

cell	CeTTa	SWI-PeTTa
HOL benchmark	1.235 s	5.977 s
Peano	0.017 s	1.520 s
MateSpace2	0.421 s	5.357 s
Nil, 233,680 proofs	1.718 s, 822.9 MiB	15.382 s, 7,416.5 MiB
grouped join and count	1.134 s	11.034 s
selector over 2,048 closures (a loss)	2.04× SWI	
Ambient (overlap)	15.653 s	14.021 s

Seven of the eight SWI-PeTTa wins read foreign data, and on those cells 45–60% of CeTTa's instructions are in the embedded Prolog engine; on every other cell but the chainer cells (7–8%) and one short Nil example dominated by the engine's start (about 11 ms), the Prolog share is at most 2.3%.

Closures (24 October). A site applying many distinct closures paid an analysis per closure. Specializing a closure from its second call, and telling values apart within their shape, took the 2,048-closure selector from 0.164 s to 0.064 s (SWI-PeTTa 0.114 s) and a 100,000-cell walk from 0.615 s to 0.064 s (SWI-PeTTa 0.264 s), best of three. A first screen missed the chainer cells, which rose by 2–2.4% in instructions; lifting closures by shape the next day brought them to 0.85–0.90 of their earlier count.

The evaluator tax. A step of a loop written with `case` cost 26,820 instructions against 3,854 for the same loop written with `if` and `let`, and a step using `not` cost 31,369, because those forms fell off the prepared machine into the general one (8,004 matches, exact counts, 24 October). Moving them into the prepared machine brought a loop called through a caller's `case` from 8,585 to 3,785 instructions per step, with the 97-cell screen unchanged (geometric mean 1.0002). Measured again on 26 October, integration had reopened the gap: from a pre-main checkpoint of 25 October the ten gate shapes cost 11,470–14,080 instructions per element against 3,560 before, because an answer producer deep-copied its data twice; no merge had run the tax gate. After the repair the shapes cost 3,200–5,500 per step, and one of the ten (`case` with a byte slice, 5,459 to 5,694 depending on the build) remains above its 5,000 budget.

`transform`, lists and drains (26 October). The native `transform` of Section 2.9 fires the library's rows over the spaces' occurrence cursors and one trail-backed binding environment, and declines whenever the library or any data constructor it uses has been redefined, so the authored definition then runs. A differential checks native, authored and specification firings on fixed and seeded programs; an independent check agreed on 1,992 of 2,018 firings, the rest being 20 null firings and 6 removals of a bare symbol. Building complete PeTTa lists flat, and binding a list built by `cons` after the body, removed most of the library's interpretation cost. Wall times on a loaded machine (load about 14 on 16 cores; the report does not state run counts):

program	native	library on CeTTa	library on SWI-PeTTa
COMM wave, 400 pairs	0.020 s	0.033 s	0.110 s
drain, 1,600 jobs	0.039 s	0.064 s	0.142 s
drain, 6,400 jobs	0.123 s	0.232 s	0.330 s
closure, 40 nodes	0.037 s	0.133 s	0.207 s
counting, zero groups	0.016 s	0.018 s	0.092 s

The 6,400-job drain peaks at 33 MB on CeTTa and 60 MB on SWI-PeTTa. Drains had been quadratic for two separate reasons: a `once-match` built and sorted every remaining candidate, and a removal settled marks that forced a trie rebuild. Reading through marks, a head index per head symbol and length, lazy matches on pinned cursors, and committing after the proposal unwinds made them linear: in instructions at 1,600 and 6,400 jobs, native 3.56G and 51.3G became 0.55G and 2.09G, and the hand-written loop 1.00G and 13.3G became 0.24G and 0.93G. The specification library `lib_transform_specs`, quadratic by design, still runs 1.8–2.3 times slower than on SWI-PeTTa.

Applications on MORK's MM2 workload families. The Opt programme used nine application families: two-hop materialization and triangle joins, five-cliques, odd-even sorting, backward proof search, Hexlife, ancestry, path-producing reachability, a unary counter machine and process-calculus communication. These exercise evolving result sets, deeply nested terms, non-ground process bodies and sustained joins, rather than only startup or arithmetic loops. Their MM2 sources and ordinary MeTTa solutions share declared input facts and result relations; they are distinct programs, not a benchmark of a wrapper calling the same MORK implementation.

For the five-clique task, the input has 200 vertices and 3,600 distinct undirected edges, each stored once with its smaller endpoint first. A result is a five-tuple whose ten required edges exist. The measured program is a conjunction over those facts with a set insertion; after the facts it reads:

```
!(bind! &t (new-space set))
(= (rule-clique)
   (match &self
     (, (edge $x0 $x1) (edge $x0 $x2) (edge $x0 $x3) (edge $x0 $x4)
        (edge $x1 $x2) (edge $x1 $x3) (edge $x1 $x4)
        (edge $x2 $x3) (edge $x2 $x4) (edge $x3 $x4))
     (add-atom &t (5-clique $x0 $x1 $x2 $x3 $x4))))
!(let $r (collapse (rule-clique)) done)
!(size-atom (collapse (match &t (5-clique $x0 $x1 $x2 $x3 $x4) t)))
```

There is no clique-specific native primitive here. The workload exercises shared variable constraints across a ten-leg join, the choice of join order and publication into a set; the input has 79 result tuples. MORK's own benchmark writes the same ten conditions in the same order, and MORK's default join follows that order, while CeTTa plans its own; the appendix therefore also times the MM2 query with its conditions reordered in MORK's favour.

The process-communication task stresses a different boundary: the space contains non-ground receive bodies and ground sends. A matching payload instantiates a receive's body and adds the resulting process; a parallel process contributes both children. These are persistent rules of MORK's benchmark, not consuming Rholang channel semantics:

```
(= (communicate)
   (match &p (, (recv $channel $payload $body) (send $channel $payload))
     (add-atom &p $body)))
(= (split)
   (match &p (par $l $r) (let $_ (add-atom &p $l) (add-atom &p $r))))
(= (rounds $k)
   (if (== $k 0) done
     (let* (($c (collapse (communicate))) ($s (collapse (split))))
       (rounds (- $k 1)))))
```

The measured program builds unary addition from these rules and runs 500 rounds, as MORK's clocked program alternates its two rules 1,000 times. Retaining variable identity and deeply shared

payload terms matters as much as finding the initial matches.

The generators of both programs of every pair are in `benchmarks/opt_suite/`. Appendix A.2 measures the suite on the November preview against both of MORK's builds, the default one and the one with its experimental leapfrog join, after checking that every pair computes the same complete result set at the timed sizes. Two earlier statements are corrected there. The counter-machine timings of the earlier drafts measured a faulty PeTTa program, which stored an unevaluated numeral in its instruction-counter field; matching row counts had concealed the differing states, and the corrected program passes the complete-state check. And the workload earlier called transitive closure materializes two-hop pairs and triangles; it does not compute a fixed point.

Set aside. An earlier broad branch of Nil support was archived on 6 October: building a term could run code and child spaces copied the whole parent. A change that returned every HE call's value at once was reverted on 17 October after it reproduced a duplicate from `unique` and let library natives ignore added equations; its non-HE parts returned the next day. An attempt to merge opt into the `cetta-ours` and weights integration was aborted on 17 October (41 conflicted files).

Integration status, 5 November. The Opt mechanisms are merged into the preview's integration base. Fresh application timings and their result checks belong to the dated appendix; earlier catalogue figures describe their original checkpoints. Remaining release qualifications, including platform and allocation-failure limits, are stated with the embedding preview rather than inferred from an old list of staged changes.

5.3 pre-main

pre-main (branch `pre-main-20261023`) was branched on 23 October from c632ecd1, the merge of opt into NIK, to make one line ready for `main`: coherent execution, smaller mechanisms, preserved performance.

The problem. CeTTa runs PeTTa through several machines: the ordinary evaluator, the prepared register machine, the open-equation machine and the search machine, with the HE evaluator beside them. Each crossing between them rebuilt facts the caller already knew; a library native called from the open-equation machine made a host round trip of about 2.4 μs; and once one tier declined a caller, the whole callee closure paid that cost.

The answer. One call and return contract, shared by every route: a borrowed invocation (`CettaInvocation`); a tagged outcome that keeps "declined" apart from "ran and gave no answers", and finished data apart from pending computation (`CettaCallOutcome`); a return cursor that hands out answers without calling again (`CettaCallReturnCursor`); owned sequencing that keeps a producer alive while its child runs and never replays an accepted answer (`CettaOwnedBind`); shared generated plans for scoped commitment and error handling; and source-fuel accounting by episode, with per-position debits. Each step is paired with a Lean model of its contract.

What it measured. Against the frozen c632ecd1 artifact, on a fixed 96-cell, 19-family panel (paired runs, medians, geometric means), the staged state of 27 October is 5.2% lower in wall time, 3.8% in instructions and 1.4% in peak memory. Pre-main's own work, measured before it merged opt's closure work on 25 October, was neutral within half a percent, so most of the step coincides

with that merge. Measured mechanism gains include an 8,004-step agent scan from 37.65 ms and 468.5M instructions to 18.82 ms and 241.7M, and an open-equation-machine native call from 2,066 host goals to 2. Correctness held at every checkpoint: the 471-program corpus in normal and statistics builds, 1,413 differential queries, and focused sanitizer runs. Against SWI-PeTTa 1.0.6, on references timed on 23 October, CeTTa was slower on Ambient (15.06 against 14.02 s) and the rho cell (3.80 against 2.87 s), and faster on the lambda cell (1.11 against 1.29 s) and a million-element map (0.416 against 1.664 s).

Integration status, 5 November. Pre-main now contains the committed Opt/NIK merge and the Python/C ABI integration, with the pending preview fixes described above. The candidate check is bounded and records which work passed. Broad proof, sanitizer, performance and distribution campaigns retain their distinct scopes; the ordinary preview's usability is not presented as an exhaustive-release certificate.

5.4 NIK

NIK stands for *native inference kernel*. Its founding offer was: a user provides a calculus or an interactive theorem prover as a formal system, and CeTTa provides a disciplined native kernel for it, an authority-indexed checking service generated from admitted GSLTs that must not become a second evaluator.

What landed. In August: plural checking authorities in Prime and direct typing authorities (an admitted type cache cut 100,000 `get-type` queries from 21.5G to 10.2G instructions), then hosting, LanguageDef-to-ParserPack generation and JSON. In September: `lib_tptp` and type interfaces. From mid-September the line became the home of native application libraries for the agents and an automated mathematician port: TPTP and SZS results, JSON values, process execution and interactive process streams, `lib_re`, tensors, sparse postings and embeddings, and a persistent connection to a Lean process. Opt merged NIK thirteen times and NIK merged opt at least fifteen; merging them on 20 October moved 17 workloads by +0.74% wall time and +0.007% instructions (seven trials). The automated mathematician's 875 supplied proofs replay through MeTTa and an independent Lean check.

Side branches. Three side lines branched from NIK. `nik-vibe-mm0-20261002` holds uncommitted work on kernel formation for VibeITP and MM0 certificates. `failed/nik-native-he-module-20261008` archives native HE modules as one commit, never merged. A detached worktree holds an Agda checker.

Set aside. The native HE modules were correct on every gate but slower (instructions +26% on forward chaining, +109% on an eight-way join), and were archived on 8 October and restarted one step at a time. An Agda checker inside NIK was wound down: it could not refute. Rebuilding the MM0 checker in MeTTa on the current tier was judged a loss against the native checker.

Incremental compression. The 30 October source-library POC learns two helpers from the unchanged PLN truth-formula library under an explicit frozen `/safe` profile: 2,207 → 2,186 → 2,180 structural nodes, including definitions and calls. Native reconstruction, finite public-call checks and stopped/resumed agreement qualify this bounded result. The profile travels with a replay capsule; it is not yet a general production codec extension. Source reflection and ambient type

declarations delimit execution-equivalence claims. `papers/IncrementalCompression.tex` records the experiment, negative controls and future integration interfaces.

Open. The kernel-theory programme has not yet run a kernel with real binders under two-way adequacy, or a measured computation tier against MM0 or Dedukti.

6 The Prime Families

Three lines carry Prime and the agents built on CeTTa: the Prime draft, the weights, cost and parallel line, and `cetta-ours`. They exchange work constantly; on 27 October 2026 the draft had absorbed every commit of the other two. This section follows them in turn and ends with that night's merges. Statements about Prime carry their kind: [draft] for the C draft's behaviour, [candidate] for a property of a candidate design, [direction] for a design direction with its reasons, and [decided] for a recorded decision.

6.1 The Prime 2.0 draft

The draft (branch `fable-prime-2.0-draft-20260910`) is Prime's one C worktree. Its README describes Prime as a higher-order MeTTa in which code is a value, programs can be checked by a native dependent type kernel, and proofs are terms the kernel checks, and the draft as the place where Prime's semantics is settled against its formal metatheory, so its interfaces may still change. Its working rule is that everything in Prime has three faces: extensional (its mathematical meaning, judged in `set:` and `hol:` spaces), intensional (how it is built and checked: the `type:` language and the kernel) and operational (what it does when run). A construct is finished when it has all three and the links between them: it runs as its equations say, what runs is typed, and what is typed means the set it should.

What landed. The draft's own history is in its branch log rather than its first-parent chain, because it was fast-forwarded onto `cetta-ours` on 21 October and onto the weights line on 22 and 25 October [draft].

date	what
10–23 Sep	the worktree begins on opt: `lang:`, `type:` and `set:` judgments with four-valued verdicts, four identity profiles, a curriculum of about 170 judgments; first commit 23 Sep
24–29 Sep	the kernel trust boundary and certified transforms; typed conversion with η; cumulative subtyping; a level-general recursor; theorems published with their context
25 Sep–8 Oct	scope machinery: names follow the rho calculus with sealed quotation; own-name records under scope profiles; `new` and run-time lambdas; lexical-fresh binding made the default on 8 Oct (115 programs translated, 480 of 480 Lean scope rows matching)
1–4 Oct	universe levels below ε_0 with computed written levels; sets ambient; native priority control, demand and resumable search; binary arithmetic and an opt-in native arithmetic oracle; the Megalodon set face quarantined behind an explicit profile
12 Oct	a HOL Light guest; declared binding languages; HE-style type declarations answered by the grounded HOL fragments, never by HE's engine
20–23 Oct	a fair first-answer scheduler and its two repairs; data versus `Atom` by construction; completed-value producers; one sequence view
26 Oct	standard terms redefinable with `std:` backups (83cb3038); a mathematical workspace library
26–27 Oct	merges of pre-main, opt, weights and `cetta-ours`; the interaction kernel, staged

Set aside. A contextual call-by-push-value lane (1,382 lines of library and its tests) was removed on 22 October to be redone after the metatheory cleanup. A tranche of collection space kinds was reviewed as not mergeable and is not in the tree. The October loop work was built outside the tree and has not been ported. The draft's set face had silently been Megalodon's higher-order Tarski Grothendieck theory; it was found and quarantined on 4 October.

Measured. The draft's measurements are diagnostic rather than competitive. Selecting 7 answers under native priority control costs 322 scheduling quanta against 10,404 for completing a 200-answer stream (about 6 ms against 24 ms). A `let`-body fold over 500 to 2,000 items costs 6.5G to 320G instructions, cubic in the length; a native iteration built outside the tree brought a 1,000-item fold from 43.9G to 0.26G but has not landed. On naive `fib 30` the Prime lane takes about 2.6 s and 271 MB against the HE lane's 0.04 s (single runs, Appendix B): the need machinery is the cost.

Open. Recorded known issues PK-3 to PK-5; thirteen fixtures that hang or differ because of the weights line (below); `==` on state handles comparing stale contents; the `cons`/`nil` contract; and the deviations of Section 8.4. Verification of execution traces and receipts is deferred until the other draft work is resolved.

6.2 The weights, cost and parallel line

This line (branch `prime-weights-parallel-20261001`, one writer) carries one goal: graded `where` and semantic weights, reflected and certified costs that programs can inspect, native parallel workers under resource contracts, and optional history, replay and debugging. The space service with multi-account authority grew inside it: generation-stamped grants, stable service-scoped principals, one-shot tickets and a durable decision ledger. Its goal requires that fine-grained native execution preserve exact occurrences, bindings, current authority, declared observations and complete retained computation.

It was created on 1 October from the draft, integrated owned execution, costs and parallel control on 11 October, mode-directed workers on 14 October, optional history on 16 October and native space workers on 18 October, and took the draft, `cetta-ours` and pre-main in three phases on 24 and 25 October. Its own whole-goal audit of 26 October puts it at about 80%, with five of nine gates and 13 of 25 contract families supported, and states that its earlier estimates of 96% and 90% overstated completion; 12 to 20 substantial blocks remain. Its measured overheads are small where they were measured: compiled-in history recording costs 0.07–0.18% in instructions over 138 executions, and recording on request 5–13%. Its first theory question, what one source interaction observes, keeps, consumes and emits, is the question Section 7 answers.

6.3 `cetta-ours`

`cetta-ours` (branch `cetta-ours` of the `godelclaw/CeTTa` remote) is the engine of the live agents. Its README describes it as Prime combined with the native process, timer, mailbox, memory and Telegram libraries the agents use; the agents pin its revision, and their cognitive loops run on the PeTTa lane. It forked from opt on 4 October and has some 200 commits of its own: managed symbols and collection experiments; unified HTTP, process and channel libraries; a revisioned space service; native tensors and memory collections with packed cosine search; native mailboxes; SQLite; authenticated service peers; and the primitives the agents' port from Python asked for. It merged opt, NIK, the weights line and the draft repeatedly.

Its fixes keep the agents running; opt takes its findings as leads. Three examples, with the line that root-caused them:

- A responder grew to 6.7 GiB over eight hours. `cetta-ours` traced it to overlapping library equations and fixed the library (403 MB to 48 MB on a 1,000-cycle probe). Opt reproduced it on its own head and fixed the engine: reclaimable arena names, a resource ledger, and equations guarded by `decons` leave no alternative behind; the reproducer went from 964.2M to 332.2M instructions and from 179.2 to 19.1 MiB.
- A cut Prolog `eval` leaked its state, one per cut. The quick fix disposed of it; opt redesigned the evaluation lifecycle so that a three-answer evaluation cut 203 times leaves no state live, and the same change designed out a use-after-free.
- The agents' native ports ran 5 to 30 times slower per step than expected. Opt measured the tax as narrower: the seam between machines, where one byte-slice call cost about 25,000 instructions as a host goal against about 2,200 inside the prepared machine (Section 5.2).

6.4 The merges of 26–27 October

On the night of 26 to 27 October the draft took three merges: opt (as dd5dbf98, which also brought pre-main), the weights line (as 18c680aa), and `cetta-ours` at d436e11b (as a0f1077a, committed on 27 October).

- The opt merge kept both atom-table declarations, split cursor admission from capture, refused detachment under an owned execution, and carried opt's three-argument cursor interface into the draft's own call sites.
- The weights merge passed 274 of 284 fast Prime fixtures, and did not build until one missing argument was supplied in the next merge.

- The `cetta-ours` merge had 22 conflict regions in the space service. The draft's multi-account authority layer and the incoming scoped observations now work together: an observation belongs to the principal and grant generation that captured it, and an observed batch is journaled with its authority stamp and resolved revision. Journals written by `cetta-ours` binaries replay in the merged service across a checkpoint. Generated files were regenerated from the merged sources, never merged by hand.

The gate after the merges: `make test` passes except the manifest check, which fails on test files of unrelated work in progress and on one fixture missing from its lane. In the full Prime gate no failure comes from the `cetta-ours` or opt merges: each also fails on the draft before them. Thirteen fixtures hang or differ because of the weights line, which fails them exactly as the merged draft does, while the draft after the opt merge runs them correctly in under a second. Their repair belongs to the weights line.

7 Spaces and Channels: the Interaction Basis

MeTTa has spaces, stores of atoms that programs query and update. Rholang and the rho calculus have channels, names on which processes send and receive messages. A language for many interacting reasoners needs both, and needs them to compose: to consult shared knowledge, to consume an obligation exactly once, to publish a result, to wait, and to do all of it on many processors without changing what a program means. A study of 26 October 2026, composed in [21], asked what single primitive serves both, what concurrency it admits, and what language sits on it. Its answer is the *direction* this paper adopts for CeTTa's spaces and channels [direction]. This section states the answer, what is proved, what runs in the Prime draft, and what is missing.

7.1 One interaction

A *store* is a finite set of *occurrences*: each occurrence has an identity that is never reused, and a label saying where it lives and which atom it carries. Two occurrences may carry the same atom, so a bag of atoms is a set of occurrences. A *commit* is one atomic step with three parts:

- it *observes*: matched reads pick occurrences, and *predicate queries* read a whole region (all of it, a count, or absence);
- it *disposes*: each matched occurrence is *taken* (consumed) or *kept*;
- it *emits*: it *makes* fresh occurrences at any locations.

A commit is enabled when what it takes and keeps is present, nothing is both taken and kept, what it makes is fresh, and every predicate query holds. In the language of graph rewriting it is a double-pushout span $L \supseteq K \to R$ with application conditions: L is what is matched, K what is kept, $R \setminus K$ what is made, and the predicate queries are the conditions. Its four parts, keep, take, make and predicate query, form a *basis*: none can be encoded by the others relative to a stated observer (a commit that encodes another must be enabled at the same stores, produce the same labels, and commute with the same commits). A fresh name is a kind of make, not a fifth part, and choice belongs to rules with several rows, not to the commit [21].

A predicate query is the part whose answer can change when an atom is *added*: "all", "how many", "none", and "everything here is like this" (the absence of a counterexample). Its answer depends on every occurrence that satisfies its pattern, including one made later, as a predicate read

does in database concurrency control [27]; so it conflicts with every take or make inside its region. Each is a fold of the region's matches through an order-blind operation: `all` collects them as a bag, and a count, absence and uniqueness are folds of that bag. A matched read stays true when atoms are added. This one distinction carries most of the theory.

7.2 Spaces and channels are one object

Four rights, called *facets*, license the four parts at a location: `receive` (take), `peek` (keep), `send` (make) and `observe` (predicate query). A commit outside its holder's facets is not *formed*, and is refused before it runs. A space is a region whose holders have all four; a Rholang channel is a region without `observe`. The other named profiles are a mailbox (senders are blind), a log or fact region (nothing leaves) and a board (readers neither add nor remove).

What makes this more than a naming scheme is that each missing right buys a law, proved in Lean [21]:

- where nobody may query a region, arrivals never invalidate anything: a commit stays enabled when occurrences arrive outside the regions it queries, the cheap per-occurrence claims check is already serializable, and a waiting guard is woken only by a make;
- where nobody takes and nobody queries, replicas merge into a legal serial run without coordination, while consuming something exactly once can never be coordination free;
- along runs of formed steps, authority is passed and never created.

A region's guarantees therefore depend on what *others* may do there, so its permitted operations are declared with the region, kept by the authority layer, and checked when a program is formed [direction].

7.3 Meta-spaces

The study's answer to unifying spaces and channels "into meta-spaces that can offer different interaction modes" is a region with independent axes [direction]:

axis	values
read policy	what "the same" means and how answers are read: bag, set, list order, or modulo declared equations
profile	the facets each holder has
predicate-query discipline	owned (one writer), stratified (sealed before the query), or shared (validated at commit)
coordination policy	free, escrow, owner or lease; home or replicated
engine	storage and index; never changes meaning

Today's dialects become read policies: Prime reads a bag, while HE and PeTTa read storage order, and a sequence reading specializes a bag reading but not conversely. The one constraint between axes the study found runs from profile to engine: a predicate query modulo equations needs a matcher complete for those equations.

7.4 Today's operators as cases of the one form

operator	as one commit
`match`	an `every` of a read: one instance per occurrence and per answer
`add-atom`	a commit that only makes
`remove-atom`	a take; which occurrences it takes differs by lane
counting by `collapse`	an `all` predicate query
`unify`	a `let` source with no footprint
`new-space`	a make of a fresh name

What the form adds is several takes in one commit (atomic joins), predicate queries with a declared stability discipline, waiting (`await`, `whenever`), rights on handles, and installed rules whose installation commutes exactly with the commits that do not query the rule base.

7.5 What is proved

The study produced 106 Lean modules (49,762 lines, none with `sorry`; every cited theorem uses only `propext`, `Classical.choice` and `Quot.sound`), covering the commit and its exactness, batches and `every`, traces, isolation modes, waiting and fairness, names and rights, reflection, a congruent equivalence under which Rholang's receipts are exact aliases, types (sessions, footprints, facets), clocks and leases, coordination across locations with its impossibility results, and the admission of rules promoted from data [21]. Representative theorems, each present in the Lean tree: `Enabled.frame` (the frame law), `readTake_one_refused` (identification), `every_refuses_clash`, `every_complete_spoiled_by_arrival` and `checkRun_isSome_iff` (the trace checker is exact). A later audit of authority in the Prime runtimes added a permissions layer of 73 theorems, among them `absence_preserved_iff`: a universal read over a region is preserved by every run of the other subjects exactly when none of them can reach `send` at its names from the initial delegation state.

7.6 What runs in the Prime draft

On 27 October 2026 the Prime draft gained a native interaction kernel with a MeTTa library, five fixtures and a make target, staged for commit [draft]. Listing 12 runs on it. Its measured state:

- **The commit.** `transform` reads, keeps, takes and makes in one step across any number of regions; `every` rows are linear on their own and the whole commit must be linear; predicate queries dispose of nothing; a `let` runs once its variables are bound. The kernel matches against the live store, checks rights at every read, and never runs code: it hands each `let` back to the library.
- **Regions and handles.** `new-region` answers an opaque handle sealed by a brand private to the kernel; `facet` attenuates it. Formation is checked when a firing is formed, when a rule is installed, and at run time for locations supplied by data.
- **Waiting.** `await`, `whenever`, `choose`, `else`, `par` and `seq` are implemented; a parked firing is woken by any change in a region its last attempt read.
- **A corpus in two styles.** Twelve classic coordination programs (producer/consumer, bank transfers, dining philosophers, barriers, reactive rules, an MM2-style closure, CHR, Linda, Erlang receive with timeout, Rholang contracts, rules as data, leases with fencing) were

written twice and run under 32 schedules each. Written with today's idioms (`match`, `remove-atom`, `add-atom`, counting by `collapse`), 11 of the 12 violate their property under some schedule, though all are correct sequentially. Written with the interaction form, none does.

- **Verified traces.** The Lean trace checker accepts all 32 kernel traces of the corpus and refuses all 3 corrupted ones.

Costs were measured on single runs. Draining 2,000 jobs on the Prime lane takes 4.9×10^{10} user instructions (3.5 s) with today's idioms, 1.9×10^{10} (1.4 s) as a `whenever` rule, and 7.4×10^{9} (0.5 s) as one `every` commit, of which filling the queue costs about 7×10^{9}. On the same task the PeTTa lane runs today's loop in 1.2×10^{8} instructions, SWI-PeTTa in 0.09 s, and Rust HE timed out at 500 jobs and was killed at 15.6 GB at 2,000. On the Prime lane the evaluator's need machinery, not the kernel, dominates (61% of the rule form's time).

The draft also carries the PeTTa-lane `transform` of Section 2.9, merged from the opt line. The two engines were run on twelve probes whose answers the theorems fix. The value-level PeTTa engine deviates on two: it fires a kernel that takes what an `every` row keeps, and it turns two stored schematic atoms into ground copies. The occurrence kernel meets all twelve. The direction is one engine, the occurrence kernel, with the PeTTa engine's techniques (trail-backed bindings, PeTTa's index pattern, presentation admission) moved into it [direction].

7.7 Authority

An audit on 27 October found that "authority" names at least five unrelated mechanisms in the Prime runtimes [draft]. Three are permission mechanisms. The space service's grant layer (principals, attenuating grants, one-shot tickets, a durable decision ledger) is unforgeable, attenuating and revocable, with one escape: a grant's ticket requirement is attenuated by subset, so a child grant may drop it and append without limit. The interaction kernel's region facets are real capabilities with two gaps: a location that is not a handle resolves to its holder's rights, and one global seal brand lets a forged board pass. The evaluator's effect "authority" is a thread-local flag checked against allowlists, which cannot be forged but cannot be delegated or attenuated either. The direction is one carrier, the sealed handles, with the grant table as an instance of the delegation core and an effect capability in place of the flag [direction].

7.8 What is missing

The native runtime is the gap. Reading the trees on 26 and 27 October, the study found no C tree with stable occurrence identities separate from content identities, per-occurrence claims in canonical order, per-region epochs bumped by takes and makes, a wait index by guard kind, or persistent receivers that are kept rather than taken and republished; the draft identifies an occurrence inside a commit by its space, the space's revision and its position. The composed study estimates the native runtime at about 5% of its scope, the theory at about 76% and the surface language at about 50%. Three choices are open: the primary spelling of the firing (the form `transform` with `await`, `whenever`, `choose` and `else`, with Rholang's receipts as aliases, is the current lean), whether `Atom` means data, and which named profiles ship. The draft's remaining deviations are listed in Section 8.4.

8 MeTTa Prime

`--lang prime` is a *distinct* MeTTa language sharing CeTTa's substrate. Its ordinary application semantics is branch-local call by need with call-time choice. It is not an extension of HE, and does not reduce to HE by erasure; the "HE unchanged" gates that accompany every Prime change establish that shared code does not interfere, not a semantic relationship. Prime is also not one fixed system: it is a web of candidate type theories with carved profiles, and no statement about "Prime" may be broader than the theorem or behaviour that supports it [decided]. This section separates what the C draft does from what is a candidate, a direction or a recorded decision.

8.1 Recorded decisions

The decisions below were recorded with their sources and are not reopened without new evidence [decided].

- **Identity.** Native identity is intensional, with J and its computation rule; K and uniqueness of identity proofs are not global, but appear only in explicitly admitted scoped regions or hosted theories; proof irrelevance is a mode to switch into, never the default.
- **η.** Prime keeps η; the known defect, untyped η contraction, is repaired by type-directed η.
- **No fallback to HE's type engine** (11–12 October). HE-style declarations in Prime are answered by the proper face that grounds their fragment; outside it the answer is *undetermined*.
- **Sets are ambient, and a type is a set** (2 October). Which set theory sits at the top is under investigation; the Megalodon set theory is an opt-in profile, never the default.
- **Metavariables denote model elements** modulo the active equational theory: a repeated pattern variable means equality modulo that theory.
- **Results are unordered multisets.** Order, `once` and `select`, depth-first search, and sharing are opt-in policies; a specification fixes no evaluation order.

8.2 The language, as the draft runs it

Need cells and call-time choice. An argument is delayed in a *need cell*: an opaque runtime capability with an immutable origin and a checked cache state machine. Forcing a completed cell replays its cached bag; completed values and stable faults are memoized, while blackholes and interruptions remain retryable. Every use of a cell sees the same value, so one delayed choice used twice yields correlated outcomes (Listing 4). The same distinction is available explicitly: `(let $p (delay (superpose (A B))) (Pair (force $p) (force $p)))` answers `(Pair A A)` and `(Pair B B)`, while with `resample` in place of `delay` each force chooses afresh and all four pairs appear [draft].

Outcomes. Prime's computational zero is the absence of occurrences. The symbol `Empty` is inert data and the empty expression `()` is a unit datum: `(collapse Empty)` answers `(Empty)`. A fault is one occurrence of the outcome bag, beside the others: `(collapse (superpose ((+ 1 1) (car-atom ()) 3)))` answers `(2 (Error (car-atom ()) ...) 3)`. Internally an outcome carries the ordered occurrence bag, its completion reason and its budget state, so an incomplete frontier cannot be read as a proof of absence [draft].

Variables. Bare `$` is a fresh anonymous variable at each occurrence: `(= (both $ $) yes)` answers `yes` for `(both a b)`. Named variables co-refer: `(= (same $x $x) hit)` leaves `(same a b)` inert [draft]. Binding is lexical-fresh by default since 8 October.

Held data. Declare `(: hold (-> Atom Atom))`, `(: hold2 (-> (Data Atom) Atom))`, and give both the equation `(= (hold $x) (quoted $x))` (and likewise for `hold2`). A parameter typed `(Data Atom)` receives its argument as written: `(hold2 (+ 1 2))` answers `(quoted (+ 1 2))`. A parameter typed `Atom` does not hold it: `(hold (+ 1 2))` answers `(quoted 3)` [draft], where the HE lane would pass the expression unevaluated. Whether `Atom` should mean data is one of the open choices of Section 7.

Typing. Prime's typing has three faces, linked by the draft's fixtures: the `type:` language and its kernel (intensional), the `set:` and `hol:` judgments (extensional), and the evaluator (operational). Ordinary function contracts consult the program's selected `get-type` relation, so a program may extend the classifier with its own equations (Listing 10) without minting a checked theorem: native theorem publication still checks the proof under its context [draft].

The standard library is editable. Since 26 October standard definitions are ordinary occurrences of the program space, and `std:`-qualified names keep the originals (Section 2.8) [draft].

Memoization is opt-in. `(pragma! search-table-mode variant)` enables the variant table for the session; with it naive `fib 30` takes about 0.2 s on the draft's Prime lane (single runs). Nothing is memoized by default. An earlier ground-call table, which an earlier version of this paper described, was retired from the code on 11 August.

8.3 Proof search and native kernels

Prime hosts proof search as type inhabitation over the lazy frontier, with claim-bearing results lowered to an external checker rather than asserted by execution. From August the draft reconstructs ordinary typed constructors at the ingress of its untrusted rule machine for a strictly positive list fragment (`rel:all`, `rel:fold`, `rel:list:member`, `rel:any`, `rel:case-list`), keeping proof-tree shape, intermediate witnesses and occurrence identity; an unrecognized tree reaches the ordinary evidence-bearing check instead of being accepted [draft]. Each semantic fiber has one target checked across the native kernels; no goal is routed to a kernel by its feature [decided].

8.4 Deviations in the current draft

These are behaviours of the draft binary of Appendix B that its own semantics or recorded decisions say are wrong, each with its reproducer [draft].

1. **`match` ignores outer bindings.** A variable bound outside a `match` is fresh inside its pattern (Listing 5). `transform` respects outer bindings.
2. **Top-level reads run stored code.** After `!(add-atom &self (stored (add-atom &self (leaked three))))`, a bare top-level `!(match &self (stored $c) $c)` runs the stored `add-atom`; wrapped in an expression, or bound with `let`, the same read returns data.

3. **Library loops nest a C frame per item** (known issue PK-3). `add-atoms` over a `let`-bound list of 1,340 rows stops with `(Incomplete stack-exhausted)` after storing 1,333 of them; a `foldl-atom` whose step binds with `let` costs instructions cubic in its length.

4. **A streamed fold is quadratic in its answers** (PK-4), and **held values reach sequence operations as a tag and their syntax** (PK-5).

5. **Thirteen fixtures hang or differ** after the weights merge, reproduced on the weights line itself (Section 6.4).

Each known issue carries its reproducer, its diagnosis and a repair direction in the draft's documentation, and is removed by the change that fixes it, together with a regression test.

9 Verified HE↔PeTTa Translation

9.1 Bidirectional Translator

A bidirectional translator converts between HE and PeTTa dialects. It is implemented both as relational Prolog (`he_petta_relational.pl`, with explicit freshness threading) and as computable Lean functions (`translateHE`, `translatePeTTa`).

The translator handles 8 HE→PeTTa rewrite rules (`chain`→`let`, `collapse-bind`→`collapse`, `nop`→`let $fresh X ()`, etc.) and 5 PeTTa→HE rules (`progn`→nested `let`, `prog1`→`let` with result capture, etc.). At its last recorded corpus run, all 437 `.metta` files of the corpus then in use parsed and translated.

9.2 The Lean development

The translation is backed by a Lean development that has grown from two files to thirteen modules of about 25,000 lines. Its sources contain no `sorry`; every module was built after its last change; and the two headline theorems audited for this paper use only standard axioms (`simpleMatch_applyBindings_comm` uses none, `mettaCall_equation_step_sound` uses `propext` and `Quot.sound`).

Matching correspondence. `simpleMatch_applyBindings_comm`: if HE's `simpleMatch` succeeds on two `TwoSortTranslatable` atoms, then applying the translated bindings to the translated pattern recovers the translated target. With `matchPattern_applyBindings_complete` from the OSLF framework, a PeTTa `matchPattern` witness exists whenever HE's `simpleMatch` succeeds.

The executable translators. `translateHE_translatable` and `translateHE_preserves_pureTranslatable`: translation preserves the `Translatable` and `TwoSortTranslatable` fragments; roundtrip and stable-common theorems cover the validated fragment, including a first-order `foldall` fixed point.

Source comparisons. The largest module, about 7,200 lines, proves the completed recursive comparison of HE and PeTTa sources on an explicitly admitted pure constructor and equation fragment, and transports successful tuple and typed-argument spines through retained PeTTa source paths. Its own documentation bounds the claim: defined-call misses, error propagation, typing, effects and search policies keep separate admissions, and the transport is not whole-program open-query equivalence.

10 Reflective ρ-calculus and eval-at-comm

Under `--lang rhocalc` the same runtime stops being a term rewriter and becomes a concurrent process engine: its unit of work is not a rewrite but a *reaction*, a rendezvous between a sender and a receiver on a shared channel. This brings channels with private names and genuine concurrency; and because the calculus is *reflective*, a received name can be *run*, so a communication can itself be a computation. The MeTTa library that exposes that last capability is *rhometta*. We build up to it from the pure calculus.

10.1 The pure calculus

The calculus is Meredith and Radestock's reflective higher-order ρ-calculus [4]. Its one reduction rule, COMM, is a handshake (Figure 4): a sender `x!(m)` on channel `x` beside a receiver `for(y <- x){P}` on that same channel react; the message is consumed and the receiver resumes with `m` substituted for its bound name `y`:

$$x!(m) \mid \mathsf{for}(y \leftarrow x)\{P\} \xrightarrow{\text{COMM}} P[y := m].$$

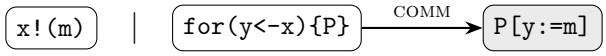


Figure 4: A reaction: a send and a matching receive on channel `x` rendezvous; the receiver continues with the message substituted in.

The rest of the vocabulary is small. `0` is the inert process; `P | Q` runs `P` and `Q` concurrently. Two reflection operators connect processes and names: `@P` *quotes* a process into a name, and `*x` *drops* (runs) the process a name quotes, with `*@P = P`. Channels are themselves quoted processes — this is what makes the calculus higher-order. CeTTa implements exactly these six strict-core constructors — `rho:nil`, `rho:par`, `rho:send`, `rho:recv`, `rho:quote`, `rho:drop` — with COMM as the only reduction. A program is reduced to quiescence by `cetta -lang rhocalc` *`file`* (Listing 13), written either in a MeTTa-style S-expression syntax (`.mrho`) or a Rholang-like syntax (`.rho`); the runtime keeps a live index of sends and receive-sites, rebuilt after each reaction. The deterministic (canonical) scheduler selects the key-least successor by streaming candidate keys against the best bound found so far instead of materializing and sorting the whole frontier, which keeps single-step selection cheap on wide parallel terms.

10.2 Reflection is enough

A surprising amount follows from COMM and quote/drop alone, with no further primitives. Each item below is a standalone `--lang rhocalc` program.

- *Name mobility.* A receive that drops its argument, placed beside a send carrying a quoted send, reduces to that inner send made live: a name has travelled across a channel and been activated.
- *Fresh names.* Quoting a distinct process yields a distinct name, so a receiver can mint and use a name it never mentioned syntactically.
- *Replicated input.* An input-guarded server can re-create its own listener: after serving a message it leaves a copy of the receiver in the residual, so the channel keeps serving. Persistent receive is *derived*, not built in.

- *Recursion and divergence.* The same self-recreation, left ungated, runs without bound (the engine reports its reduction limit); recursion needs no dedicated operator.

Private channels and information hiding come along for free: a payload exchanged on a name no other process holds is unobservable apart from whatever completion signal the protocol chooses to publish.

The capstone is *eval-at-COMM*. Let the receiver do nothing but drop what it receives:

$$\texttt{eval_drop}(x, p) \;\equiv\; \underbrace{x!(@p)}_{\text{send the quoted program } p} \;\mid\; \underbrace{\mathsf{for}(y \leftarrow x)\{\, *y \,\}}_{\text{run whatever arrives}}.$$

The COMM delivers the name `@p`; the drop `*@p` runs `p`; the process settles at the result of running `p`. A handshake has become an evaluation step, and since `p` may itself evaluate to more process, a reaction can plant the next reactions.

10.3 Profiles

A *profile* names a variant of the reduction relation together with the grammar it accepts. `strict-core` is the pure asynchronous spine above: it admits only the six core constructors, and a non-core form is *rejected* rather than left inert. The `cost` profile accepts a richer resource-annotated dialect in which every reaction must be *funded* from purses located on its channel, and every run emits a causal receipt of its spending (Section 10.7).

10.4 Certified reduction

Does eval-at-COMM report the result of running the payload, or could the machinery quietly change the answer? We settle this in Lean. Run `eval_drop(x,p)` to quiescence and collect every outcome observable *up to structural rearrangement* of the process; that set is exactly the results a correct evaluation of `p` would produce, and the statement refers only to what is observable, never to an internal "this is the answer" tag. The theorem `rhometta_bridge_scObserved` (in `RhometaReduction.lean`) equates the concrete quiescent run of `eval_drop(x,p)` with `RhometaSCObservedOutcomes` of the same process — the outcomes visible up to structural congruence — with no result-certificate carried in the answer. Two ingredients are load-bearing. First, a complete canonical-form normalizer `nfAtom`: two processes are structurally congruent exactly when their normal forms coincide, which unblocks inductions that otherwise stall on the symmetry and transitivity of congruence. Second, one hygiene side condition `noBoundUnderQuote`: the communicated variable must not occur under a quote, since a payload that hides a bound occurrence inside a quoted name defeats the characterization (a concrete counterexample exhibits exactly that failure). The development carries no `sorry`, and on 27 October the bridge theorem's axioms were exactly `propext`, `Classical.choice` and `Quot.sound`. A separate caveat on the trusted base: its conformance and DSL fixtures use `native_decide`; the bridge theorem's own module does not.

10.5 rhometta: MeTTa at the reaction

eval-at-COMM becomes a way to program once the payload is a MeTTa term. The library `lib/rhometta.metta` provides `rhometta:eval`, which defers a MeTTa term as the payload to be run at COMM; `rhometta:run`, which reduces a process to its quiescent outcome set; and `rhometta:transitions`, which returns one-step successors. It sits on `lib/rho.metta`, the strict-core constructor syntax, and delegates all reduction to the engine. One consequence is central: when a deferred payload evaluates to *several* results, the COMM forks — one successor per

result. Nondeterministic computation becomes a branching reaction, and `rhometta:run` returns one residual per branch (the *may-set*).

10.6 What reactions compute

The minimal cell. The smallest rhometta program is a single cell: a send carrying a deferred term beside a receiver that drops (runs) it. `rhometta-eval-cell` sends `(+ 2 3)` this way and the reaction leaves `5`; with a multi-result payload `(superpose (a b c))` the COMM forks into one branch per result, and `rhometta:transitions` confirms the fork is a single reduction step with three successors. Communication does computation, and result-multiplicity becomes branching.

A self-unfolding prover. Seed a *single* cell for a goal. Each COMM evaluates one backward-chaining step whose result is the cells for that goal's subgoals, so `rhometta:run` keeps reducing and the reaction network unfolds itself into a proof tree. Where a goal has several rules the payload has several results, so the COMM forks: `rhometta:run` returns one quiescent residual per complete proof tree (Figure 5) — two for the sample "can I bake a cake?" goal, while a single-rule goal stays confluent at one.

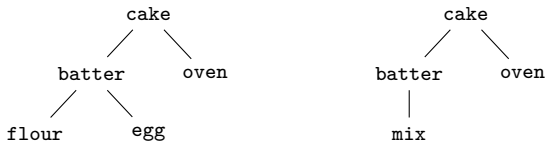


Figure 5: The self-unfolding prover's may-set for "cake": two complete proof trees, one per rule for `batter`. The trees are not enumerated up front — they are *grown* by reactions whose payloads plant their children.

Type inference as proof search. A typing judgment `(: proof (HasType e t))` is proved by reactions over typing rules, and the proof term is the derivation. In `rhometta-type-inference`, an overloaded `plus` yields a genuine may-set — an `Int` and a `Float` typing of `(plus one one)` — while `(plus x one)` with `x : Int` prunes the `Float` branch at the unification join. Proof terms stay pure, `(: (app pf pa pb) (HasType e t))` — a typing is crisp, so no truth value is attached. Each result is cross-checked differentially against a sequential chainer over the same knowledge base, which derives the same proof-term skeletons.

A truth value that is earned. Crisp proof terms are the right output for a typing, but some judgments are genuinely probabilistic, and there a reaction can carry evidence rather than certainty. In `rhometta-evidence-revision`, two agents each count their own observations of "ravens are black" into an opinion — a strength and a confidence — computed at COMM by a deferred payload. The two agents share two observations, so merging their opinions *naively* double-counts the shared evidence and overstates the confidence (0.889); an overlap-corrected revision discounts the shared stamps for the honest value (0.857), at the same strength. The revision can run after the parallel fan-in, or *at* the final rendezvous itself. This is the one place a truth value belongs — it is counted from data, not attached by hand.

Composition. These pieces compose: a further program runs an independent proof search, an overlap-corrected evidence revision, and an attention-spreading step concurrently, then rendezvouses their channel-gathered results into a single synthesis.

10.7 Cost-accounted reactions

A reaction is the natural unit of metering: each COMM is one indivisible piece of work. The `cost` profile makes that intuition operational — and it does so with money rather than a stopwatch. Processes are *signed*, channels carry *purses* of tokens, a reaction fires *only if it is paid for*, and what a run returns is not a number but a *receipt*: a causal, auditable record of every spend. The profile is the executable part of the programme to equip the ρ-calculus with compositional resource accounting [5, 6]. Its design commitments are locality of funding, causality from consumption, and an explicit separation between exact authority receipts and lossy numerical valuations.

Purses and funded reactions. In the `.rho` syntax, `{P}alice` is the process `P` signed by `alice`, and `purse pay {alice : ()}` is a store on channel `pay` holding one token signed by `alice`. The funding rule is *located*: a COMM on `pay` consumes one token per participating endpoint signature, and those tokens must be found in purses *on `pay` itself*. There is no ambient bank account, and raw process data never funds anything. The smallest funded program has two signed endpoints and two purses (`tests/rhocalc_cost_run/split_tokens_basic.rho`):

```
{for ($m <- pay) {{0}cont}}alice | {pay!({0}payload)}bob
  | purse pay {alice : ()} | purse pay {bob : ()}
```

which reduces to `purse pay {()} | purse pay {()} | {0}cont` (rerun on 27 October on the pre-main binary, with this and the next two examples' expected outputs reproduced exactly): each signature paid for its own endpoint, and the two purses are left empty beside the continuation. Now delete `bob`'s purse (`missing_token_quiescent.rho`) and the program is *quiescent with the send and receive still facing each other*: the reaction is not slowed, it is *disabled*. This is the first thing to internalize about the profile — cost is not an observer bolted onto the side; an unfunded reaction cannot fire. Because channels are quoted processes, a purse's location follows the calculus's own name equivalence rather than spelling: a purse on `@{*pay}` funds a reaction on `pay`, since the two names denote the same channel (`name_equivalence_quote_drop.rho`) — aliases of a channel are the same account.

Receipts are causal records, not counters. A single number would forget who paid, what was consumed, and what depended on what — and its intermediate readings would depend on which interleaving the scheduler happened to choose. Instead, every reaction emits an event record with four fields: a fresh event identifier; a *cause list* — the identifiers of the events that produced the exact occurrences this event consumed, with repetitions kept (consuming two outputs of event 0 yields the cause list `(0 0)`); a nonempty list of *funding* records naming the purse location and head signature of each token spent; and the consumed signature itself. The two-reaction chain below (the first case of `tests/test_lts_rho_cost_causal_trace.metta`) shows the shape — event 1 consumes both a continuation and a purse tail produced by event 0, so its cause list holds two arcs to 0:

```
(lts:rho:cost:receipt
  ((lts:rho:cost:event 0 ()
     ((lts:rho:cost:funding chain spend)) spend)
   (lts:rho:cost:event 1 (0 0)
```

```
    ((lts:rho:cost:funding chain spend)) spend))
  (rho:cost:par
    (rho:cost:purse chain rho:cost:stack-empty)
    (rho:cost:signed rho:nil done)))
```

The algebraic reading is the load-bearing one. A receipt is a *located, measured pomset* of spend events: a partially ordered multiset whose order is generated purely by consumption — event e' follows e exactly when e' consumed something e produced. The engine's emission order is one linearization of that partial order. Any permutation of one fixed compatible matching yields the same target and receipt bag; the emitted sequence linearizes that matching's causal order. Alternative endpoint or funding matchings remain distinct branches. The Lean theorem `boundedBudgetedCausalPrefix_emission_linearizes` confirms that a budgeted prefix's emitted sequence linearizes its causal order. Locations make purses *fibers* over channels: funding never crosses a channel, and each per-location account is conserved (`CostPath.per_location_restriction_account_conservation`). Finally, the receipt itself carries no notion of "the cost": costs are *valuations* — monoid-valued maps out of the receipt (total spend, per-signature spend, critical-path depth) — so the receipt is the free object and each accounting policy is a homomorphism out of it, with the lawful combination rules derived rather than posited, in the sense of Knuth and Skilling's measure-assignment axioms [12]. Event identifiers are deliberately polymorphic in the same spirit: a local counter suffices in one process, and a hash-chained identifier can replace it at trust boundaries with the receipt algebra unchanged.

Honest budgets. Everything above is unbounded by default: `lts:rho:cost:steps` and `lts:rho:cost:causal-trace` enumerate exactly and never truncate silently. Bounded execution is strictly opt-in through `lts:rho:cost:causal-prefix`, which takes two separate allowances — reaction fuel and exact-cover search work — and returns a receipt together with a status that never lies: `quiescent` only after exhaustive search *proves* the residual has no funded successor; `fuel-exhausted` when the firing allowance ends first; `search-exhausted` when cover search stops before finding a successor *or proving absence*. Search exhaustion is never reported as quiescence: an out-of-fuel machine says "I ran out," never "there was nothing there." The distinction has teeth because funding search is a genuine exact-cover problem. A send whose signature product multiplies ten `coin`s, beside twenty unit purses on the same channel, admits $\binom{20}{10} = 184{,}756$ distinct covers (`tests/test_lts_rho_cost_search_budget.metta`):

```
; Zero reduction fuel performs no exact-cover search, even when
; the latent cover family has C(20,10) members.
!(assertEqualToResult
  (test:prefix-status
    (lts:rho:cost:causal-prefix 0 0 (test:large-cover-state)))
  (lts:rho:cost:fuel-exhausted))

; The first exact occurrence cover is yielded without
; materializing the other 184,755 covers.
```

At fuel 0 the machine performs no search at all; with a small search allowance it yields the first cover lazily; and the unlimited frontier remains available and exact — all 184,756 successors, measured at about 11 s — combinatorial by design and by request, never by accident. A budget-monotonicity gate (59 paired runs at increasing allowances) checks that raising an allowance never flips a determinate verdict and never removes a receipt event: more fuel can only extend what is known.

Certified correspondence. The formal side lives in the `Cost/` theory in the Mettapedia repository: cost steps, paths, receipts, valuations, conservation laws, and a budgeted executor, proven equal to one another where they meet. The capstone `BudgetedPrefixPathRefinement` states that every budgeted machine run produces a prefix of the occurrence-bearing Lean executable `CostPath`, with the same receipt and the correct status, and its corollaries pin each verdict separately: sufficient allowances yield an actual path; exhaustive candidate failure — unlike bounded exhaustion — certifies quiescence; zero fuel and zero search-work report their own exhaustion and make no negative claim. The machine frontier is proven sound and complete up to structural equivalence (`costStep_sound_runtime`, `costStep_complete_runtime_up_to_struct`); these are the separate one-step correspondences with the declarative `CostStep` relation. The compositional theorem `budgetedCausalPrefix_exists_rhoLanguageDefRewritePath` closes the corresponding path square for the Lean executor: every publicly admitted budgeted run yields an occurrence-bearing runtime path, a declarative funded trace, and a rewrite path in the pure GSLT mechanically derived from `rhoCalc`, with exactly one pure step per runtime firing.

The pure-rho boundary is theorem-backed as well. The source rho `LanguageDef` supplies the strict syntax and primitive COMM definition; its compiled matcher, normalization, substitution, and supported COMM contracta are proved to agree with the independently established semantic development. The pure GSLT used for erasure is itself derived from that source declaration; full equivalence of its complete contextual relation with the independently established relation remains a named theorem obligation. On the derived binder-safe fragment, initialization and funded reduction preserve scope; each declarative `CostStep` erases to one derived pure COMM, and each declarative funded or executable path erases to a derived rho rewrite path whose length is exactly its ordered demand-receipt length. The equivalence used here is pure structural congruence: no executable free-Drop step is admitted. The canonical configuration erasure is defined by quotient lifting rather than by a noncomputable representative choice. The pure-erasure binder-safety judgment covers exactly syntax observable by pure erasure. The separate public runtime-scope check validates every raw field, including purse locations, and is proved to imply the erasure-domain judgment. A rerunnable Lean audit prints the axioms of the exported results; they use only Lean's standard quotient and classical infrastructure, with no project axiom and no unfinished proof.

On the compiled side, 146 sequential/threaded prefix cases agree, and 308 compiled-C receipts replay as Lean derivations while two altered receipts are rejected. The complete runtime suite passes in the ordinary, thread-sanitized, and address/undefined-behavior sanitized builds. The differential harness (`scripts/rhocalc_cost_differential.py`) drives the same terms through the C engine and the Lean executor, including fuel-0 and fuel-1 boundary cases, with an out-of-fragment case correctly rejected. This remains bounded differential evidence about the compiled engine, not a universal theorem about C.

Parallel funded execution. Compatible reactions execute in OS threads in waves. Before a worker can fire, it atomically claims the exact endpoint occurrences and the exact funding-cover occurrences in a fixed resource order; a wave admits only pairwise disjoint claims. The sequential semantics therefore remains the reference, while the threaded implementation is tested as a legal parallel refinement: both threaded CLI modes, six contention and disjointness stress families, and the sequential/threaded prefix gate exercise same-channel disjoint reactions, competing funding covers, duplicate-looking occurrences, and causal receipt linearization. Receipt emission is an *optional observer* on this path: operational components are plain terms, causal provenance rides in a parallel producer vector, and a scripted transparency gate checks that a state-only run and a receipt-observed run consume identical resources and reach the same residual. These gates,

together with thread and memory sanitizers, are implementation evidence; the Lean step/path theorem above supplies the declarative semantic boundary.

At the modeled parallel boundary, a list of enabled runtime candidates whose complete endpoint-and-funding source-index family is globally duplicate-free lifts to a `SelectedRuntimeWave`. Its witnessed source partition constructs an exact `CostMatching`, and every permutation of that fixed matching serializes as an ordinary declarative cost trace. This discharges the runtime-model wave-to-matching square under the occurrence-claim postcondition; the compiled-C correspondence remains the bounded differential, receipt-replay, and sanitizer evidence stated above rather than a universal theorem about C.

A strict source LanguageDef and the corrected algebra. The profile implements funded rho and no Datalog architecture. Its strict syntax and primitive COMM definition come from the source `LanguageDef`; the compiled artifacts are proved to agree with the established pure semantics on supported COMM contracta, while the GSLT used for the cost-erasure theorem is mechanically derived from that source declaration. Other implementations are bounded encoding references, and their executable free-Drop rule is not part of this profile.

The formalization also identifies why the original ordinary-monad faithfulness claim cannot be repaired by choosing a more cancellative signature monoid. Joint injectivity of a monoid multiplication together with a two-sided unit forces the carrier to be a subsingleton; words, multisets, and numerical costs all forget some factorization. The landed repair is an Atkey-style pre/post-indexed semantics [7]: a funded execution has complete source and target resource states, so incompatible funding transitions are untypeable. Exact authority receipts compose without erasure; price and budget totals are explicit, generally non-injective observations of those receipts. This is the resource-transition specialization of the wider graded and parameterized-effect account [8, 9]. The companion theory paper states the obstruction, the indexed laws, and the authority/observation factorization precisely [6].

The continued category and repeated Cost. Since the July development, the Lean side (`GSLT/LanguageDef/ContinuedCategory.lean`, `Cost/Construction.lean`) treats Cost not as a one-off wrapping of rho but as a construction on *continued interactive GSLTs*: a language mechanically derived from one source `LanguageDef`, equipped with one ordered interaction cut (a contact constructor plus program and environment operands with selected continuation positions), a computable canonical section of every typed open fiber, and a *position-sensitive continuation-retyping plan* under which every non-principal constructor receives a hereditarily wrapped copy while the two principals receive cut-aware base copies whose continuation parameter alone moves to the wrapped fiber. The generated funded language — the base/wrapped signature plus a small contact/signing/funding apparatus and exactly one whole-redex rewrite that consumes a funding-stack head and carries the tail through — is itself such an object, and the object-closure laws of the resulting endofunctor programme are largely discharged generically: bare-collection constructors of the generated language are provably non-principal at the next layer, the generated interaction envelope is stable under the next retyping, a single declaration-derived typing morphism validates the next layer's signature, and the typed transport along a retyping plan is proved over the minimal non-circular signature (theory, cut, plan). The remaining generated-object laws are active work with identified routes. Two corrections landed on the way and are recorded as compiled counterexamples rather than folklore: the global raw fiber-stability hypothesis of the earlier normalization architecture is *false* for rho (a mixed-colour reflective generator has a typed left endpoint whose canonical right endpoint is ill-typed), and its repair carries the entire equational theory on

typed open carriers with proof-relevant region-tree elaboration — raw-to-typed lifting exists only edge-wise, with both endpoints certified in the same typed fiber. Exact typed canonical laws then cut out the full subcategory of *Cost-canonical* objects, which, ordered by collision-free structural canonical keys, is the honest domain of strict Cost_1. The theory paper develops all of this precisely [6]; the engine consequence is concrete: the same source declaration that generates this profile's grammar and its single funded rewrite is the object the endofunctor consumes, so iterated cost-accounting is a construction on the specification, never a second hand-written engine. The public API is `lib/lts/rho/cost.metta`; the engine is `src/rhocalc_core.c`. Runnable showcases ship with the engine in `examples/rho/` — a funded vending machine (an unfunded order left facing a live endpoint), a causal notary chain whose receipt is the provenance audit trail, an honest-budget meter exhibiting all three prefix verdicts and a two-cover spend, and a threaded settlement wave whose residual is byte-identical at every thread count — each with expected outputs checked by the test suite, alongside a documented walkthrough of the calculus, both syntaxes, and the profile.

11 Historical Cross-Runtime Benchmarks

The rows in this section were measured between June and September 2026, on the commits named with them, and are kept as dated receipts; several engines and lines have changed since. The October measurements of each line of development are in Section 5.

Benchmark artifacts are labeled by dialect route. A row may be *shared* (one dialect-agnostic source), PeTTa-*native*, HE/CeTTa-*native*, mechanically translated, or hand-normalized after translation to expose the same algorithmic contract in both dialects. HE and PeTTa are different MeTTa dialects, so a real comparison is not "whatever file happens to run"; it is a named pair of artifacts plus a stated output contract. Timing rows in this section were measured on 2026-06-27 (the pre-GC tree), except the grounded genomic-inference rows (PLN evidence core, full WM-PLN raw-loader, mine→chain focus, and deep genomic backward chaining), which were re-receipted on 2026-07-02 on commit 9237afd; rows explicitly described as excluded, methodological, or unresolved carry their own status. The two heavy genomic rows (full WM-PLN raw-loader and deep genomic backward chaining) were re-measured on a quiescent host with the same binary at commit 9237afd; because the two engines are always compared within one session, the cross-engine ratios are the load-bearing claims. For rows whose single-run wall time is below 0.1 s, we report averaged full-process wall time over repeated invocations to avoid timer quantization. Output is verified identical across runtimes whenever a row is presented as a clean cross-runtime benchmark. So a reader can see precisely what is measured, each subsection below opens with the core of the program it runs; the full sources are linked inline and collected at `benchmarks/chaining_crossengine`.

Most of the inference benchmarks are driven by one small dependent-typed *backward chainer*. To prove a goal `(: $proof $theorem)` it either matches a fact in the knowledge base, or—for a rule application `($rule $prem)`—looks up the rule's type and recurses on its premise (the binary and n-ary cases follow the same shape):

```
(: bc (-> $a Nat $b $b))                                  ; KB, depth, goal -> goal
(= (bc $kb $_ (: $proof $theorem))                        ; base case: it's a fact
   (match $kb (: $proof $theorem) (: $proof $theorem)))
(= (bc $kb (S $d) (: ($rule $prem) $theorem))             ; step: apply a rule
   (let* (((: $rule (-> (: $prem $t) $theorem))
           (bc $kb $d (: $rule (-> (: $prem $t) $theorem))))
         ((: $prem $t) (bc $kb $d (: $prem $t))))
     (: ($rule $prem) $theorem)))
```

Each benchmark below supplies its own typed rules and facts to this chainer (or, where noted, a forward chainer or a truth-value kernel).

11.1 Shallow Evaluation

Nil Geisweiller's typed backward chainer (the `bc` above) over 60 ground entities at depth 1 (20 researchers, 20 engineers, 20 artists). The rules are typed binary implications; for example, `r-mortal` combines a researcher fact with a curious fact to derive a mortal witness:

```
(: r-mortal (-> (: $r (researcher $x))
                (: $c (curious $x))
                (mortal $x)))
!(bc &kb (fromNumber 1) (: $proof (mortal $x)))      ; -> 60 witnesses
```

All three runtimes produce identical 60 proof-carrying witnesses, e.g. `(: (r-mortal r01 r01c) (mortal r01))`.

Runtime	Time	vs. CeTTa	Output
CeTTa	4.0 ms avg	1×	60 witnesses
PeTTa	45.4 ms avg	11.3× slower	60 witnesses
HE	402 ms avg	100× slower	60 witnesses

This is a startup-dominated sanity row, not a scaling claim. It is useful because it exercises the typed proof-carrying API while remaining small enough that translator and runtime drift are easy to spot.

11.2 Deep Proof Search: PeTTa Wins

Metamath proof search at depth 5 (2 axioms, proving $t = t$, 85,013 proof trees) reverses the performance ordering. The knowledge base is Geisweiller's `demo0` (axiom `a1`: equality is right-Euclidean; `a2`: 0 is a right identity; `mp`: modus ponens), and the goal is `t = t` (`01_nilbc/nilbc_-th1.metta`; glyphs shown ASCII here):

```
(: a1 (-> (: $t term) (: $r term) (: $s term)
          (-> (= $t $r) (-> (= $t $s) (= $r $s)))))   ; right-Euclidean
(: a2 (-> (: $t term) (= (+ $t 0) $t)))               ; 0 right-identity
(: mp (-> (: $maj (-> $P $Q)) (: $P wff) (: $Q wff)
          (: $min $P) $Q))                            ; modus ponens
!(bc &kbh (fromNumber 5) (: $prf (= t t)))            ; prove t = t
```

Runtime	Time	vs. PeTTa	Status
PeTTa	7.60 s	1×	85,013 proofs
CeTTa	36.42 s	4.8× slower	85,013 proofs
HE	>120 s	timeout	killed

PeTTa's Prolog search remains decisively better on deep branching proof search; the recorded rerun widens the gap to roughly 4.8×. HE did not finish within that recorded 120 s run.

11.3 Forward Chaining at Scale

Nil Geisweiller's propositional-calculus forward chainer at depth 4 uses a solution space that is extended in place by forward application of modus ponens:

```
(= (fs-add-to-ss $rb $ss)
   (with-space-snapshot $ss0 $ss
     (match $ss0 (: $x1 $a1)
       (match $ss0 (: $x2 $a2)
         (match $rb (: $f (-> $a1 $a2 $b))
           (add-atom $ss (: ($f $x1 $x2) $b)))))))
!(count-atoms &ss)
```

The recorded CeTTa rerun of `tests/nil_pc_fc_d4.metta` completes in 100.86 s and reports 8.19M accumulated results after the four forward passes. We do *not* reuse the older "11.7M generated pairs" sentence as benchmark evidence, because the recorded HE lane did not time out on this driver; it errors immediately (`match expects a space as the first argument`). So the forward row remains a useful workload, but it is not yet paper-ready as a clean cross-runtime benchmark.

11.4 Translator Validation

A PeTTa-native benchmark using `prog1` (PeTTa-specific construct) was translated PeTTa→HE and run on CeTTa. The translator converts `prog1` to nested `let` with fresh variables:

```
; PeTTa original:
(= (run-query) (prog1 (bc &kb ...) (println! "done")))
; Translated to HE:
(= (run-query) (let $__tr_result_1 (bc &kb ...)
                 (let $__tr_discard_2 (println! "done")
                   $__tr_result_1)))
```

The recorded source file `benchmarks/bench_backchain_petta_native.metta` produces 30 proof-carrying witnesses on native PeTTa, and the recorded translated artifact produces the same 30 witnesses on both CeTTa and HE:

Runtime	Time	vs. CeTTa	Output
CeTTa	3.7 ms avg	1×	30 proofs
PeTTa	44.8 ms avg	12.1× slower	30 proofs
HE	319 ms avg	86× slower	30 proofs

This row is a translator benchmark, not just a syntax example: the observable runtime behavior survives translation.

11.5 PLN Evidence Aggregation over Genomic KB

To benchmark inference over real data, we run PLN evidence aggregation over the Rejuve genomic knowledge base (chr16 pilot: 41 GWAS SNPs, 183 SNP–gene pairs, 3 evidence channels). Each pair carries eQTL tissue counts (GTEx, 49 tissues), ABC enhancer–gene contact counts, and a regulatory presence flag. The PLN evidence quantale combines channels via revision (`evidence-hplus`), producing a calibrated SimpleTruthValue per pair. The evidence operations are Lean-verified: `power`, `toLogOdds_power_mul`, and `power_power_inv` are proved in `Mettapedia/PLN/Evidence/BinaryEvidence.lean`.

The core arithmetic is ordinary MeTTa over numbers:

```
(= (Truth_ModusPonens (stv $f1 $c1) (stv $f2 $c2))
   (stv (+ (* $f1 $f2) (* 0.02 (- 1 $f1))) (* $c1 $c2)))
(= (Truth_Revision (stv $f1 $c1) (stv $f2 $c2))
   (let* (($w1 (Truth_c2w $c1)) ($w2 (Truth_c2w $c2)) ($w (+ $w1 $w2))
          ($f (/ (+ (* $w1 $f1) (* $w2 $f2)) $w)) ($c (Truth_w2c $w)))
     (stv (min 1.0 $f) (min 1.0 $c))))
```

The benchmark is dialect-agnostic (shared MeTTa, no translation needed). The portable `benchmarks/bio_pln` evidence-score drivers produce numerically identical 183 scored pairs on CeTTa, PeTTa, and HE:

Runtime	Time	vs. CeTTa	Output	Status
CeTTa	10.9 ms avg	1×	183 bio-scores	identical
PeTTa	58.1 ms avg	5.4× slower	183 bio-scores	identical
HE	7.56 s	696× slower	183 bio-scores	identical

Scores carry genuine biological signal: `rs9923732→ENSG00000166455` scores strength 0.7348066298342542 (133/181) and confidence 0.9945054945054945 (181/182), while most pairs score 0.04 (minimal evidence). All three engines reproduce every one of the 183 scores to full numeric precision on the portable drivers; the earlier non-portable driver diverged only because upstream HE integer-divided the evidence ratio to zero and PeTTa left imported evidence-library calls unreduced. Exact three-engine numeric agreement is this row's validation story.

What we no longer claim here. Earlier drafts included a synthetic scaling table from 143 to 1.4M atoms. We still do not treat that older synthetic table itself as benchmark evidence in the paper. What *is* now grounded again is the true full raw-loader checkpoint lane: the fresh CeTTa and PeTTa reruns both execute the same TFBS-inclusive 1,399,997-line data tier and agree on the labeled observable contract (`9` seed targets, `9` contextual targets, `85` disease proofs, `34` supported-drug proof witnesses, and `8` unique supported-drug hypotheses). The explicit proof-count vs. hypothesis-count split matters here: the `34` traces are genuinely multiple proof routes collapsing to `8` unique `(gene,disease,drug,tissue)` hypotheses, not a stale counting mistake. Fresh timings on this grounded full row are 16.4 s on CeTTa and 44.5 s on PeTTa (about 2.7× faster). Upstream HE did not in that run run this loader shape directly: its `metta` runner rejects the relative-file `import! ./support/...` syntax as an illegal module name, so the HE column for this row remains a loader blocker rather than a performance number. The 183-pair core therefore remains the grounded small PLN row, while the full TFBS-inclusive checkpoint is now the grounded large WM-PLN row.

11.6 Genomic Pattern Mining

To exercise real algorithmic depth, we run the Hyperon pattern miner [1] on real GTEx eQTL data, discovering surprising co-regulation patterns—pairs of SNPs or genes that share eQTL evidence more often than expected by independence. The miner is run unmodified from github.com/iCog-Labs-Dev/hyperon-miner; its core loop alternates `build-specialization` (extend a candidate pattern by one link) with a `surprisingness` score that keeps only patterns whose support exceeds the independence expectation. The data is extracted from the Rejuve genomic KB (338M Prolog lines, 57 GB) and discretized into categorical attributes (tissue breadth, effect size) for pattern mining.

The recorded audit does *not* reuse the older cross-engine mining speed table as benchmark evidence, and no current in-tree driver re-emits the earlier specialization-core atom count, so we do

not carry that number forward. We keep the pattern miner as a methodological workload because it contains stateful `superpose`: a remove/add pattern that mutates a shared space in the middle of a non-deterministic branch family:

```
(superpose ((remove-atom $specspace $spec1)
            (add-atom    $specspace $spec1)
            (remove-atom $specspace $spec2)
            (add-atom    $specspace $spec2)
            (remove-atom $specspace $spec3)
            (add-atom    $specspace $spec3)))
```

The earlier paper classified one observed traversal as correct and another as atom loss. Without a current driver and an explicit effect-order contract, that conclusion is not retained. The case instead motivates the factorization in Section 14: compare complete result bags and multiplicity where the observer is a bag, record effect traces where effects are observable, and require an explicit mutex, transaction, snapshot, or sequence policy when the program needs a stronger order. The miner therefore remains a portability and stress workload, not a fresh cross-engine speed or scheduler-correctness table.

11.7 Backward Chaining over Mined Patterns

The mined co-regulation patterns serve as typed inference rules for Geisweiller's `nilbc` backward chainer (the same dependent-typed `bc` listed at the start of Section 11), closing the *mine→chain* pipeline. The knowledge base combines:

- 100,000 real GTEx eQTL links (40,484 SNPs × 14,043 genes),
- 13 co-regulation patterns and 7 co-expression patterns (mined, surprisingness > 0.98),
- 3 typed PLN rules: *direct-target* (eQTL → target), *coreg-transfer* (co-regulation + target → target), and *coexpr-transfer* (target + co-expression → target).

The three rules are typed implications fed to the `bc` chainer of Section 11; *direct-target* reads an eQTL fact, *coreg-transfer* chains a co-regulation edge onto an existing target:

```
(rule-sig dt (-> (: $e (eqtl $snp $gene))            ; direct-target
                 (target $snp $gene)))
(rule-sig ct (-> (: $e1 (coreg $snp1 $snp2))         ; coreg-transfer
                 (: $e2 (target $snp2 $gene))
                 (target $snp1 $gene)))
!(bc &kb (fromNumber 2) (: $proof (target rs10000544 $gene)))
```

Bridge base cases let `nilbc` consume untyped data atoms (`eqtl-link`, `co-regulate`) directly, avoiding a format conversion pass over the 100K-atom KB.

On the portable `benchmarks/mine_chain_focus` focus-summary drivers, CeTTa and PeTTa agree exactly on the distinct direct and depth-2 target genes for three representative query SNPs (the raw proof rows dedupe by target gene to these counts):

Query SNP	direct targets	depth-2 targets	novel
rs10000544	9	12	3
rs10000620	5	5	0
rs10000960	6	8	2

The five novel depth-2 targets are:

Gene	Symbol	Via SNP	Evidence path
ENSG00000150961	SEC24D	rs10000795	`(ct cr3 (dt eq37))`
ENSG00000172399	MYOZ2	rs10000795	`(ct cr3 (dt eq39))`
ENSG00000178636	MRPL42P1	rs10000726	`(ct cr1 (dt eq30))`
ENSG00000248394	FOSL1P1	rs10001048	`(ct cr25 (dt eq87))`
ENSG00000273077	(lncRNA)	rs10001048	`(ct cr25 (dt eq89))`

The proof terms are machine-checkable evidence chains: `(ct cr3 (dt eq37))` reads "co-regulation transfer via `cr3` (rs10000544↔rs10000795), applied to direct eQTL `eq37` (rs10000795→SEC24D)." On these focused three-query drivers the total wall-clock time is 0.13 s on CeTTa versus 4.27 s on PeTTa. We therefore keep the novel-gene discovery story, but we do *not* reuse the older proof-count/time row as benchmark evidence in this paper.

Biologically, the novel discoveries include MYOZ2 (hypertrophic cardiomyopathy gene, OMIM CMH16) and SEC24D (Cole-Carpenter bone syndrome), both on chr4q26—suggesting a trans-regulatory link from a metabolic locus (PDE5A/FABP2) to cardiac and skeletal gene targets via shared cis-regulatory architecture.

The older "all 7 query SNPs plus reverse queries" timing line is not used as a fresh paper benchmark here; the audited evidence in this revision is the focused cross-runtime driver above.

Complete benchmark ladder. The following table summarizes all benchmarks, including both genomic inference (where CeTTa wins) and pure search (where PeTTa wins):

Benchmark	Scale / note	CeTTa	PeTTa	HE
Shallow typed BC	60 proof witnesses	4.0 ms avg	45.4 ms avg	402 ms avg
Translator (`prog1`)	30 proof witnesses	3.7 ms avg	44.8 ms avg	319 ms avg
PLN evidence core	183 scored pairs	10.9 ms avg	58.1 ms avg	7.56 s
Full WM-PLN raw-loader	1,399,997 lines, 34 proofs / 8 hyps	16.4 s	44.5 s	import block
Mine→chain focus	3 query SNPs, 5 novel genes	0.13 s	4.27 s	—
Deep 17-phase genomic BC	1,411,430 hypotheses	46.33 s	64.05 s	—
8-Queens (direct shared lane)	92 solutions	8.22 s	0.05 s	2:08.77
Deep proof search	85,013 proofs	36.42 s	7.60 s	>120 s

Peak resident memory on the two heavy rows. On the full WM-PLN raw-loader, CeTTa uses 2.10 GiB against PeTTa's 4.75 GiB (CeTTa lower). On the deep 17-phase genomic BC the direction reverses: CeTTa uses 14.5 GiB against PeTTa's 4.85 GiB—about 3.0× PeTTa's footprint, the memory-multiplicity cost of CeTTa's answer materialization on the high-fan-out row.

CeTTa still wins on several real inference rows, but the revised ladder is intentionally smaller than earlier drafts. In the 2026-06-27 audit we excluded the historical cross-engine miner speed table, the old full-1.4M WM-PLN scaling table, and BTree from this cross-engine ladder because at that point the tree either lacked a stable current cross-runtime driver for them or exposed a correctness/comparability issue. The tilepuzzle row has since been rerun and is reported separately as a search benchmark and GC capacity result in Section 12.3. The benchmark rows shown here are only those rerun directly in that audit.

11.8 Cross-Engine Chaining Suite

To ground the inference-side comparison on community-written code rather than bespoke benchmarks, we assembled a cross-engine chaining suite that builds matched cross-engine

rows from the same chainer families on CeTTa (`--lang he`) and PeTTa, with a per-row cross-engine contract (every result is validated against the other engine's output, never against CeTTa's own output). The sources are Geisweiller's dependent-typed DTL chainers and Metamath `demo0` proofs (github.com/ngeiswei/chaining), Treutlein's `PeTTaChainer` (github.com/rTreutlein/PeTTaChainer), and the PeTTa engine and its PLN library (github.com/patham9/PeTTa). The suite, generators, and driver are in the CeTTa repository under `benchmarks/chaining_crossengine`. In this paper revision we use the suite as audited reproducibility material rather than as an extra source of unevaluated claims: the benchmark rows retained in the paper are only the subset rerun directly in this audit.

Multi-relational deep chaining. The strongest recorded multi-relational row is the 17-phase snapshot `tests/bench_bio_bc_deep_snapshot_20260327.metta`. It extends the mined-pattern chainer with pathway membership, same-pathway transfer, disease associations, compound–disease treatment links, and compound–gene binding links. The full variant-to-drug row is still a typed backward-chaining program:

```
(rule-sig dg (-> (: $e1 (target $snp $gene))
                 (: $e2 (has-disease $gene $disease))
                 (disease-reach $snp $disease $gene)))
(rule-sig td (-> (: $e1 (disease-reach $snp $disease $gene))
                 (: $e2 (treats $compound $disease))
                 (drug-hypothesis $snp $compound $disease $gene)))
```

The recorded rerun produced exactly **1,411,430 hypotheses** across 17 query phases, identical on both runtimes, with the bucket breakdown:

Hypothesis bucket	Count
`pw-target`	613,978
`deep-pw`	796,964
binding	11
disease	11
drug	210
deep-drug	240
pathway-reach	16
Total (17 phases)	1,411,430

Runtime	Wall time	Peak RSS	vs. PeTTa	Output contract
CeTTa	46.33 s	14.5 GiB	1.38× faster	1,411,430 hypotheses
PeTTa	64.05 s	4.85 GiB	1×	1,411,430 hypotheses

The recorded timings still favor CeTTa, but the table makes the memory tradeoff explicit: this high-fan-out row is one where CeTTa materializes many answers and pays roughly 3× PeTTa's peak footprint. The textbook chain rs10000544 → PDE5A → hypertension ← Sildenafil composes three independent data sources (GTEx, Hetionet, DrugBank) by typed backward chaining with proof terms at every step.

Dependent binder encoding. PLN rule declarations use `(rule-sig name type)` instead of `(: name type)` to avoid a HE specification limitation in that version: the `(: $e T)` syntax in arrow-domain positions is treated as a literal expected type, not a dependent binder, causing `BadArgType`

on both HE and CeTTa (see Section 15). The `rule-sig` encoding is semantically equivalent on all runtimes.

11.9 Four-Engine Study: JeTTa and MeTTaScript Join the Ladder

A 2026-07 study extended the ladder with JeTTa (a Kotlin/JVM ahead-of-time compiler for MeTTa) and, via its upstream slide deck, MeTTaScript (TypeScript) [10, 11]. Every timed run was gated on answer correctness; a fast wrong answer counts as a failure. Three findings matter for CeTTa's roadmap:

- **Competence zones, not a single ranking.** JeTTa dominates deterministic arithmetic recursion (its compiler auto-memoizes pure recursion: naive Fibonacci runs in $O(N)$); PeTTa dominates unindexed nondeterministic search (8-Queens roughly 150× over CeTTa) and is the only engine with production tabling and bignums; CeTTa keeps near-zero startup and wins indexed knowledge-base queries (the mined-chain workload roughly 26× over PeTTa); on the heaviest typed proof search in the study (the 85,013-proof Metamath depth-5 run) PeTTa completes in 13.0 s under its shipped 8 GiB stack configuration versus 62.7 s for CeTTa — the 4.8× margin matching the earlier audited row. An initial rerun under SWI-Prolog's 1 GiB *default* stack overflowed and was briefly misrecorded as an engine failure; the harness, not the engine, was at fault.
- **The per-call and sharing taxes, measured.** On naive Fibonacci the per-call interpretation costs were approximately 0.23 μs (PeTTa, WAM) versus 9.6 μs (CeTTa tree walk) — a $\approx 42\times$ machine tax — while the tabling three-way showed declared tabling and auto-memoization both answering `fib 46` in 0.10 s where the untabled tree walk extrapolates to tens of hours. These are the evidence rows behind the abstract machine hypothesis (Appendix on MAM) and the call-keyed sharing direction in Prime.
- **Methodological hazards.** Auto-memoization silently turns repetition-scaled benchmarks into cache-lookup measurements; one engine returned fast *wrong* sums through silent 32-bit wraparound; and a harness that launches an engine without its shipped resource configuration (here, SWI-Prolog's 1 GiB default stack in place of PeTTa's configured 8 GiB) manufactures failures that read as engine limits. All three hazards argue for correctness-gated, semantics-aware benchmarking rather than wall-clock tables.

11.10 Pinned Candidate Revalidation

The four-engine conclusions above were rechecked on ordinary optimized binaries against a fixed CeTTa candidate (`7707cef`), JeTTa `c95d87d` (version 0.7.0), and the corresponding pinned PeTTa, HE, and MeTTa-on-MORK revisions. Timed cells were admitted only after exact answers agreed; reported values are medians of five interleaved runs where the harness supports repetition. Sanitizer runs, failed cells, and semantically unequal cells are not timing evidence.

Shared-program conformance. Twenty of twenty-one selected HE programs produced identical output bags on HE with GroundingSpace, HE with MORK, CeTTa native space, and CeTTa explicit PathMap space. The remaining documentation example failed on both HE stores and was excluded rather than attributed to a CeTTa backend. A separate duplicate-atom discriminator exposed the important carrier boundary: HE's MORK store collapsed three insertions to set support, whereas both CeTTa spaces preserved the two requested occurrences in the query result. Thus

PathMap agreement is not merely cardinality agreement: CeTTa's counted encoding preserves the language's bag contract.

At the pinned `7707cef` candidate, the command-line `--space-engine pathmap` route stored only the bootstrap portion in PathMap and then fell back to the ordinary evaluator space. The tables in this historical subsection therefore used explicit `(new-space pathmap)` objects. This limitation no longer held on the later 1.4 development line: `--space-engine pathmap` made a counted PathMap inside the Rust bridge's MORK `Space` container the primary occurrence store for generic MeTTa queries. It uses MORK expression bytes as trie keys but does *not* run those queries through MM2's `space_step`; raw MM2 stepping and generic counted queries are distinct modes over related substrate.

One-million-fact space lifecycle. The Roman/friend matrix performs one million insertions, one exact hit, one exact miss, a full scan, one removal, a post-removal miss, and a post-removal count. Every lane returned the same rows and multiplicities. Table 1 reports whole-process wall time and peak RSS; the embedded operation timer is shown separately for the exact hit because it isolates lookup and reconstruction from process startup.

Lane	Wall time	Peak RSS	Exact hit
PeTTa ordinary space	0.85 s	295 MiB	129 ms
CeTTa native space	4.56 s	643 MiB	25.2 μs
CeTTa native $\rightarrow$ resident MORK	6.53 s	1,317 MiB	113 μs
CeTTa explicit PathMap	6.43 s	1,015 MiB	56.3 ms
CeTTa PathMap $\rightarrow$ resident MORK	4.45 s	422 MiB	116 μs
CeTTa resident MORK	4.46 s	349 MiB	167 μs
PeTTa MORK bridge	13.31 s	668 MiB	111 μs

Table 1: Pinned Roman/friend lifecycle at one million facts. Conversion lanes include transfer cost; resident lanes do not transfer the store for each query.

The table does not support "always outsource to MORK." PeTTa's ordinary indexed space wins the complete lifecycle, while resident MORK offers CeTTa the smallest footprint among its million-fact lanes. At that candidate, CeTTa's explicit PathMap exact-hit cost was dominated by its single-pattern adapter and result reconstruction, not by trie lookup. Conversely, transferring a native store before a query loses both time and memory. The engineering implication is local: retain resident indexed stores, push conjunction and folds into their query algebra, and stream or factorize results instead of repeatedly materializing space tuples.

Conjunctive matching. On a generated two-hop join with 400,000 edges and 399,999 answers, CeTTa's stock evaluator took 953 ms per query and its streamed conjunctive fast path 797 ms. MeTTa-on-MORK's product-zipper query took 819 ms at the same scale. These close values have different internal profiles: MeTTa-on-MORK prunes a multi-pattern product during trie descent, while CeTTa still reconstructs candidate bindings around a shared evaluator. At smaller explicit-store sizes on that candidate, native CeTTa was faster than its then-current PathMap adapter (31.3 ms versus 73.6 ms at 20,000 edges). The appropriate adoption path is to expose multi-pattern cursor traversal and bounded row batches through CeTTa's backend interface, not to copy data across an FFI boundary for every join.

MAM workload revalidation. Table 2 uses the same five semantic workloads across the engines. JeTTa's core time excludes JVM startup; its wall/RSS cell includes it. PeTTa uses native tail-recursive drivers for the million-step interpreter and differentiation loops. A dash means the exact workload did not complete successfully and is not a performance number.

Workload	JeTTa wall/RSS	PeTTa wall/RSS	CeTTa HE wall/RSS	CeTTa Prime wall/RSS
Fibonacci(30)	0.26 s / 76 MiB	0.40 s / 19 MiB	15.85 s / 294 MiB	97.09 s / 11.4 GiB
Interpreter (10^6)	1.70 s / 875 MiB	0.80 s / 19 MiB	— (allocation failure)	— (timeout)
Differentiation (10^6)	2.74 s / 741 MiB	1.40 s / 19 MiB	— (allocation failure)	— (timeout)
Synthesis(40)	0.45 s / 104 MiB	0.10 s / 19 MiB	0.14 s / 68 MiB	0.66 s / 139 MiB
Backward chaining(1000)	0.41 s / 102 MiB	0.10 s / 23 MiB	0.05 s / 29 MiB	— (stack failure)

Table 2: Pinned MAM-workload revalidation. Every numeric cell returned the stated exact checksum before timing.

The refreshed evidence sharpens rather than overturns the competence-zone picture. JeTTa's compiled and memoized path is strongest on Fibonacci; PeTTa's WAM is strongest end-to-end on the million-step deterministic loops; and CeTTa HE remains strongest on the low-output deduction row. Prime's failures and high Fibonacci cost are not presented as a competitive result: they are the measured obligations for the experimental Need-machine work—linear administrative cost within a demand path, followed by sound sharing across repeated pure calls. Measured again on 27 October, naive `fib 30` takes 0.04 s on the pre-main HE lane, whose prepared pure route compiles it, about 0.06 s on its PeTTa lane, and about 2.6 s with 271 MB on the Prime draft (single runs, Appendix B).

11.11 August 9 Answer-First Backend and Dialect Audit

The 1.4 development line was audited again after the revision-pinned machine and PathMap work. Every timed row below was admitted only after the exact answer bag, multiplicity, and boundary answers were checked. Values are single whole-process observations on the audit workstation, not publication-quality medians; they are retained as structural regression evidence and are intentionally kept separate from the five-run tables above.

Counted PathMap conjunction. The HexLife witness joins two `alive` relations around one `neighbors` relation over triangular boards. All four configurations return the same normalized pair bag at each scale.

Cells	Result pairs	Lane	Native	PathMap
1,891	10,980	Prime	0.36 s / 138,512 kB	0.34 s / 135,316 kB
1,891	10,980	PeTTa	1.84 s / 328,688 kB	1.84 s / 331,264 kB
7,381	43,560	Prime	2.89 s / 876,992 kB	2.53 s / 851,424 kB
7,381	43,560	PeTTa	22.21 s / 2,447,440 kB	22.71 s / 2,452,192 kB

Table 3: Answer-checked HexLife conjunction on the recorded native and counted PathMap spaces. Each cell is wall time / peak RSS for one process.

Earlier in the audit the PathMap route exposed an effectively complete candidate scan. The repair gives the counted trie one synchronized native discrimination view for candidate coordinates, while the counted PathMap remains the occurrence authority. This is why the dated table measures a real index rather than storage followed by a full native unifier scan. PathMap crosses over slightly

on Prime and reaches PeTTa parity on this trie-suitable join; the large remaining PeTTa time at the higher scale is therefore a control-machine obligation rather than evidence that PathMap queries are intrinsically slow.

Logical update under mutation. A second witness consumes a two-hop match frontier over 5,000 edges and adds one derived path per answer. Both execution orders derive exactly 4,999 paths, including the first and last expected paths.

Execution order	Native	PathMap
Stream each `add-atom` over the frozen frontier	0.05 s / 27,216 kB	0.06 s / 31,196 kB
Collapse the frontier, then fold the writes	0.06 s / 31,660 kB	0.08 s / 35,332 kB

Table 4: PeTTa logical-update mutation witness at 5,000 edges.

Before repair the streaming PathMap form took approximately 5.2 s and grew approximately quadratically. The matcher had already captured the correct logical-update candidate snapshot; the defect was physical. Every append invalidated the derived index, and the next choice point rebuilt it over the growing space. Append-only updates now extend the live discrimination and substitution indexes incrementally. Retraction remains the conservative invalidation boundary. Thus the repair changes index maintenance, not which occurrences a running match may observe, and avoids forcing Prime to materialize an entire answer bag merely to protect a reader from its own writes.

Dialect-honest PeTTa comparison. Table 5 runs the same declared source in CeTTa's PeTTa lane and SWI-PeTTa. HE, Prime, and MM2 cells are deliberately absent: on these programs they are different language routes, not alternative implementations of the same dialect contract.

Workload and checked answer	CeTTa native	CeTTa PathMap	SWI-PeTTa
Ackermann$(3, 9) = 4{,}093$	1.35 s / 78,292 kB	1.34 s / 78,480 kB	1.29 s / 20,448 kB
Naïve reverse(3,000), length 3,000	0.45 s / 209,508 kB	0.45 s / 209,612 kB	0.32 s / 21,332 kB
Symbolic derivative, depth 12: 2,142,147	0.49 s / 281,324 kB	0.49 s / 281,908 kB	0.62 s / 313,508 kB
Roman PLN assertion	0.50 s / 108,216 kB	0.48 s / 108,740 kB	0.21 s / 24,452 kB

Table 5: Same-source, answer-checked PeTTa audit rows. Cells are wall time / peak RSS for one process.

There is no catastrophic broad slowdown in this compact portfolio: CeTTa is near SWI on Ackermann and faster on the symbolic derivative. Roman remains about 2.3× slower, while reverse and Roman retain much larger memory footprints. Those are honest remaining control/allocation targets; the table is not evidence that every PeTTa program should be optimized through the Prime machine or through PathMap.

11.12 September 29: PeTTa Dialect Against SWI-PeTTa

This benchmark compares CeTTa's PeTTa dialect (`--lang petta`) with SWI-PeTTa on the reference's whole example corpus and on scaling kernels, variable-transport walks, the pattern miner, a constraint ladder and `static-import!`, after the September conformance work (delayed goals, rational terms, one numeric equality policy, uncaught errors, `static-import!` through SWI's own

consult). The reference is PeTTa at upstream `main` with pull requests #219 and #225–#227 (revision `9b016a3`) under SWI-Prolog 10.1.9; the candidate is CeTTa source tree `1f3d58b9`. A program is timed only after a per-query correctness gate: each query's answers are compared between the engines as multisets, normalized for variable spelling alone, together with effects, assertions, exit status and standard error, with no stored reference output reused. Qualified programs then run unframed, five times per engine, serially and interleaved, pinned to one CPU of a Ryzen 7 9800X3D; peak memory and user-mode instructions are recorded, and every input, executable and linked library was hashed before and after. A companion report gives every row.

lane	cases	exact	bag	agree-failing	unframed alike	once	other	fail
PeTTa examples	167	161	0	1	1	0	0	4
79-row panel	79	79	0	0	0	0	0	0
large scaling rungs	15	15	0	0	0	0	0	0
pattern-miner modes	10	10	0	0	0	0	0	0
constraint-chain ladder	5	5	0	0	0	0	0	0
regression programs	7	6	0	0	0	0	0	1
conformance fixtures	20	9	0	0	9	0	0	2
registered-name probes	344	332	0	0	8	0	0	4
PeTTaChainer tests	21	2	0	10	0	0	0	9

Table 6: Correctness verdicts per lane (*unframed alike*: both engines stop the program alike; *agree-failing*: the same assertions fail in both).

161 of the reference's 167 examples agree exactly; the other six depend on the outside world (the clock, a network build, a hosted model, console input, PyTorch). Every kernel, walk, miner and ladder program agrees. The open application gap is PeTTaChainer: 9 of its 21 test files diverge.

family	rows	median	geomean	range	below 1×	startup-dom.
examples	160	11.0×	9.04×	0.04×–19.7×	3	142
panel applications	32	11.2×	7.69×	0.04×–20.3×	2	24
scaling batteries	41	4.53×	3.50×	0.07×–9.34×	2	9
variable-transport walks	20	8.65×	8.35×	1.29×–53.3×	0	10
pattern miner	10	1.29×	1.34×	1.01×–1.68×	0	1
constraint-chain ladder	5	0.05×	0.05×	0.004×–0.67×	5	0
family-balanced geometric mean			1.76×			

Table 7: Complete-process wall-time ratios (reference over CeTTa) by family. Startup-dominated rows are those where more than half the time gap is the gap between the process floors (38 ms and 3 ms).

Programs that do real work are faster on CeTTa in every family but one: 2.9× on the substantive examples and on the scaling kernels, 12× on the walks, 1.4× on the miner (geometric means); at their largest sizes all kernels but the state cell run 1.7× (binary trees) to 8.1× (Peano Ackermann) faster. Ten programs are slower, and their causes are measured. A chain of CLP(FD) constraints built one call at a time is flat on the reference (41–48 ms) and quadratic on CeTTa (0.06 to 11.2 s for 100 to 1,100 links): each call carries the whole network across the Prolog boundary and reads it back (90% of the instructions). A 28-leg conjunction runs at 0.04× because CeTTa recomputes

kernel	size	SWI-PeTTa	CeTTa	ratio	RSS SWI MB	RSS CeTTa MB
Ackermann	$A(3, 11)$	18.26 s	4.03 s	4.53×	28	35
Takeuchi	$(30, 20, 10)$	8.01 s	1.82 s	4.39×	20	29
tail loop	10^8	11.53 s	2.39 s	4.83×	20	30
map/map/fold	10^6	1.39 s	237 ms	5.87×	789	471
Peano Ackermann	$(2, 3000)$	2.37 s	293 ms	8.08×	2,745	64
n-queens	$n = 12$	8.57 s	3.11 s	2.76×	251	43
differentiation	13 steps	1.38 s	658 ms	2.09×	1,170	776
naive reverse	30,000	16.37 s	8.26 s	1.98×	47	81
binary trees	depth 24	8.33 s	4.92 s	1.69×	2,450	3,760
state cell	10^6	221 ms	3.28 s	0.07×	20	142

Table 8: The largest size of each scaling kernel: median of five runs.

every remaining leg's candidates before choosing the next (79% of its instructions on a smaller instance). A state cell updated in a loop runs at 0.07× because the named-state operations take general dispatch on every step. A Metamath backward chainer and a minimal-MeTTa interpreter run at about 0.7× with equal instruction counts but fewer instructions per cycle. `static-import!` of a million rows loads warm at 0.16×: CeTTa mirrors each fact into its native space. These costs mark the next work on the shared machine: keeping a constraint network where it lives, maintaining leg selectivity incrementally, and giving named state the compiled path.

12 Performance Analysis

12.1 Asymptotic properties

The engine's asymptotic behaviour on the covered workload families is a maintained property, enforced by paired growth ladders (candidate against a pinned baseline, per-size time and memory cells plus log-log slope checks) rather than by review. The load-bearing facts, with the algorithms in Appendix I:

- **Intern probing is full-cycle and tail-gated.** Stored record hashes and slot indices are distinct roles. An avalanche finalizer fixes the low-bit structural families that originally made bulk insertion quadratic, but the August adversarial family showed that ordinary linear probing could still form a 71-slot primary cluster. Lookup and insertion now share an odd hash-derived stride over the power-of-two table, so every probe sequence visits the full capacity before exhaustion. The gate uses a workload-scaled tail bound rather than imposing an arbitrary 64-slot runtime limit.
- **Judgment recording is constant per step.** Prime's evidence machinery appends at $O(1)$ per evaluation step, so budgeted judgments scale linearly with the runs they observe.
- **Branch-local arena discipline.** Evaluation branches whose results cannot escape release their scratch per branch; peak evaluation memory on insertion workloads sits below a Prime-free build's (paired-ladder ratios 0.60–0.78×).

- **Counting is engine-uniform and streamed.** Match-set counting flows through the backend visitor seam at the evaluation site, never through per-engine fast paths in grounded dispatch; cross-engine golden tests pin the uniformity.
- **PathMap logical update is not rebuild-per-write.** A running PeTTa match consumes the candidate snapshot captured at entry, while append-only writes incrementally update the derived discrimination view for later matches. Removal takes the conservative dirty/rebuild path. This separates snapshot visibility from index invalidation and removes the measured quadratic mutation loop without materializing the frontier.
- **Deterministic-tail loops are flat.** Unbounded tail-recursive effect loops (the classic bulk `add-atom` pattern) run with constant frames and per-iteration arena reset. The transformation's semantic license — iterative execution equals recursive evaluation, answers and effect multisets alike, with per-unfolding invariants transferring to whole runs — is a kernel-checked Lean theorem set (`Machines/DeterministicTail`).

On the bulk-ingest family the native engine matches the reference Prolog engine's complexity class ($O(N)$); the residual difference is a constant factor addressed by revision-keyed compiled views (Appendix I). Fresh, answer-checked measurements of the November preview are in Appendix A; every other table describes the binary and the date named with it.

12.2 Earlier heap profiles

Valgrind reruns on the grounded targets, measured on the pre-GC tree (2026-06-27), split into one success mode and one blocker. Massif reruns succeeded on both the restored 8-Queens row and the full TFBS-inclusive WM-PLN row. On direct 8-Queens, the fresh peak useful heap is 173.5 MB, dominated by arena allocation (`arena_claim_block`), `atom_symbol_id`, `atom_deep_copy_impl`, and the typed dispatch / match / eval path. On the full WM-PLN row, the fresh peak useful heap is 2.09 GB, dominated first by import-time term-universe growth and symbol interning (`cetta_realloc`, `term_universe_insert_new_record`, `tu_intern_symbol`, `parse_metta_file_ids`), then by query-time delayed result storage (`stree_insert`, `native_query`). By contrast, fresh Callgrind reruns on both grounded targets do *not* yet yield a usable callgraph file: Valgrind hits a `brk segment overflow` before emitting a nonempty callgrind output. We therefore report fresh Massif measurements here, but continue to withhold exact Callgrind percentages on the current build.

12.3 Tilepuzzle: PeTTa comparison and GC capacity repair

The 8-puzzle breadth-first search (BFS) remains a useful WAM-class stress test. The normalized mathematical contract is the size of the solvable parity class reachable from the solved board, including the start state: $9!/2 = 181{,}440$ board configurations. The CeTTa port (`benchmarks/test_tilepuzzle.metta`) is a hand-normalized HE/CeTTa counterpart of the PeTTa example. The algorithmic contract is the same BFS over the solvable parity class, but the supporting builtins differ: PeTTa uses its `lib_datastructures` queue and `add-unique-or-fail` visited-set idiom, while CeTTa uses queue-space, hash-space, and the HE-portable zero-results idiom `case(collapse(match))`. It now returns the labeled board-count result `(tilepuzzle-reachable-state-count including-start 181440)`. The PeTTa native example (`examples/tilepuzzle.metta`) retains its historical self-check of 181,441 as its native observable. The comparison below separates that observable from the normalized board-count contract; it does not claim an extra reachable board.

Artifact	Native observable	Contract	Time	RSS
CeTTa pre-GC port	`[181440]`	181,440 boards	26.41 s	5.86 GiB
CeTTa post-GC port	`including-start 181440`	181,440 boards	8.80 s	233 MiB
PeTTa native	`test ... 181441`	181,440 parity class	1.40 s	121 MiB

This row should therefore be read in two ways. As a cross-runtime search benchmark, PeTTa still wins decisively: the Prolog/WAM lane is about 6.3× faster than post-GC CeTTa on the native example and uses about half the peak memory. As a CeTTa capacity regression, however, the row is repaired: the full board-count contract now completes in single-digit seconds instead of exhausting the old evaluation-arena envelope.

The CeTTa search runs as a single top-level form, and each BFS step builds a fresh evaluation environment. Before the fix, those per-step environments were never reclaimed inside the evaluation arena, so memory grew linearly in the number of visited states. The fix is a tail-safe garbage collector for the evaluation arena, on by default. It (i) skips environment merging for *closed* queries—queries with no free variables escaping to the caller—and (ii) at tail-call-safe points evacuates the live root set (the current term, its environment, and its evaluation type) into a small survivor arena, then resets the evaluation arena to a per-call anchor; the survivor arena is itself reset at each top-level form boundary. Because only the roots that outlive a step are copied forward, the transient per-step environments are freed in bulk instead of accumulating.

With the collector on (commit 9237afd), the full 8-puzzle BFS completes well within its envelope, and the reclaimed memory is stable under the collection-budget spot checks:

Full run, collector on (commit 9237afd)	Value
States visited	181,440
Wall time	8.80 s
Peak resident memory, default budget	233 MiB
Peak resident memory, 1 MB budget	178 MiB
Eval-arena memory reclaimed	5,704,882,536 B (≈5.3 GiB)
Survivor-arena peak	135,432 B (≈132 KiB)
Survivor resets	1

At the 1 MB budget the run returns the identical 181,440-state payload as at the default budget (with even lower peak memory), so the collector preserves the computed result rather than changing it. It is differentially validated, not verified—a collector-on versus collector-off comparison that requires identical outputs on every case:

Collector-on vs. collector-off	Cases	Agree
Fast regression corpus	406	406
ASan/UBSan + provenance asserts	404	404

Two further ASan/UBSan cases are skipped only because the collector-off reference exceeded its bounded time limit.

12.4 Structured Space Kinds

CeTTa now supports three ordered space kinds beyond plain atomspaces:

Kind	Discipline	Use case
`queue`	FIFO push/pop	BFS frontiers, work queues
`stack`	LIFO push/pop	DFS, continuation stacks
`hash`	O(1) membership	Visited sets, deduplication

These are created via `(new-space queue)`, `(new-space hash)`, etc., and support universal pattern matching through the standard `match` API. The tilepuzzle uses queue-space for its BFS frontier and hash-space for visited-set deduplication. The current paper claim for that workload is the post-GC capacity baseline: 181,440 reachable boards in 8.80 s at 233 MiB peak RSS (Section 12.3). Older plain-atomspace versus queue/hash memory comparisons are kept as historical diagnostics rather than current benchmark evidence.

12.5 Architecture: Three Machines

Profiling and design analysis (informed by an external architectural review) identifies three semantically distinct machines that should not share mutable state:

1. **Local search machine.** Branch-local bindings, trail/rollback, choice frames, scratch arenas. WAM-inspired: cheap undo via watermarks, not clone/free churn. This is no longer merely the next priority: `PettaSearchMachine` owns an explicit heap of clauses, environments, continuations, choice points, rollback marks, and answer projection, while `SearchContext` supplies the shared binding/scratch save protocol. Remaining work is structural dispatch and retention reduction, not inventing the seam.

2. **Indexed shared-space machine.** Counted PathMap/MORK-backed spaces, snapshot/epoch discipline, conjunction query planning, and eventually product-cursor triejoin. The current engine has backend-primary counted storage plus a derived native candidate view; direct trie descent remains the route that can retire that dual representation. The index is deterministic and committed—no backtracking through its physical state.

3. **Type/elaboration machine** (partially realized). Scoped binders, metavariables, constraint solving, delayed assignments, separate from runtime bindings. PeTTa's `typecheck-v2` already crosses a provider-neutral `PettaAnalysisService` boundary carrying provider/ABI/policy identity and mutable authority; the fully generic constraint machine is still future work.

Language profiles (`he_compat`, `he_extended`, `he_prime`, `petta`) are compositional assemblies over these machines, not ad-hoc forks in the evaluator.

12.6 Faithful Backend Obligations

The indexed shared-space machine needs a sharper correctness contract than "the backend is fast and seems to return the same answers." Recent Lean work factors the backend story into three explicit runtime seams:

1. **Candidate selection.** The index narrows candidate atoms, while CeTTa's native matcher performs exact variable-sensitive matching. This is the two-phase architecture now validated by concrete selector theorems over PathMap tries and by a concrete matcher instantiation on the HE side.

2. **Canonical storage.** Storage may quotient results to canonical support, but it must preserve co-reference. Repeated source variables must remain repeated after storage and materialization; distinct variables must remain distinct.

3. **Revision-based caching.** Cached query answers are valid only across mutations that preserve the relevant support boundary. Duplicate adds or high-count decrements may preserve cache validity; first adds and last removes must invalidate. Variant-normalized cache keys must also preserve right-hand-side answers and result shape under materialization.

This yields a practical division of labor for the PathMap/MORK backend:

1. PathMap stores canonical support and performs prefix-based candidate narrowing;

2. the native matcher remains the source of truth for exact bindings and binder hygiene;

3. count metadata, not trie-key distortion, carries bag semantics.

Current realization and logical-update boundary. The current generic `pathmap` engine is a counted PathMap inside the Rust bridge's MORK `Space` container. That counted store is the occurrence authority. A synchronized native AtomId array and discrimination trie are a *derived candidate accelerator*, not an independently writable space: private candidate order may differ, but public iteration, declared order where the dialect observes it, multiplicity, removal, and destructive replacement are answered at the authority boundary. PeTTa captures candidate coordinates at match entry, giving a logical-update view; new assertions extend the derived indexes incrementally for subsequent queries, while removals conservatively dirty them. This split both preserves the running snapshot and avoids the former invalidate/rebuild cycle.

The synchronized view is intentionally a transitional realization. It avoids row-granular bridge translation and supplies ordered candidates today, but costs a second resident representation and an invalidation proof. The durable PathMap route is trie-native candidate and product-cursor descent, with decoding only for surviving bindings and native fallback whenever the declared observer cannot be preserved.

Verified runtime contracts. Recent Lean work extends the backend correctness story with three new contract files:

- `CeTTaRuntimeContracts.lean` (20 theorems): canonicalization injectivity and BEq preservation (`term_canon.c`), search-context save/rollback correctness (`search_machine.c`), effect classification safety (`runtime_effect_classes.json`), and profile hierarchy (`session.c`). Each theorem maps to a concrete refactoring risk it reduces.

- `IncrementalTableSemantics.lean` (15 theorems): specifies partial-table soundness, answer monotonicity, and completion for future tabling in `table_store.c`. Any correct implementation must satisfy `TableEntry.Sound` (partial reads are genuine) and `TableEntry.Exact` (completed tables equal `queryEquations`).

- `ProducerChoicePoint.lean` (10 theorems): formalizes the search machine's save, rollback and nested choice-point protocol over combined bindings and scratch state.

Crucially, CeTTa's imported backend is binding-correct not through trusted direct bridge bindings, but because candidate narrowing is followed by seeded native rematch (`space_subst_match_with_seed`). The theorem `imported_seedRematch_binding_parity` in `SeedRematchContract.lean` formalizes this: when imported candidates are sound, the two-phase query produces the same result set as direct matching.

Positive example. Adding an existing atom to a counted space may increment multiplicity without changing trie support or invalidating unrelated cached queries.

Negative example. Letting the backend bridge perform full unification over serialized query terms invites exactly the binder-hygiene failures that arise when canonicalization preserves shape but breaks co-reference.

These obligations are now mapped one-for-one to runtime seams: `space_match_backend.c` for candidate selection, `term_universe.c` for canonical storage, and `table_store.c` for cache invalidation. The companion note *Faithful Backends for HE MeTTa* develops the full proof story, including the variant-query and binding-cache lemmas; the present paper uses those results to justify the architecture of CeTTa's indexed shared-space machine.

12.7 Open Realization Laws for Heterogeneous Backends

The corresponding Lean development deliberately specifies an open law interface rather than freezing a final Prime or CeTTa ABI. Its semantic waist is

$$\mathsf{snapshot} \times \mathsf{operator\ plan} \longrightarrow \mathsf{delta} \times \mathsf{receipt}.$$

Plans, occurrence identities, deltas, and receipts are revision-scoped; a receipt binds the named observer, plan identity, delta identity, source and target endpoints, execution mode, and exact ordered occurrence lists. Stale plans and receipts are rejected. If semantic commutation permits parallel execution but a backend's resource algebra declines it, the admitted realization serializes the same operator rather than changing its answer. Lossless native-to-PathMap-to-native transfer composes only at matching backend/revision boundaries and retains every intermediate delta and receipt.

This distinguishes two questions that a universal "parallelizable" flag would conflate: whether two events commute under the language's named observer, and whether a particular backend can coexecute them with its physical resources. Native space may admit fine-grained mutable occurrence plans; PathMap may admit persistent-root, bulk-monotone, and structural merge plans. Both realize one MeTTa semantics, and both must serialize or fall back when their physical algebra cannot preserve it. Richer cost, provenance, capability, observer, and backend evidence may refine the current records provided accepted evidence erases to these laws. This is sufficient guidance for current realizations, not a claim of minimality, completeness, initiality, or generated-C adequacy.

The proof layers are separated accordingly. `QueryRevision` supplies proof-relevant histories, snapshots, and literal versus observer-relative commuting squares; `EventValuation` separates behavioral commutation from resource compatibility and permits sound backends with different admitted plan inventories; `QueryRevisionBridge` instantiates evidence revision and located MeTTa access modes; and `OperatorRealization` checks the revisioned plan/delta/receipt boundary and its open refinements. These are mathematical principles and separating examples. Native and PathMap refinement instances, OS-thread lowering, reclamation safety, and generated-C execution adequacy remain explicit obligations.

12.8 MM2 Integration

MM2 is a deterministic, priority-scheduled, forward-chaining term rewriting language implemented by MORK. On the native bridge route its programs are atoms in MORK-backed spaces and `step!` invokes MORK's scheduler over them; ordinary MeTTa evaluation does not silently interpret an

exec atom. The hybrid interaction requires no special protocol: MeTTa creates spaces, adds data, explicitly steps MM2 rules, and reads results through ordinary match.

```
!(bind! &ws (new-space mork))
!(add-atom &ws (exec (0 step)
  (, (edge $x $y) (edge $y $z))
  (0 (+ (path $x $z)))))
!(add-atom &ws (edge a b))
!(add-atom &ws (edge b c))
!(step! &ws)
!(match &ws (path $x $y) ($x $y)) ; -> (a c)
```

Three import paths exist: .mm2 file import (import! or include into a target space, which auto-enables the MORK backend), inline add-atom of exec rules from MeTTa, and compiled .act artifact loading via mork:open-act.

Because MM2 rules are ordinary atoms, MeTTa can pattern-match over them as data—inspecting priorities, rewriting source/sink templates, or extracting rules for analysis:

```
!(match &ws (exec $p $src $snk) (rule $p))
```

CeTTa's mm2_lower.c handles bidirectional source syntax↔IR lowering (exec↔mm2_exec, ,↔mm2_pattern_and, etc.), preserving round-trip fidelity between MM2 concrete syntax and CeTTa's internal representation.

An indexed MM2 GSLT rather than one overloaded boundary. The Lean model now packages deterministic MM2 work queues through one ExecutionProfile → GSLT construction over a shared carrier of finite support-valued spaces. The historical comma-pattern core is one member. A strict source-aware member admits comma or I inputs and comma or O outputs only when every source factor and sink belongs to the modeled vocabulary; it rejects an entire directive instead of filtering unknown children. Its boundary is then selected explicitly: leaveInert excludes unsupported directives from scheduling, whereas consume selects the raw four-field exec shell and erases it if the selected interpreter declines it. A closed canary proves that a concrete malformed input is a normal form in the former profile and an erasing step in the latter. This captures remove-before-interpret without declaring that factory cases absent from Lean are absent from upstream MORK.

All members are deterministic, and their canonical drivers have only unit private state: pending work and scheduling locations remain atoms in the space. Bounded runs carry the residual space and distinguish quiescent completion from live expiration. On an explicitly shared compatible candidate, source-aware and historical firing are proved equal; full candidate-set equality remains conditional on completing the parser and byte-order refinements for arbitrary locations. Finally, the generic OSLF construction maps each profile to predicate-valued native types, and exact target inhabitation is proved equivalent to one step of that same profile. The corresponding native claim is already in the generic CertificateGSLT authority form. A concrete MM2 wire encoding and native checker lowering remain realization obligations, not a second semantics.

Fail-closed hosted translation (work in progress). Alongside that native route, tools/mm2_to_metta.py structurally translates a deliberately bounded MM2 fragment for execution by a tool-isolated MeTTa engine. It renames MM2's +/- sinks so they cannot collide with MeTTa arithmetic, normalizes variadic output templates, and orders static rules by MORK's

encoded expression key. The admitted fragment covers ground static `exec` locations, comma queries, add/remove/comma outputs, and one guarded distinct-count form. Dynamic or reflective `exec`, `I` sources, non-ground program data, unbound outputs, mixed unsupported sinks, and unimplemented reductions fail translation rather than receiving an approximation.

The native corpus gate records 43 programs: 40 reproduce their recorded run/panic status, three examples pinned to a different MORK fork are explicit skips, and none differ or disappear. Of those, 21 are admitted by the hosted translator. Each admitted program is compared with native MM2 as a multiset of canonical MORK-encoded atoms across HE, Prime, and PeTTa, on both native and PathMap spaces. This is strong basic differential evidence, not a general MM2 compiler claim. The queue is compiled statically and does not yet model runtime `exec` creation or consumption; the distinct-count path still groups quadratically; and upstream source/sink coverage remains incomplete. The prototype therefore remains under `tools/`, with no new unnamespaced standard-library builtins.

The application comparison is different from this front end: ordinary MeTTa programs execute through CeTTa's own evaluator and query machinery, while standalone MORK, in its default build and in its build with the leapfrog join, executes its MM2 programs. The Opt application suite, its scale ladder and MORK's showcase workloads are measured in Appendix A. Timing CeTTa's MORK-backed MM2 front end against standalone MORK would measure hosting and publication overhead, not a competing kernel.

13 The MeTTa Dialect Landscape

13.1 Shared Fragment

A substantial common surface uses `let`, `let*`, `match`, `if`, `case`, `bind!`, `add-atom`, and `new-space`. It includes dialect-agnostic workloads such as the Metamath proof-search kernel, but the paper does not attach an unmeasured percentage to it or claim that every effectful program is identical across engines. The executable translator and its Lean correspondence theorem cover the explicitly named pure fragment; state, free-variable scope, observation, imports, and profile-specific operators are tested and reported as separate boundaries. This scoped surface is the common core of the upstream MeTTa specification, with HE, PeTTa, and CeTTa providing distinct realizations and, in places, distinct dialects.

13.2 Translated Constructs

Direction	Construct	Translation
HE→PeTTa	`chain`	`let`
	`collapse-bind`	`collapse`
	`superpose-bind`	`superpose`
	`nop X`	`let $fresh X ()`
	`function (return X)`	`X` (unwrap)
PeTTa→HE	`progn A B`	`let $fresh A B`
	`prog1 A B`	`let $r A (let $d B $r)`
	`@<`	`<s`

13.3 Semantic Divergence: Constraint Store vs. Independent Scope

The translation preserves semantics on the *pure fragment* (ground terms, or variables scoped by `let*`). A genuine divergence exists for free variables in non-deterministic branches:

```
(= (deduce (And $a $b)) (And (deduce $a) (deduce $b)))
```

When `deduce` returns via `match &self` with a free variable `$x`, HE/CeTTa bind `$x` consistently across both branches of `And`, while PeTTa gives each branch its own copy.

This divergence has a precise theoretical characterization. Saraswat et al. [2] showed that concurrent constraint languages based on Tell-and-Ask over a shared store satisfy a *monotonicity property*: constraints accumulate monotonically, and any reduction possible in a later configuration was already possible in an earlier one (their Proposition 2.1). HE/CeTTa's `match &self` operates on a shared constraint store—bindings produced by one branch propagate to all others via the shared space. PeTTa's Prolog backend evaluates branches with independent variable scopes, closer to what Saraswat et al. identify as the "read-only variable" model of Flat Concurrent Prolog, which they explicitly note does *not* possess the monotonicity property.

The depth-1 standard form. Saraswat et al. define a clause to be in *standard form* when each atom in the body is depth-1 (no shared variables between conjunction branches). Nil Geisweiller's typed backward chainer uses exactly this form: `let*` threads variable bindings explicitly between premises, eliminating implicit sharing. This is why the typed chainer works identically on all three runtimes—it stays within the standard-form fragment where the constraint-store and independent-scope semantics coincide.

Lean formalization. The `Translatable` and `TwoSortTranslatable` predicates in `HEPeTTa-Sound.lean` characterize precisely this fragment: atoms where `atomToPattern` succeeds and produces `isMatchCorrect` patterns (no lambda, subst, or collection nodes). The sorry-free proof establishes that equation-matching semantics are preserved under translation on this fragment. Extending the proof to the shared-variable conjunction fragment would require formalizing the monotonic constraint-store semantics of HE's `match &self`—a natural next target for the Lean development.

14 Observation, Non-determinism, and Shared State

An earlier version of this paper ranked sequential, interleaved, forked, and illegal execution as four global alternatives and promoted an unreceipted "6/8 cases" result. That framing was too coarse. A traversal policy, a result observer, a shared-state trace, and definition visibility are distinct semantic coordinates. Collapsing them into one scheduler order would both over-specify pure MeTTa and under-specify effectful MeTTa.

14.1 Four Separate Contracts

Coordinate	Promised	Deliberately not promised
Complete result observation	Visible occurrences and exact multiplicity	Incidental engine traversal order
Explicit sequence observation	The sequence selected by the named policy	Equality with a different policy's sequence
Shared effects	A memory-safe realization and a trace permitted by the operator/profile	Serializability, fairness, or freedom from logical races unless requested
PeTTa definition calls	Translation-stage callability and one stable occurrence view per entered call	One frozen view for an entire Hyperpose episode, or an unrestricted live rescan during backtracking

14.2 Bags Do Not Owe an Order

Complete materializing observers such as `collapse`, `collect`, and the corresponding PeTTa collectors expose a finite occurrence bag. The concrete result is necessarily printed as a sequence, but the language contract preserves its contents and multiplicity rather than the incidental order in which an engine found them. A conformance gate therefore compares canonicalized bags without deduplicating them. Reversing one engine's result only to imitate another engine's traversal would add work and remove valid scheduling freedom.

An order-sensitive fold is different: the fold itself is a sequence observer. Its result follows the selected engine or explicit search policy, so the sequence is observable for that run without becoming a cross-policy ordering theorem. Likewise, `once` observes one permitted witness rather than the complete bag. These distinctions are represented explicitly in the Prime observer algebra instead of being inferred from list layout.

No lane promises to *hide* its order. A program may rely on an engine's documented order by being engine-specific; the default observation is the bag, and making one engine imitate another's incidental order is never worth a slowdown. Where an engine is slower than it could be because it reproduces a reference's answer order, the order gives way.

14.3 Effects Remain Permissive

Bag equality does not erase effects. Two executions can return the same occurrence bag while publishing different lawful interleavings to a shared space. Sequential execution does not make this issue disappear: two sequential branches that edit the same space can still have an order-dependent outcome. Consequently, neither "always sequential" nor "effects forbidden under non-determinism" is an adequate language law.

The PeTTa compatibility policy is permissive. A program may issue concurrent or order-sensitive edits and accepts responsibility for its logical races, including lost updates. The runtime remains responsible for representation integrity: no user-level race licenses a torn index, stale pointer, use-after-free, or memory corruption. Conversely, the possibility of writing a racy program is not a reason to reject a safe effectful one. Users can request stronger local contracts with `with_mutex` and `transaction`. In the HE-extended family, `with-space-snapshot` remains an orthogonal read-from-frozen, write-to-live interface useful for semi-naive forward inference; it is not a mandatory wrapper around all stateful search.

14.4 PeTTa Hyperpose: Stage Then Call Snapshot

The PeTTa translator and SWI-Prolog's logical update view compose two boundaries. A literal Hyperpose branch list is translated before parallel execution begins, so that translation event fixes which heads are calls and which remain data. A computed branch list invokes `eval` in its worker, and therefore receives a later translation event. Once a translated dynamic call is entered, its visible equation occurrences are frozen at that call's generation. Adds and removals do not change the alternatives of the entered call, while a later call may observe them. SWI-Prolog realizes this rule with clause create/erase generations and a generation tag on each goal [13].

$$\begin{array}{c}\text{translation event} \longrightarrow \text{call-versus-data classification}\\ \downarrow\\ \text{call entry} \longrightarrow \text{ordered stable-occurrence lease at revision } r\\ \downarrow\\ \text{later mutation leaves that lease unchanged; a later call enters at } r'\end{array}$$

This contract is neither a whole-episode clone nor an unrestricted live database. Branch-local continuations, bindings, scratch arenas, and newly allocated transient values remain private. Published space identities and their user-visible effects are shared. Equal equation payloads remain distinct occurrences, so multiplicity survives both snapshots and parallel collection.

14.5 Revision-Derived Realization and Its Evidence Boundary

The current qualified development realization places the ordered equation occurrences behind an exact-key revision authority. Acquiring the same space/revision key reuses an owned immutable view; replacing the cache drops only the cache's ownership, so an earlier call lease continues to denote its frozen view. A reusable worker authority removes repeated operating-system thread creation while leaving queues, tasks, arenas, cancellation, and result lifetime episode-local. Nested, busy, or over-capacity episodes take a total one-shot fallback rather than making a legal computation unavailable.

Dependency algebra. Let A be a live Space authority, $\ell(A)$ its process-local lifetime, $\pi_E(A)$ its ordered equation-occurrence projection, and $e(A)$ the revision of that projection. For an overlay, $b(A)$ additionally summarizes the actual base chain whose visible prefix it reads. The equation-view key is therefore

$$D_E(A) = \big(\ell(A), e(A), b(A)\big).$$

A match-decision plan reads more than the equation view: its key extends D_E with the host callability authority h and semantic configuration σ,

$$D_M(A, h, \sigma) = \big(D_E(A), h, \sigma\big).$$

The cache law is consumer-relative. If a stored view denotes $\pi_E(A)$ and the complete dependency tuple for consumer C is unchanged, then reusing the view must give the same C-observation as rebuilding from the current projection. If any coordinate differs, equality is not licensed and the implementation rebuilds or takes its complete generic fallback. Data-only publication carries a witness that π_E is unchanged and hence preserves e; equation-relevant and opaque publication advance e conservatively. A changed host-callability authority invalidates a match-decision even when e is unchanged.

This factorization is important operationally. It adds two 64-bit fields to each Space (16 bytes) but no equation record and no second search index. The existing head index still selects candidates; the new token only states when a derived view of that index may be reused. Runtime receipts are checked by closed equations such as

$$attempts = reuses + stale,$$

not by fixing a particular number of cache hits, threads, scans, or allocations into language conformance.

This is a standard incremental-computation shape rather than a Prolog-specific trick. Salsa tracks the inputs read by a memoized computation [14]; rustc stores a query DAG and can mark a recomputed query green when its result fingerprint is unchanged [15]. The verifying traces of *Build Systems à la Carte* record dependency hashes and separate the rebuilding strategy from scheduling [16]. Adapton's demanded computation graph separates inner incremental computations from their observers [17]. Differential Dataflow arrangements maintain and share indexed `(data,time,diff)` batches across consumers rather than asking each operator to rebuild an index [18]. CeTTa's present token is deliberately coarser than these general systems: it does not discover dependencies automatically, fingerprint equal rebuilt projections, or transport deltas. It adds the MeTTa-specific obligations of stable occurrence identity, exact multiplicity, translation stages, and a total generic evaluator whenever a compiled representation cannot be transported lawfully.

Measured development boundary. A September 2026 audit compared the complete development tree with the frozen staged checkpoint, the exact repository head, its parent, and SWI–PeTTa. Every engine produced the same output on all six rows below. Entries are medians of three interleaved runs; time is process CPU seconds and memory is peak resident MiB.

	Development tree		Exact head		SWI–PeTTa	
Workload	Time	RSS	Time	RSS	Time	RSS
`scale`	5.69	318.0	5.99	322.6	0.81	398.5
`tilepuzzle`	10.90	148.2	10.95	147.9	1.34	121.2
`matespacefast`	14.92	761.5	15.21	766.3	4.01	3340.6
`matespace2`	5.60	20.1	5.45	24.7	5.52	562.9
`permutations`	1.67	14.6	1.60	19.5	0.61	143.3
`peano`	2.20	16.6	2.22	21.0	1.58	22.7

Across these deliberately hard rows, the geometric-mean CPU ratios are 0.994 against the staged checkpoint, 0.998 against exact head, and 1.001 against its parent: the development tree is cost-neutral at this resolution. Its geometric-mean RSS ratio against exact head is 0.883. SWI–PeTTa remains 3.06 times faster in geometric-mean CPU, while memory is workload-dependent: CeTTa uses far less on `matespacefast` but more on `tilepuzzle`. An independently seeded sample of eleven ordinary example programs was also output-exact on all five engines. Those programs finish in under ten milliseconds on CeTTa, so their timings are treated as a startup and timer-resolution control rather than as a speed claim; their median peak RSS fell from 19.1 MiB at exact head to 14.5 MiB in the development tree.

A direct token-consumer control gives the narrower ruling. Paired token-on and token-off executables agreed exactly on `peano`, `matespace2`, `scale`, and `permutations`; their medians differed by at most 1.2% in either direction. This control disables reuse by the consumer but does not remove the token fields or mutation classification, so it establishes neither a whole-token cost nor a

speedup. The token remains a sound dependency coordinate with a live consumer, not an admitted performance result.

The orthogonal integer-discrimination map does have a measured consumer. It replaces linear search among rigid integer edges by a finite key-to-subtree map, leaving wildcard selection, occurrence order, and multiplicity unchanged. On 100,000 numeric clauses it reduced process CPU from 1.39 to 0.36 seconds relative to the recent CeTTa realizations, with essentially unchanged peak RSS. A 20,000-clause instruction profile fell from 1.647 billion to 651 million retired instructions. A 10,000-clause control completed on all five engines with the same answer; SWI–PeTTa did not complete the 100,000-clause run within the audit boundary, so no cross-engine speed ratio is claimed there. The generic finite-index and rigid-coordinate Lean developments justify the representation refinement; the runtime controls check its concrete ordering and multiplicity obligations.

Two profiling-directed repairs were required before making these comparisons. A context-local direct-mapped cache removed repeated linear string-policy search from builtin admission, reducing the `peano` instruction count from 1.379 to 1.064 billion, close to the staged checkpoint's 1.042 billion. The foreign-runtime capability token now reads the already-maintained atomic revision instead of acquiring the global adapter mutex for every validity query. Neither representation is named by a language test: conformance observes answers and effects, runtime receipts obey conservation equations, and performance tests compare interchangeable realizations. The full normal suite, focused AddressSanitizer/UndefinedBehaviorSanitizer gates, and the cross-engine rows above all pass. Per-head generations, fingerprint backdating, and delta transport remain possible refinements, but each requires its own measured false-invalidation consumer and semantic controls.

For Prime, the same semantic waist supports a more structured design: declared effect footprints, affine fuel leases, explicit observers, and transactional or disjoint-effect realizations can justify stronger parallelism. Unknown footprints still select a complete conservative realization; failure to prove commutation is an optimization decline, not a language-level refusal.

15 HE′: Dependent Telescope Extension

Both HE and CeTTa treat `(: $e T)` in arrow-domain positions as a literal expected type, not a dependent binder. This means Curry-Howard backward chainers cannot store rule types as `(: dt (-> (: $e (eqtl $snp $gene)) (target $snp $gene)))` in `&self` without triggering `BadArgType`. This is a current HE spec limitation, not a CeTTa bug.

We formalize an explicit extension, HE′ (he_prime), as a *telescope interpretation* of arrow domains. The extension is exactly one function: `splitDomain`, which recognizes `(: $x T)` as a dependent binder instead of a literal. Everything else (matching, bindings, space operations, equation dispatch) is inherited from HE unchanged.

The telescope walk checks arguments left-to-right: each successful type match extends the binding environment, and later domains and the codomain are instantiated under accumulated bindings. This is formalized in Lean (230 lines, 0 sorries) at `HEPrime/Telescope.lean`, with kernel-checked positive and negative examples:

- **Positive:** applying `dt` to `eqtl_rs1_gene1` infers return type `(target rs1 gene1)` via dependent substitution. Binary rule `ct` with cross-domain dependency (`$snp2` bound by first argument, constraining second domain) also kernel-checked.
- **Negative:** mismatched SNP (`rs2` $\neq$ `rs1`) correctly produces `none`. Standard HE telescope parsing treats the same domain as a plain atom, proving the semantics are distinct.

CeTTa implements HE′ as an opt-in profile (`he-prime`), gated by a single flag (`enable_dependent_telescope`). The implementation mirrors the Lean formalization: `split_dependent_domain`, `function_domain_type`, `bind_domain_binder`, and `infer_dependent_application_types` in `eval.c`.

15.1 Experimental: Typed Search over Dependent Codomains

The telescope elaboration above is the kernel-backed part of HE′: one interpretation function, mirrored in Lean. The `he-prime` profile additionally carries a larger *experimental* typing module (`src/he_typing.c`): consistency analysis, checked type-level computation, and proof search by type inhabitation. Its evidence is executable — a blessed adversarial regression suite (`tests/profile_he_prime_dtt_adversarial.metta`) and a reproduced end-to-end demonstration — and deliberately *not* a soundness theorem: no Lean correspondence exists for this module yet, and its final proof check reuses the same C implementation it validates. The two authority levels are distinct and should not be conflated.

The judgment. The module implements three-valued *success typing* over declared types. `check-typing` accepts a term against an expected type when *some* declared type is consistent with it, rejects when *every* declared type is a proven mismatch, and otherwise answers `unknown` with a reason (fuel exhaustion, ambiguous computation, inadmissible computation). A rejection is proven; an acceptance is not a promise. Consistency is licensed per edge by exactly one named reason — `exact`, `structural`, `dynamic` (`%Undefined%` on either side), `top` (`Atom` as the expected type), or `meta-staging` — and composite types combine children by *weakest license*: a single dynamic child keeps the whole edge dynamic at any nesting depth, so a value cannot be laundered from one base type to another through `%Undefined%` by wrapping it in a constructor.

Type-level computation. Dependent codomains may contain computation, so conversion is capability-gated at two levels with different strengths. A *direct* grounded operation reduces inside a type only if it lies on an explicit allowlist of deterministic, effect-free operations; any other direct grounded call — space mutation, I/O, a foreign call — is refused with `(he-reject (type-computation-inadmissible))` before dispatch and never executes. The typing operations' declared types stage their term and type arguments (`Atom` parameters), so argument evaluation cannot run an effect before the checker rules on it. A *user* function marked `type-level-function` is a weaker boundary: it evaluates against a fuel-bounded snapshot clone, so its *space* mutations are isolated and discarded, but external effects (for example I/O routed through the evaluator) are *not* capability-contained. That path is trusted and experimental; full effect exclusion would require evaluator-level effect capabilities or a separate pure type-reduction evaluator, not a marker plus a space clone.

Search as inhabitation. `type-inhabit-first` performs Curry–Howard backward chaining: rules are arrow-typed declarations marked `chaining-rule`, goals are types, and answers are terms returned together with a re-check of their type. Only proof-grade edges (exact or structural all the way down) admit a candidate: a witness whose type merely gradually matches the goal — through a dynamic or top license — can pass `check-typing` but never stands as an inhabitant. Forward chaining (`type-forward-closure`) reports `complete` only on a reached fixpoint, and otherwise `resource-incomplete` or `rounds-exhausted`, under one aggregate fuel budget.

Multi-answer enumeration. `type-inhabit` generalizes the existential search to a lossless multi-answer interface built on an explicit choice-point machine rather than eager recursive vectors. Premise proofs are pulled on demand from immutable resumable continuations, so distinct witnesses of one premise stay distinct top-level proofs — for a rule $A \to B \to C$ with two inhabitants of A, both proofs of C are returned — and the requested top-level answer count is never propagated into a child search. A ranked work queue schedules the finite depth-bounded candidates fairly, so a recursive rule listed before a direct one cannot starve the shallower finite proof. Answers are identified *losslessly*: two derivations are the same answer only when their canonicalized proof term, the substitution of the original query variables, and the re-checked normalized type all coincide, so proofs that differ only in a computed truth value or evidence term remain distinct. The interface is *prefix-preserving* — increasing the answer count or the fuel budget never removes or reorders an earlier answer; reaching the requested count with work still pending reports `more-possible`, and exhausting fuel returns the answers already found under `resource-incomplete`. This is an operational fairness guarantee over the bounded search space, not a general enumeration-completeness theorem: outside a certified-deterministic or completed-finite fragment the honest verdict remains `more-possible`.

Dependent-typed PLN, running. The payoff is the standard PLN idiom with truth-value formulas in rule codomains — the capability Geisweiller's DTL chainers had to leave disabled pending a type checker that runs functions inside types. Under `he-prime` the rules run as written (a derived encoding of the DTL constructor types, not an unchanged execution of the original source tree):

```
(: Deduction
   (-> (≐ $p $ptv) (≐ $q $qtv) (≐ $r $rtv)
       (≐ (→ $p $q) $pqtv)
       (≐ (→ $q $r) $qrtv)
       (≐ (→ $p $r)                        ; codomain COMPUTES the
          (deduction-formula $ptv $qtv $rtv $pqtv $qrtv))))  ; conclusion TV
(chaining-rule Deduction)

(: pP  (≐ P (STV 0.8 0.9)))            ; evidence facts (pQ, pR,
(: pPQ (≐ (→ P Q) (STV 0.75 0.8)))     ;  pQR analogous)

!(type-of &self (quote (Deduction pP pQ pR pPQ pQR)))
; => (type-set (≐ (→ P R) (STV 0.6333333333333334 0.7)))

!(type-inhabit-first &self (≐ (→ P R) (deduction-formula ...)) 2 500000)
; => (he-accept (: (Deduction pP pQ pR pPQ pQR)
;                  (≐ (→ P R) (STV 0.6333333333333334 0.7))))
```

The deduction truth value is computed *inside the type* during inference, and backward search returns the proof term with that computed type. Modus ponens and recursive evidence-counting rules run the same way. (The artifact writes the annotation constructor with the glyph U+225E, rendered ≐ here.)

Checked refinements. `validate-type-properties` runs declared per-index refinements: with a natural-index property declared for `VecN`, `(VecN String 2)` is accepted and `(VecN String -1)` is rejected with `(type-property-failed -1)` — an underflow that unrefined HE typing happily derives. The operation validates declared properties only; it is *not* a well-formedness judgment (no universes, no binder-domain formation, no positivity).

Open. A C↔Lean correspondence for this module; Σ/Id/ι formers and universe discipline (the module does dependent *elimination* plus checked refinements, not full dependent type theory); certificate generation; backward search over computed codomains whose truth values still contain free variables.

16 Generated Readers: the LanguageDef as Syntax Authority

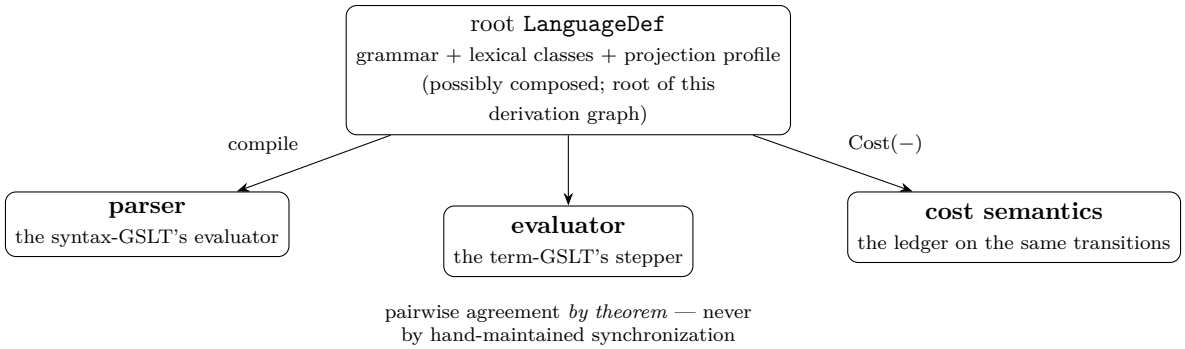
A tranche begun in July 2026 replaces hand-maintained readers with parsers *generated from the source grammars themselves*. Each language's concrete syntax is written as a `LanguageDef` (grammar productions, lexical scalar classes, and an atom-projection profile); a validating compiler normalizes the `LanguageDef` into a parser intermediate form, classifies it into an admissible deterministic fragment, and emits native C readers linked into CeTTa. General GLL/GLR engines are retained as differential validators and as fallbacks for grammars outside the deterministic fragment.

- **Per-language ownership.** The HE family, a two-stage PeTTa reader (document splitting, then per-form parsing — explicitly a syntax-only work-in-progress, not PeTTa evaluation semantics), and the Prime syntax each compile from their *own* LanguageDefs into a shared projection-neutral runtime; no language is routed through another language's grammar.
- **The specification catches its own bugs.** The source-faithfulness classifier rejected two genuine defects *in the source grammars* before execution: a token-start class that wrongly admitted `@` (making `@doc` both a quotation and an ordinary word), and a derivation ambiguity in which `*(e)` parsed both as a bare `*` followed by an expression and as one unquote atom. Both were repaired at the specification level — the relation was fixed, not the validator weakened — and re-verified in an independent Horn-clause model of the grammar.
- **Evidence.** Compiled-versus-legacy differentials (76/76 on the Prime syntax, including Unicode, numeric projection, open-name rejection, and alpha-equivalent variable identity), compiler mutations killed, purity gates showing zero guest-language leakage across the generic engine and compiler files, and byte-identical regeneration of generated artifacts from their sources.

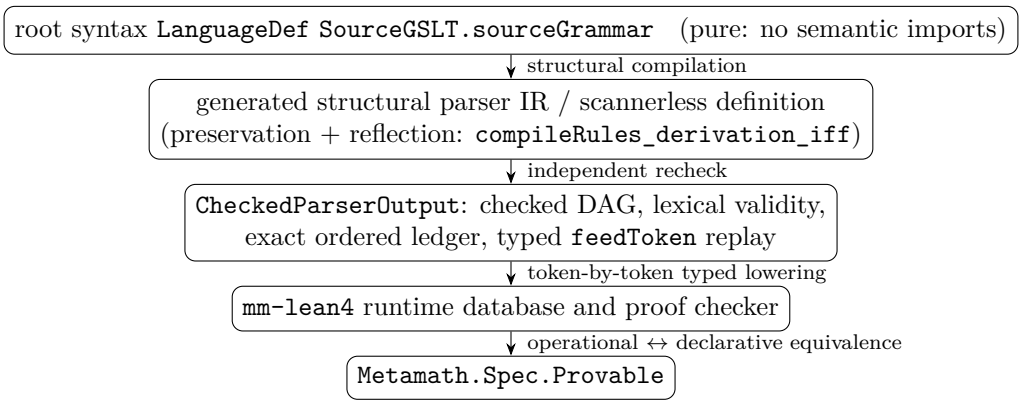
The architectural claim is deliberately bounded: this is certified *specialization* of admitted grammar schemata, with every generated component remaining an untrusted producer subject to differential and (ultimately) checker replay — not yet a general compiler from arbitrary grammars. Measured on the earlier prototype lane, a generated HE reader ran at 0.75×–0.87× the hand-written reader's time on standard-library parsing with exact canonical-tree agreement, indicating that grammar-as-authority and native performance are not in tension.

16.1 GSLT2Parse: syntax is a GSLT

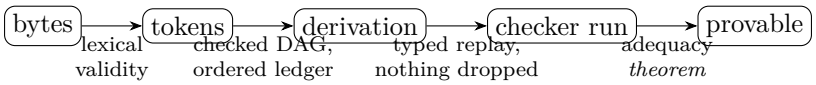
The engine tranche above is the executable half of a formal programme we call *GSLT2Parse*, whose thesis is compositional: a language's *syntax* is itself a generated syntactic labelled transition system, and a parser is that syntax-GSLT's evaluator. One root `LanguageDef` is therefore the single root from which the evaluator, the parser, and (through the Cost construction of Section 10.7) the resource semantics are all mechanically derived — three semantic artifacts, one authority, no hand-maintained agreement obligations between them:



The compositional discipline is enforced starvation: no derived component has any information channel back to the language except the proved interfaces, so there is nothing left *to* drift. The formal side lives in the Mettapedia Lean development and is sorry- and axiom-clean across the roughly fifteen-thousand-line module family; the axiom reports below were re-checked against the kernel when this section was first written. The pipeline, for the Metamath instance:



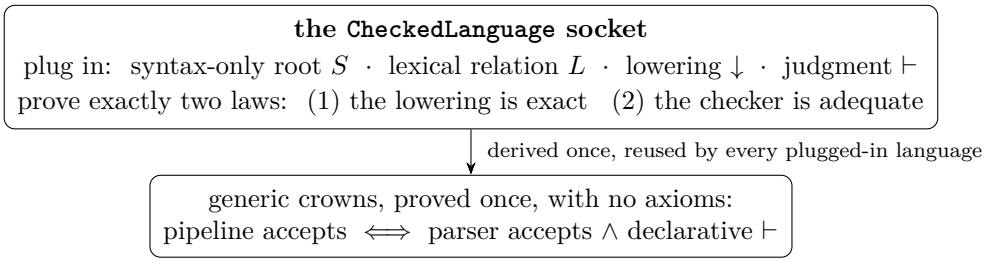
Parsing and checking remain separate judgments at every stage; the composition modules join them by theorem. Generically, the same chain reads as a sequence of seams, each crossed only inside a certificate:



every seam a separate judgment with its own checkable certificate;
the final equivalence is a theorem, never a trust relationship

The generic compiler and its two-sided correspondence. A generic, language-agnostic compiler turns the grammar fragment of any `LanguageDef` into structural parser rules. Three theorems bound its authority. Changing any non-syntax field of the declaration cannot change the generated rules (`compileRules_eq_of_terms_eq`); generated-rule actions retain the independently decoded terminal/child skeleton, so a rule cannot silently drop, duplicate, or reorder children (`compileRules_action_shape`); and — the load-bearing result — derivability in the generated structural intermediate form is *equivalent* to derivability in the source grammar at the ordered-token boundary (`compileRules_derivation_iff`: preservation *and* reflection, a genuine second relation rather than equality by definition; kernel axioms exactly `propext`, `Classical.choice`, `Quot.sound`; `GSLT/Parsing/LanguageDefSyntaxCorrespondence.lean`).

Checked languages: two laws, crowns for free. The reusable `CheckedLanguage` interface packages what any language must supply to close the loop from bytes to meaning: a syntax-only root, a lexical relation, a lowering into a checker, and a declarative judgment, subject to exactly two laws — exact lowering and checker adequacy. From those two laws the certificate-indexed and whole-pipeline crown theorems (`pipelineAccepts_iff_parserAccepts_and_declarative` and kin) are derived *once, generically*; the generic composition layer (`GSLT/CheckedLanguage.lean`) proves them with *no axioms at all*. Parsing and checking remain separate judgments throughout: the composition joins them by theorem, and neither the parser nor its certificate is ever allowed to authorize a theorem.



The expensive theorem is proved once, about the socket; each further language pays only its own two laws. That, rather than any property of Metamath itself, is why the Metamath instance below is a template.

The Metamath vertical slice: a parser-to-provability crown. The first full instance runs from raw Metamath bytes to formal provability. The root Metamath syntax `LanguageDef` is a *pure* grammar — a semantic firewall: it imports only the generic grammar-inference layer, and the generic compiler contains no Metamath-specific names. On top of it, `CheckedParserOutput` demands an independently checked grammar DAG, lexical validity, exact ordered-token agreement, and a successfully constructed verified reader; `TypedLoweringCertificate` pairs every classified source token, in order and without insertion or omission, with an actual `ParserState.feedToken` transition of the verified `mm-lean4` checker, and proves that the instrumented execution erases to the production checker database. The crown (`generatedParserAndCheckerAcceptance_iff_specProvability`) then states that generated-parser-plus-checker acceptance is equivalent to declarative acceptance by `Metamath.Spec.Provable` — kernel axioms again exactly `propext`, `Classical.choice`, `Quot.sound`. The reverse direction cannot manufacture syntax evidence: it is indexed by an existing checked parser output whose certificates have already passed their independent checkers. The crown, verbatim from `SourceGSLTCheckerAlignment.lean`:

```
def GeneratedParserAndCheckerAccepts
    {bytes : ByteArray} (output : CheckedParserOutput bytes)
    (label : String) (formula : Metamath.Verify.Formula) : Prop :=
  TypedLoweringCertificate bytes output.source /\
    ImplementationAccepts bytes label formula

theorem generatedParserAndCheckerAcceptance_iff_specProvability
    {bytes : ByteArray} (output : CheckedParserOutput bytes)
    (label : String) (formula : Metamath.Verify.Formula) :
    GeneratedParserAndCheckerAccepts output label formula <->
      DeclarativeAccepts bytes formula
```

Executable closure evidence: seven checked ordered ledgers (demo0, MIU, Peano, disjoint-variable and proof-format cases, and an authentic propositional `set.mm` slice) in exact object-level agreement with independent `mm-lean4` output; normal/compressed proof agreement; five malformed-source rejections; killed mutations across labels, disjoint-variable conditions, compressed proofs, declaration order, AST, lexer, and semantic references; and byte-identical regeneration of both generated Metamath artifacts from the Lean root.

From parser-GSLT to checker-GSLT: source-owned inference. Since the vertical slice above was sealed, the same root `LanguageDef` has grown a second semantic derivation: Metamath *checking* itself as a source-owned GSLT. `SourceGSLTOperations.lean` is the single declared bridge from the source grammar to the streaming fold and indexed-checker operations: node contracts are *computed* from the parameter sorts of the source productions, and lexical references resolve through the same literal tables that generate the parser. `SourceGSLTState.lean` gives those operations their scoped-state semantics (frames, scope closing, global object identities) with no reference to the `mm-lean4` runtime database, and `SourceGSLTNormalTheorem.lean` admits a normal-theorem transition only through an exact source-owned proof-occurrence tree over the pre-insertion database prefix — independent of the `mm-lean4` execution function. Operational and declarative adequacy (`SourceInferenceOperationalAdequacy`, `SourceInferenceDeclarativeAdequacy.lean`) connect this source-derived checker semantics to Metamath's declarative semantics with every database, frame, declaration, and disjointness premise derived from the same validated source prefix; the crown (`NormalTheoremStep.toSemanticProvable`) was re-verified for this section at kernel axioms exactly `propext`, `Classical.choice`, `Quot.sound`, with the module family sorry-free. Export modules (`SourceGSLTParserExport`, `SourceGSLTOperationsMeTTaExport`) regenerate both engine-facing artifacts from the root, and a native inference-staging consumer (`lib_parse_inference_native`, with its benchmark harness) carries the operations export into the engine. The Metamath instance is thereby the template the programme intends to replicate: *one* root `LanguageDef` from which the parser, the checker operations, and their adequacy theorems are all derived, ready to be instantiated across a curriculum of further language definitions.

Declaration-level co-evolution with the shipped checker (August 2026). The source-owned semantics above has since been joined to the verified implementation at the level of individual transitions. Every accepted source declaration step — `$c`, `$v`, `$d`, `$f`, `$e`, assertion insertion for `$a`/`$p`, and scope push/pop/completion — is now matched by the *shipped* `mm-lean4` database operation it corresponds to (the per-token declaration inserts, the checked hypothesis insertion, the axiom insertion including its mandatory-frame trim), and the whole statement fold co-evolves with the implementation database from the initial states with no supplied premises (`default_initial_applyPayloads` in `SourceGSLTRuntimeCoEvolution.lean`). Two identifications carry the weight: the implementation's mutable mandatory-frame computation is proved equal to the specification's mandatory frame (`trimFrame'_eq_mandatory`), and the accepted pipeline's own classified stream supplies its lexical evidence (`runSource_selfLexicalized_derives`), so neither the frame nor the token boundary is ever assumed. A deliberate honesty distinction structures the remaining work: results proved from one accepted source run and one checked byte representation are kept as *parallel* certificates until the classified stream is bound to the exact bytes the verified reader consumed, each `$p` obligation's normal or compressed proof is discharged from the reader's own chronology (with verified and incomplete-`?` outcomes as disjoint results), and the source fold is related to the production reader by a prefix-wise simulation in which reader microsteps are administrative and completed statements are the observable steps. The normal-proof discharge has been constructed

against the production reader trace; the compressed analogue and the whole-prefix simulation are in active development.

Second consumers and the engine loop. Megalodon has an honest second-consumer skeleton that reuses the generic signatures and crown — deliberately *not* yet claimed as an instance, since its verified checker and adequacy laws remain to be built. On the engine side, the generated Metamath grammar artifact is consumed directly by CeTTa (`lib_parse_metamath_grammar_generated_v0.metta`) alongside the `lib_parse` family and the signature-driven ABT binder layer in the Prime integration lane; the full engine-side parse gate (Metamath, ABT, and native-reader tests) was rerun green for this section. Honest boundaries: the tested `set.mm` artifact is a nontrivial authentic slice, not the full corpus; raw byte-to-token completeness is secured per accepted certificate by exact typed reader replay, with a universal raw-byte characterization theorem remaining future work; and the declaration-level co-evolution above becomes a single bidirectional statement only once the same-`LanguageDef` binding, per-`$p` discharge, and prefix simulation land, as itemized in the preceding paragraph.

16.2 `lib_tptp`: standard formats around pluggable calculi

TPTP is a family of standard problem and derivation formats, not a theorem prover or a mandate for one proof-search strategy [19, 20]. This distinction determines the library boundary. `lib_tptp` owns parsing, admitted semantic carriers, format transformations, and derivation-DAG verification plumbing. A resolution or superposition searcher belongs in an example or an explicitly named prover package; a TSTP rule belongs to a declared calculus service whose soundness is checked independently.

The current native input boundary. The checked-in versioned TPTP `LanguageDef` generates a native ParserPack for the official syntax family and a source-defined syntax-tree-to-abstract-syntax projection. The maintained MeTTa API exposes four useful stages:

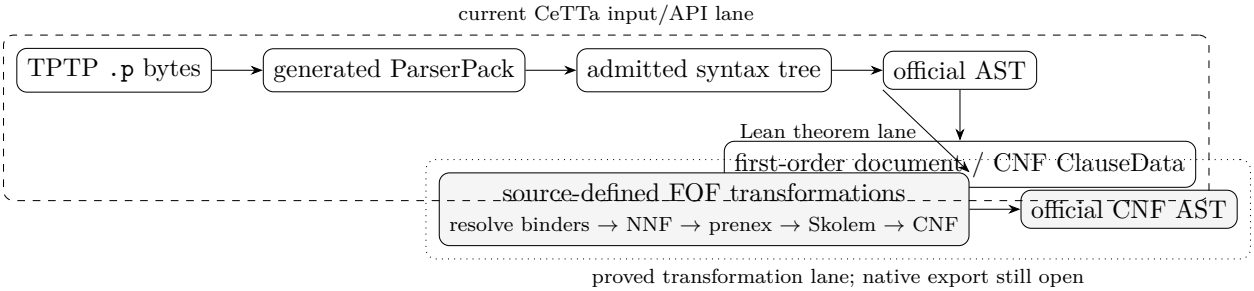
- `tptp:file->syntax-tree`;
- `tptp:file->abstract-syntax`;
- `tptp:file->first-order-document`;
- `tptp:cnf-file->clause-data`.

Each returns an explicit stage value or stage fault, and each published value is admitted against its own `LanguageDef`; no result is recast as a legacy `ATPInput`. The syntax gate includes representative TPI, THF, NHF, TFF, NXF, TCF, FOF, CNF, include, annotation, numeric, Unicode, and source-span cases. For example:

Listing 15: Representative inputs accepted by the versioned TPTP grammar

```
thf(identity,axiom,![P:$o]:(P=>P)).
tff(typed,axiom,![X:$i]:q(X)).
fof(first_order,axiom,![X]:(p(X)=>?[Y]:q(Y))).
cnf(clause,axiom,p(X)|~q(X)).
include('Axioms/SET003+0.ax',[first_order]).
```

Parsing all of these families is not a claim that all of their semantics are already implemented. The deeper admitted transformation surface is currently first-order FOF/CNF. The four-profile executable gate checks the public examples under Prime, PeTTa, HE-extended, and HE-prime; the umbrella target also checks artifact-digest closure and mutations that remove or alter load-bearing generated rules.



FOF-to-CNF as a compositional pipeline. On the theorem side, official FOF elaborates into named FOF, resolves to a locally nameless binder representation, normalizes to NNF and prenex form, Skolemizes with explicit generated-symbol evidence, introduces definitional names, creates source-indexed clause batches, allocates collision-free TPTP names, and serializes to the official CNF AST. Each arrow is a declared GSLT with an independent recursive semantics and exact agreement/no-invention theorems. The composed `endToEndExact_exists` theorem obtains a serializer result for every evidence-bearing admitted batch. It does *not* yet produce concrete text; canonical printing and the parser/printer round trip remain the next format boundary.

The shadowing fixture

```
fof(shadowing,axiom,![X]:(p(X)=>?[X]:q(X))).
```

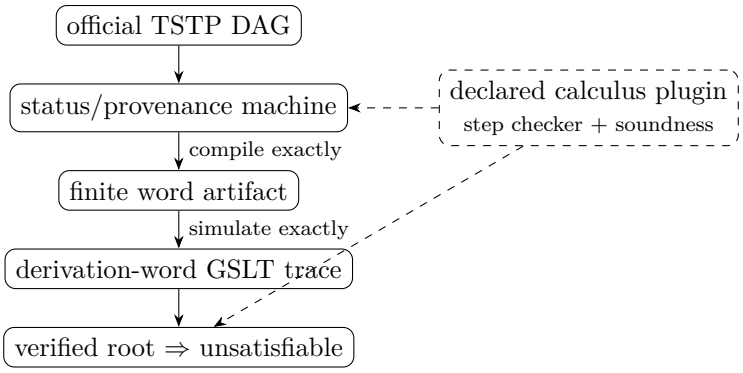
is the capture-avoidance canary: `q(X)` resolves to the inner binder. Malformed numeric applications, unallocated variables, duplicate lexical keys, and generated/original symbol collisions are rejected by negative theorems rather than repaired by the serializer.

TSTP verification through a compact word language. The first logically closed profile accepts this ordinary ground-resolution derivation:

Listing 16: The ground TSTP canary

```
cnf(p_or_q,axiom,p|q).
cnf(not_p,axiom,~p).
cnf(not_q,axiom,~q).
cnf(q,plain,q,inference(resolution,[status(thm)],[p_or_q,not_p])).
cnf(empty,plain,$false,
    inference(resolution,[status(thm)],[q,not_q])).
```

Admission retains node identity, roles, source annotations, parent order, and `status(thm)`. A status-indexed projection produces the semantic program. A generic compiler replaces formulas, rules, evidence, provenance, obligations, and the finite service state with proof-local arena references and seven compact records. Decoding those words recovers the exact semantic program. A shared execution invariant then supplies a continuous trace in the actual derivation-word `LanguageDef`, and the selected ground-resolution service's independent soundness theorem turns its verified empty root into unsatisfiability.



This design follows a maximal-native-calculus principle without hardcoding resolution into CeTTa: the graph/word machinery is calculus-neutral, while each calculus supplies its most direct checker and semantic theorem. The current MeTTa resolution example deliberately begins only after `lib_tptp` has produced admitted ClauseData. Its unification, bounded search, trace object, and independent trace checker live together under `examples/atp/tptp_resolution`; neither search nor its local `fo-proof` representation is a public TPTP API.

What remains before this is a native general TSTP verifier. The current closed theorem covers one ground `status(thm)` resolution profile. General TSTP support requires status-sensitive composition for `thm`, `cth`, and `esa`; a registry of independently sound calculus services; finite arenas for evolving service state; include-DAG resolution; canonical concrete printing; E/Vampire ingestion and reconstruction; and typed TFF/TCF/THF/NHF semantic carriers. The word checker must then be lowered by a source-defined transformation to StructuredC with a commuting/reflection theorem. Until that final step, the compact word model is the proved native target shape, not a claim that the production C implementation has been derived from it.

16.3 MeTTa Zero: a two-operation kernel with native proof evidence

MeTTa Zero is the compact executable specimen for extending the preceding programme from parsing to language semantics and compilation. Assuming s-expressions with variables, an occurrence bag S, matching with substitution, and a declared grounding capability, its kernel has only two public semantic operations. Write v for a fresh wildcard and abbreviate the interpreted result bag by $R_S(x)$:

$$\mathsf{query}_S(p,t) = \biguplus_{a \in S} \{\, t\theta \mid \theta \in \mathsf{match}(p,a) \,\}, \tag{1}$$

$$R_S(x) = \left(\biguplus_{(=\, l\, r) \in \mathsf{query}_S(v,v)} \{\, r\theta \mid \theta \in \mathsf{match}(l,x) \,\} \right) \uplus \mathsf{ground}(x), \tag{2}$$

$$\mathsf{eval}_S(x) = \begin{cases} R_S(x), & R_S(x) \neq \varnothing, \\ \{x\}, & R_S(x) = \varnothing. \end{cases} \tag{3}$$

Equations are therefore ordinary queryable atoms, not entries in a private rule store; evaluation obtains them through the same public query as user programs. Grounding is a declared library capability, and a capability's decline leaves an uninterpreted subject inert rather than turning ignorance into an empty result. The coproducts are occurrence coproducts: duplicate facts, rules,

matches, and answers remain distinct. A recursive semantic runner, continuations, pending work, and `reify` compose over this kernel; they are not silently folded into Equations 1– 3. The current runner is consequently retained as a versioned experimental profile while its abstract laws and cross-dialect interpretations are established.

One semantic root, three derived uses of native types. The Lean development sends Zero's actual composed `ExtendedLanguageDef` and relation environment through OSLF, rather than constructing a parallel transition relation:

Zero language definition $\longrightarrow$ relation-backed OSLF span $\longrightarrow$ native types and certificates

The resulting information has three distinct, proved uses.

1. **Spatial classification.** Generated headed types recognize equation atoms and the exact query/evaluation request and answer constructors. The constructor list is generated from the language's term grammar rather than maintained as a second inventory.

2. **Behavioral evidence.** Exact query and evaluation diamond theorems identify the OSLF diamond with existence of an answer in the corresponding Zero occurrence bag. Proof-relevant finite certificates are sound and complete for this admitted fragment. Generic predecessor-universal `box` evidence deliberately fails closed until finite predecessor enumeration, an invariant, or coinductive evidence is supplied.

3. **Compilation analysis.** Constructor/arity head signatures select the query or evaluation production without inspecting every rule. Unsupported forms take the complete rule-table fallback. The selected plan is proved equal both to the generic executor and to the source Zero interpreter under the exact occurrence-bag observer (`executeAuthoredNativePlanned_eq`, `nativePlanned_eq_authored`). Thus the analysis establishes a safe shape for generated native dispatch; exporting that index into the C plan ABI is an implementation step, not a new semantic decision.

The distinction between the second and third uses is load-bearing. A diamond records that at least one suitable transition exists; it does not record how many occurrences produced it. The negative theorem `diamond_equal_but_edge_counts_differ` exhibits two spans with the same diamond observation and different edge multiplicities. Native types may therefore guide dispatch and justify existence claims, but they cannot replace Zero's bag observer.

The generic native-type construction is in `GraphTheory/Walk/NativeTypeTheory.lean`; the concrete Zero theorems are in `MeTTaZeroLanguageAdequacy.lean`.

From a trace to an independently replayed proof. CertificateGSLT's open derivations now carry an ordered context of explicit premises, with a generic semantic-induction theorem: if every local rule application preserves meaning and every open premise has its declared meaning, the derived conclusion has its meaning. Zero instantiates this interface with one admitted `LanguageDef` of spatial rules and two behavioral liveness rules, one for query rows and one for evaluation rows. A liveness rule cannot mint an answer edge by itself; it requires a query-row or evaluation-row premise tied to an actual occurrence in the interpreted Zero relation.

A generic waist (`OSLF/Framework/OSLFCertificateGSLTAuthority.lean`) turns an accepted open `WireArticle` into semantic authority for any OSLF-generated native type system, and Zero specializes it to its MIK authority, exposing the corresponding checked-lowering obligation for

a native NIK. A positive article plus its true row premise entails the OSLF behavioral claim (`sample_article_semantic`); the negative canary changes the article's premise reference to an out-of-range occurrence and is rejected (`sample_wrong_premise_rejected`). This is proof-producing compilation structure, not merely a diagnostic trace.

The current C boundary. The executable Zero compiler separately emits a deterministic, hash-linked build certificate for

$$\text{source LanguageDefs} \longrightarrow \text{admitted rules} \longrightarrow \text{CGP1 plan} \longrightarrow \text{generated C/H bytes.}$$

An independent build-time C checker rereads the sources, replays admission, compares every compiled rule structurally, and checks the exact generated artifacts; tampering and nondeterministic regeneration are gated. This checker is not linked into the ordinary CeTTa executable, so normal execution pays no certificate-parsing or trace-retention cost. The claim is nevertheless bounded: the certificate proves structural chain of custody, not yet that executing CGP1 or the generated C realizes all declared Zero observations. The open row premises above are not yet closed by authenticated C execution receipts, and the C checker is not yet generated from the single CertificateGSLT acceptance predicate. These are the remaining seams between the working build attestation and end-to-end NIK replay. The C checker is independent of the Python producer, but deliberately reuses the generic C parser, admission path, and plan decoder; the resulting common-mode boundary is another reason the CertificateGSLT replay correspondence remains necessary.

17 Related Work

Concurrent constraint programming. Saraswat et al. [2] established the theoretical framework for detecting stable properties of networks in concurrent logic programming languages. Their cc(↓,|) language, with Tell/Ask on shared variables over a monotonic constraint store, provides the semantic model that most closely matches HE/CeTTa's `match &self` evaluation. The distinction between monotonic constraint stores and independent-scope evaluation (Flat Concurrent Prolog) precisely characterizes the semantic boundary between HE and PeTTa.

Coordination and the interaction basis. The commit of Section 7 is a double-pushout rewrite with application conditions in the sense of algebraic graph transformation [22], specialized to bags of occurrences. Its derived forms recover the classic coordination languages: Linda's tuple space [23], where `in` is a take and `rd` a keep; Constraint Handling Rules [24], whose simplification and propagation rules are takes and keeps with a history read as absence; the join calculus [25], whose join patterns are multi-take commits on one location; software transactional memory [26], whose `retry` parks a commit on what it read and whose `orElse` is an ordered choice; and the reflective rho calculus [4], whose COMM is two takes. What the basis adds to these is a single form in which predicate queries are a separate, declared part with their own stability discipline, and in which the laws a region enjoys follow from the rights others lack there.

Incremental computation. The revision tokens of Section 14 follow the dependency-tracking shape of Salsa, rustc's query system, build systems à la carte, Adapton and differential dataflow [14, 15, 16, 17, 18], coarsened to what MeTTa's occurrence identities and translation stages need.

MeTTa ecosystem. The MeTTaIL boot compilation pipeline [3] provides a contract-first, Lean-backed compilation path for MeTTa stdlib modules, complementing CeTTa's direct evaluation approach. The Mettapedia formalization project provides the Lean specifications (`EvalSpec`, `MeTTaEval`, `MatchSpec`) that ground both the translation proof and the CeTTa evaluator design. JeTTa compiles MeTTa to JVM bytecode [10, 11], and MORK runs the MM2 rewriting language over PathMap tries; both appear in this paper's comparisons. The PLN (Probabilistic Logic Networks) framework uses MeTTa's backward chaining extensively; the typed chainer pattern from Nil Geisweiller's `nilbc.metta` is the standard PLN idiom that works across all three runtimes.

18 Conclusion

CeTTa is now one binary for several MeTTa dialects, each a named semantic product with its own oracle. The listings of Section 2 show both what the dialects share and where they differ, and the differences are principled: inert data against Prolog failure, argument types that decide evaluation, call by need, and an editable standard library. The semantics is kept honest by result-bag conformance against upstream HE and SWI-PeTTa, by a reflective modal layer, and by a rule table whose removal tests show the evaluator depends on its declared rules.

The lines of development divide the work. `main` is the published baseline of August. Opt does the careful fixes: in October it realized `transform` natively over its MeTTa library, made space drains linear, brought the cost of crossing between machines toward the cost of an ordinary step, and built the native query path, set spaces and joins with which ordinary MeTTa programs now run MORK's own application tasks faster than standalone MORK, with the same result sets (Appendix A.2). Pre-main unifies execution across the machines behind one call and return contract, as the candidate for the next `main`. The NIK line grew native kernels and native application libraries. The Prime draft develops Prime in C in step with its metatheory, has absorbed the weights, cost and parallel line and `cetta-ours`, and runs the interaction basis as a native kernel. The measured results are honest in both directions. In the November measurements CeTTa's PeTTa lane is faster than SWI-PeTTa on all but one of the timed PeTTa examples and its HE lane faster than Hyperon Experimental on every shared program both complete, while MORK's leapfrog join ties CeTTa on two of its 36 showcase workloads and needs less memory on 17 of them, and the ai-mathematician port's premise selection does not run on the preview binary. On the panel of 23 October CeTTa was still slower on foreign-data reads, the chainer cells, the Ambient program and a rho cell; the Prime lane is the slow one, and its need machinery is the cost.

For spaces and channels the direction is one interaction form. Its theory is largely proved; its surface runs in the Prime draft; its native runtime, with stable occurrence identities, per-occurrence claims, per-region epochs and a wait index, is the main thing still to build.

Remaining work.

- **Opt.** Make the ai-mathematician port's search preparation answer on the preview, as it does on the NIK line's binary; close the measured losses of Appendix A (the two showcase ties with MORK's leapfrog join, the 17 showcase workloads on which MORK needs less memory, and `patrick_iterate_quad.metta`, on which SWI-PeTTa is twice as fast); and settle the two PeTTa-reference decisions that the readiness run left open.
- **Pre-main.** Commit the staged repairs, run the exhaustive `main` qualification, and advance `main`.

- **Prime draft.** Repair the deviations of Section 8.4, with the weights line repairing its thirteen fixtures; make the occurrence kernel the single engine for `transform` on both lanes; build the native runtime of the interaction basis; and close the authority gaps: ticket requirements that a child grant may drop, names that confer their holder's rights, and an effect capability in place of the evaluator's thread-local flag.
- **HE one-step certification.** Certify the five live rules that are still bag-tested (`let`, `let*`, `case`, `switch`, `switch-minimal`), through the matcher bridge of Section 4.9, and carry the ground-query adequacy result to queries that contain variables.
- **Zero native kernel closure.** Generate the native checker from the one CertificateGSLT acceptance predicate, authenticate query-row and capability receipts, and compose the build certificate with an execution receipt.
- **Reaction semantics.** Extend the witness-free characterization of evaluation at communication to concurrent and persistent reactions.

A Application Measurements: 5 November 2026

This appendix measures the November preview against the reference implementation of each comparison: standalone MORK on its own MM2 programs, SWI-PeTTa on the PeTTa examples and on a learning program, Hyperon Experimental on shared HE programs, and the Python ai-mathematician on its premise-selection stage. Every timed run was checked: a run counts only if its answers agree with an independent oracle or with the reference's complete result set, as stated with each table, and a run that did not finish is reported as such, never as a time. Measurements from earlier dates are kept only in Section A.10, separate from these.

A.1 Systems and method

CeTTa. Version 1.5.0-dev, the default build (Python support, no MORK bridge), built from pre-main commit `2ca1c923` with the preview repairs then staged; executable SHA-256 prefix `18091c10633b`. The PeTTa lane is `--lang petta`. The Opt suite and the MORK side-by-side programs also select `--profile extended`, whose `(new-space set)` gives the set semantics MM2's spaces have.

MORK. Commit `1c44f70` (the current upstream `main`) with PathMap `fb979b4`, release builds with nightly Rust. The *default* build joins relation by relation through MORK's product zipper (prefix `88c740354eae`); the *leapfrog* build enables MORK's experimental variable-at-a-time leapfrog join for flat conjunctive bodies (prefix `0e5082db27cb`). Each run is `mork run IN.mm2 OUT.mm2 --instrumentation 0`.

SWI-PeTTa. PeTTa commit `b74af1b` with local, uncommitted fixes (typed calls use only declarations of their own arity, an empty `superpose` fails, reader and launcher changes), on SWI-Prolog 10.1.9, run as `run.sh FILE --silent`.

Hyperon Experimental. `metta-repl` 0.2.10 built from the fork branch `fixes` (`b1b2206a`): upstream release 0.2.10 plus three fixes (imports, space index and numbers; argument type errors; evaluated flags after nested substitutions). Prefix `32b45f7dbeec`.

Machine. A shared workstation with an AMD Ryzen 7 9800X3D (eight cores, sixteen logical CPUs) and 59 GiB of memory, Linux 7.0. Other agents' jobs kept its one-minute load average between about 5 and 21 at the starts of the measurement batches, and above 20 at times. Each batch waited briefly for a quieter moment and then ran every job pinned to the logical CPU that was least busy at its start; the start load of every batch is recorded with the data. Repeated trials varied little (MORK's backward chainer, for instance, took 14.4–14.8 s), but the absolute times are those of a shared machine. Jobs ran with `RAYON_NUM_THREADS=1` and an unlimited stack for every engine (MORK recurses over term depth; with the default stack its unary-clock programs abort).

Every cell is the median of three fresh processes, run in alternating engine order; times are in seconds unless a column says otherwise. *Wall* is the whole process: start-up, loading, execution and output. *CPU* is user plus system time. *MiB* is the peak resident set. Some batches also counted user-mode instructions, whose counter adds about 7 ms to each wall time. In the application table, CeTTa's and the leapfrog build's times come from runs without the counter; the default build's and SWI-PeTTa's include it, which matters only for their cells below 0.3 s. The scale ladder counted throughout. MORK's *execute* column is the execution time MORK itself reports; it excludes reading its input and writing its final store, which it serializes completely, while the MeTTa programs print only their relation sizes. The MORK columns therefore bracket its time: the execute column alone is the comparison least favourable to CeTTa, whose cells always include loading and output. Ratios in the text compare CeTTa's wall time with MORK's execute time.

A.2 The Opt application suite

The suite pairs nine tasks written as MM2 programs with ordinary PeTTa programs over the same generated facts. The MM2 programs are MORK's: the two-hop and triangle join, the five-clique query, the process calculus, the odd–even sort and the backward chainer `bc3` come from MORK's kernel benchmarks (several at MORK's own benchmark sizes), the counter machine from its resources, and Hexlife and the path-producing reachability from its tutorials. The PeTTa programs are idiomatic solutions of the same tasks, not translations of the MM2 rules, and three of them use a different algorithm: the counter machine runs as a function over the machine state that records every state, where MM2 fires one clocked rule per step; backward proof search is a depth-bounded top-down search per target, where MM2's `bc3` saturates goals and evidence on a clock; and Hexlife counts each candidate cell's live neighbours directly, where MM2 uses count sinks. The comparison is between two systems' solutions of one task, checked to compute one result relation. The graph workload computes the distinct two-hop endpoint pairs and the directed closed triples of a random graph; it is not a fixed-point transitive closure.

Checks. Before any timing, every pair was run with complete output: the full set of every result relation, alpha-normalized, with the process calculus's `!`, `?` and `|` renamed to its PeTTa constructors, was compared with MORK's final store. The comparison was made at the small control sizes and again at the timed sizes, where the largest set has 1,080,571 rows; all 27 runs at each size agreed for CeTTa, SWI-PeTTa and the default MORK build, and the leapfrog build agreed too. Every timed run then checks every relation's size against MORK. Independent Python computations confirm the graph counts (bit sets or hash sets: 1,064,228 pairs and 16,343 triangles at 49,727 edges; 4,794,018 and 16,961 at 200,000 draws; 19,913,332 and 8,935 at one million; 99,909,346 and 8,356 at five million), the five-clique counts (79 and 637) and the counter machine's state rows (an integer machine: 16,218 at register value 300, 54,018 at 1,000).

A corrected program. The complete-set check exposed an error in the original PeTTa counter program: it stored `(add-atom &m (state $pt (IC (peano $ic))))`, and an atom being added is data, so the instruction counter held the unevaluated `(peano $ic)`. SWI-PeTTa and CeTTa agree on that reading. Row counts matched MORK's, but at register value 3, 30 instruction-counter rows differed. The corrected program binds the computed numeral first. Earlier counter timings measured the faulty program and are withdrawn.

	CeTTa	MORK default		MORK leapfrog		SWI-PeTTa
application	wall	execute	wall	execute	wall	wall
Two-hop pairs, triangles; 49,727 edges	0.109	0.881	0.997	0.612	0.724	4.12
Five-cliques; 200 vertices, 3,600 edges	0.006	18.68	18.69	0.032	0.037	18.28
the same, MM2 in early order	0.006	0.094	0.103	0.032	0.038	
Odd–even sort; 300 items	0.059	0.141	0.157	0.142	0.152	1.33
Backward proof search; 1,000 targets	0.028	14.48	14.49	5.97	5.98	0.258
Hexlife; 80 cells, 20 generations	0.038	17.65	17.66	17.31	17.32	0.171
Ancestry; 1,000 people	0.172	1.25	1.39	0.439	0.558	12.27
Reachability with paths; 500 states	0.020	6.93	6.95	7.55	7.56	0.133
Counter machine; register 300	1.14	100.4	100.5	7.67	7.83	1.71
Process calculus; 200 + 200	0.285	23.77	23.79	5.19	5.20	24.75

	CeTTa		MORK default		MORK leapfrog		SWI-PeTTa	
application	CPU	MiB	CPU	MiB	CPU	MiB	CPU	MiB
Two-hop pairs, triangles; 49,727 edges	0.10	27.7	0.96	62.0	0.70	62.4	4.10	686
Five-cliques; 200 vertices, 3,600 edges	<0.01	13.3	18.67	6.4	0.03	6.7	18.25	29.4
the same, MM2 in early order	<0.01	13.0	0.09	6.5	0.03	6.6		
Odd–even sort; 300 items	0.05	18.3	0.14	8.4	0.14	8.4	1.31	178
Backward proof search; 1,000 targets	0.02	14.4	14.47	8.8	5.97	12.1	0.24	38.0
Hexlife; 80 cells, 20 generations	0.03	14.5	17.60	6.7	17.30	6.8	0.15	22.5
Ancestry; 1,000 people	0.16	48.6	1.37	20.9	0.54	21.0	12.25	898
Reachability with paths; 500 states	0.01	15.1	6.93	37.6	7.55	37.9	0.12	28.0
Counter machine; register 300	1.13	22.6	100.5	36.4	7.81	26.7	1.69	1093
Process calculus; 200 + 200	0.28	15.8	23.77	11.8	5.19	11.6	24.72	48.1

Reading the table. CeTTa's whole process is faster than MORK's execution alone in every row, against both builds. Against the faster of MORK's two builds for each task the factor is 2.4 for the odd–even sort and 2.5 for ancestry; 5–7 for the five-cliques, the two-hop pairs and the counter machine; 18 for the process calculus; and 200–460 for backward proof search, reachability and Hexlife. For those three and for the counter machine the two programs use different algorithms, so their factors compare solutions rather than engines. MORK's two builds differ by up to 580 times (the five-cliques in the benchmark's order) and 13 times (the counter machine); the leapfrog join is slower than the default one only on reachability (7.55 against 6.93 s) and ties on the sort, so a comparison with one build alone would misstate MORK. CeTTa uses more memory than MORK on most of these small inputs, where its start-up footprint of about 13 MiB dominates, and less on the two-hop pairs, reachability and the counter machine. SWI-PeTTa is faster than both MORK builds on the four tasks whose PeTTa programs search differently, slower than MORK's faster build on the other five, and needs up to 1.1 GiB where CeTTa needs under 50 MiB. User-mode instruction counts, taken in a separate batch of the same programs, are independent of the machine's load and tell the same story: 0.04 billion for CeTTa's whole five-clique process against 419 billion for

MORK's default build in the benchmark order and 2.1 billion in the early order, and 1.9 against 17 billion for the two-hop pairs. On the counter machine SWI-PeTTa executes fewer instructions than CeTTa (20 against 27 billion) and still takes longer.

Order of a conjunction. MORK's default join follows the written order of an `exec` body. The five-clique row uses MORK's own benchmark query, which lists the ten edge conditions lexicographically. Written so that each new vertex's edges are checked as soon as it is introduced, the same MM2 query is the row marked "early order"; the PeTTa program is unchanged, since CeTTa chooses its own join order. With that order MORK's default build executes the query in 0.094 s instead of 18.7 s; the leapfrog build takes 0.032 s in either order, and CeTTa 0.006 s.

A.3 The scale ladder

The larger rungs of the same tasks, against MORK's leapfrog build: on the base tasks above it was the faster of MORK's two builds, or level with the default one, everywhere except reachability, where the default build was 9% faster. SWI-PeTTa was not run at these sizes. These runs also counted user-mode instructions, which adds about 7 ms to each wall time.

	CeTTa				MORK leapfrog			
application	wall	CPU	MiB	G instr.	execute	wall	MiB	G instr.
Two-hop, triangles; 200,000 draws	1.05	0.95	145	9.3	4.92	6.62	380	47.2
Two-hop, triangles; 1,000,000 draws	9.04	8.21	590	46.9	24.46	30.91	1743	208.9
Two-hop, triangles; 5,000,000 draws	97.39	93.22	2898	244.3	125.8	158.4	8625	1110.0
Odd–even sort; 2,000 items	7.12	6.84	22.8	71.8	14.36	14.97	99.1	131.5
Odd–even sort; 5,000 items	68.56	67.15	2020	522.2	127.4	130.6	582	823.7
Five-cliques; 8,100 edges	0.169	0.13	13.4	2.1	0.237	0.265	7.0	2.9
Backward proof search; 3,000 targets	13.93	13.02	18.5	199.6	74.59	74.65	23.3	1110.4
Ancestry; 3,000 people	24.39	22.28	1466	142.7	24.87	42.48	412	500.8
Reachability with paths; 1,500 states	0.097	0.07	16.1	0.9	188.3	188.4	283	1430.1

Every run agreed with MORK on every relation's size, and the graph and five-clique rungs also with the independent computations of Section A.2. CeTTa's whole process is faster than MORK's execution alone on all nine rungs: by 1.3 to 5.4 times on the graph, sort, five-clique and proof-search rungs, level on ancestry (24.4 against 24.9 s, while MORK's whole job takes 42.5 s), and by three orders of magnitude on reachability. The instruction counts, which do not depend on the load, give factors of 1.4 to 5.6 on the same rungs and 3.5 on ancestry. Memory goes both ways: CeTTa needs a third of MORK's at five million draws (2.8 against 8.4 GiB) and about 3.5 times MORK's on the 5,000-item sort and on ancestry. This batch ran at a one-minute load of about 20, which slows memory-bound runs most: at five million draws CeTTa executes 4.5 times fewer instructions than MORK but is only 1.3 times faster in wall time.

The instruction counts also show where the preview has fallen behind the opt line's binary of 29 October, whose ladder was counted the same way: backward proof search over 3,000 targets now executes 200 billion instructions against 0.8 billion then, ancestry over 3,000 people 143 against 88 billion, the 300-vertex five-cliques 2.1 against 1.0 billion, and the 5,000-item sort 522 against 450 billion, with a peak of 2.0 GiB where that binary needed 76 MiB. Run back to back at today's load, the opt binary of 30 October takes 0.09 s for the 3,000-target proof search, against 14.5 s for the preview. The other rungs are level or better on the preview (the graph rungs execute up to 15% fewer instructions), and the two base cases run back to back, proof search and ancestry, are level. These are regressions of the preview to be repaired, not properties of the programs.

A.4 MORK's showcase workloads with its leapfrog join

A second suite was written to meet MORK on ground it was built for: graph analytics, knowledge-graph queries and combinatorial search, at sizes from thousands to millions of facts, each with an MM2 program for MORK, a PeTTa program for CeTTa and an independent Python oracle in its generator. LDBC Graphalytics kernels run on Graph500 Kronecker graphs (breadth-first depths, weakly connected components, deterministic label propagation, local clustering coefficients and single-source shortest paths at scales 12 and 14; the first three at scale 16); LUBM queries run on 1, 5 and 10 generated universities; four LDBC SNB business-intelligence queries run on LDBC's own SF0.1 and SF0.3 data, returning their full results without the queries' final top-k limit. The rest are Hilbert-style proof search by backward chaining done as forward chaining (MORK's own `bfc` benchmark), simulation signatures of a 4,096-bit adder for equivalence checking, condensed-detachment saturation in three axiom systems, CTL model checking of the dining philosophers, approximate matching of sequencing k-mers by masked views, five-cliques on 800 vertices with MORK's best conjunction order, Apriori frequent itemsets, and analytics over a million-fact knowledge graph shaped like MORK's FlyBase workload. The reference is MORK's leapfrog build. A run passes only if every relation's size equals the oracle's and, for the 34 tasks whose oracle lists rows, the complete row set equals the oracle's. The generators are in `benchmarks/mork_leapfrog/`; the SNB inputs need the official LDBC datasets.

	MORK leapfrog			CeTTa			
task	wall	CPU	MiB	wall	CPU	MiB	checked
g500s12_bfs	0.127	0.11	15.3	0.107	0.09	23.1	rows
g500s12_wcc	0.149	0.13	19.8	0.124	0.10	24.1	rows
g500s12_cdlp	2.10	2.08	36.6	0.719	0.67	19.9	rows
g500s12_lcc	9.52	9.42	26.8	7.06	6.94	19.5	rows
g500s12_sssp	1.97	1.94	35.1	0.153	0.14	33.8	rows
g500s14_bfs	0.513	0.47	45.1	0.491	0.47	41.3	rows
g500s14_wcc	0.655	0.63	63.9	0.360	0.33	36.9	rows
g500s14_cdlp	11.18	11.06	138	3.46	3.39	38.6	rows
g500s14_lcc	97.00	79.89	97.3	56.13	55.51	35.9	rows
g500s14_sssp	11.11	10.92	126	0.878	0.85	99.9	rows
g500s16_bfs	2.90	2.78	170	2.75	2.71	92.4	rows
g500s16_wcc	3.61	3.54	252	2.26	2.23	66.8	rows
g500s16_cdlp	62.61	62.23	642	18.91	17.71	75.6	rows
lubm_1	0.471	0.45	28.0	0.348	0.32	44.1	rows
lubm_5	2.72	2.66	139	2.39	2.25	180	rows
lubm_10	5.86	5.47	276	5.69	5.34	365	rows
snb0.1_bi9	1.54	1.52	105	1.25	1.22	148	rows
snb0.1_bi11	0.168	0.14	14.8	0.055	0.03	18.0	rows
snb0.1_bi12	2.10	2.06	125	1.99	1.95	193	rows
snb0.1_bi18	0.668	0.65	26.4	0.180	0.16	44.0	rows
snb0.3_bi9	4.78	4.72	312	4.29	4.13	415	rows
snb0.3_bi11	0.503	0.48	31.8	0.160	0.13	30.1	rows
snb0.3_bi12	6.72	6.51	382	6.84	6.66	585	rows
snb0.3_bi18	3.51	3.45	88.0	0.873	0.79	178	rows
bfc_19	7.50	7.00	396	7.67	7.56	538	sizes
circuit_add4096	15.11	15.03	710	9.97	9.85	472	rows
cd_KS_r5	9.35	9.17	172	8.33	8.27	168	rows
cd_XGK_r5	0.196	0.17	10.6	0.181	0.16	18.1	rows
cd_lukC_r6	0.495	0.47	15.1	0.316	0.30	26.3	rows
ctl_dp8	2.63	2.61	64.7	1.28	1.25	39.0	rows
ctl_dp9	11.36	11.34	238	7.28	7.24	67.2	rows
hamming_r100000_k12	4.70	4.56	227	3.76	3.42	295	rows

task (continued)	wall	CPU	MiB	wall	CPU	MiB	checked
hamming_r300000_k12	15.02	13.95	557	11.07	9.95	668	rows
clique_n800_m57600_k5	17.72	17.63	116	12.17	12.13	44.2	sizes
apriori_T50000	17.79	17.74	130	2.81	2.79	42.9	rows
kg_1000000	100.3	100.2	2896	86.48	86.02	2243	rows

All 216 runs passed their checks. CeTTa finishes first on 34 of the 36 tasks and is within 2% of MORK on the other two (bfc_19 and BI 12 on SF0.3). Its wall time is a geometric mean of 0.55 of MORK's, from 0.08 on single-source shortest paths to 1.02, and its CPU time a geometric mean of 0.53. Memory goes both ways: CeTTa needs more on 17 tasks, at most about twice MORK's (BI 18 on SF0.3), and less on the others, down to an eighth (label propagation at scale 16). This batch ran while other jobs kept the one-minute load between about 11 and 25, so its absolute times are up to about twice those of the quieter runs above; both engines ran in alternation under the same conditions.

A.5 The HE lane against Hyperon Experimental

The same source file and the same independent expected answer bag for both engines; CeTTa runs `--lang he`. Twenty-three cases were attempted with a 600-second limit per run. Seventeen completed on both engines with the expected answers and are timed below. Hyperon Experimental aborted on three of the larger joins (triangle_dense_32_count, guarded_triangle_32_count, path_product_12_cardinality_count) at an assertion of its expression index (size < TK_MAX_EXPRESSION_SIZE); CeTTa answered all three correctly. Three path cases (path_product_12_count, path_product_12_separator_count, path_isolated_8_512_count) use CeTTa's extended fold, which Hyperon Experimental, following MeTTa's rule for unknown symbols, leaves uninterpreted; they are not portable programs and are not compared. No run reached the limit.

	Hyperon Experimental		CeTTa HE lane	
case	wall (s)	MiB	wall (s)	MiB
ancestor_chain_256_natural	6.92	93.6	0.007	14.9
ancestor_chain_256_levelwise	56.93	1947	0.023	19.7
closure_ring_32_levelwise	16.02	47.8	0.066	22.7
counter_64_natural	10.09	960	0.044	31.0
sort_64_natural	1.28	29.0	0.007	13.3
arithmetic_10000	5.59	46.9	0.099	28.3
tally_1000x8	4.76	14.8	0.036	14.6
ancestor_chain_64_natural	0.283	19.4	0.010	14.7
counter_4_natural	0.121	19.6	0.019	15.5
sort_16_natural	0.159	13.4	0.011	13.1
triangle_duplicates_witnesses	0.030	12.3	0.011	12.2
triangle_empty_witnesses	0.021	12.2	0.013	12.0
triangle_dense_8_count	0.729	14.8	0.014	12.5
guarded_triangle_8_count	0.053	12.9	0.009	12.6
path_correlated_6_witnesses	0.055	13.4	0.012	12.9
path_product_8_cardinality_count	0.403	16.9	0.008	12.6
path_product_4_witnesses	0.026	12.3	0.011	12.3

CeTTa's HE lane is faster on all seventeen: from 1.6 times on the triangle join with no answers, where both processes are dominated by start-up, to about 2,500 times on the level-wise ancestor

chain (56.9 s against 0.023 s), with a geometric mean of 36. Hyperon Experimental's peak memory reaches 1.9 GiB on that case and 960 MiB on the counter, against at most 31 MiB for CeTTa. The four cases that the earlier five-second screen reported as Hyperon Experimental timeouts all complete, in 6.9 to 57 s.

A.6 The PeTTa examples against SWI-PeTTa

The population is all 164 tracked `examples/*.metta` files at the top of the PeTTa checkout. Each was first run once on both engines through the corpus harness, with its fixture policy (scripted input, isolated workspaces, deterministic stand-ins for external services) and output normalization, under a 600-second limit; no run reached it. Examples without a fixture were then timed. Of the 164, 161 print the same normalized output on both engines; the six with fixtures are listed without times. Most examples are dominated by start-up (the median example takes 54 ms on SWI-PeTTa and 4 ms on CeTTa), so the overall ratio says little: over the 155 timed examples whose outputs agree, CeTTa is faster on 154 (median factor 14) and slower on one, `patrick_iterate_quad.metta` (1.31 against 0.65 s). Of the nine examples that take SWI-PeTTa more than a second, CeTTa is faster by 1.3 (`he_minimalmetta.metta`) to 103 times (`peano.metta`). The five examples that the earlier two-second screen reported as incomplete (`he_minimalmetta.metta`, `holbenchmark.metta` and the three `matespace` programs) all finish on SWI-PeTTa, in 2.0 to 5.8 s.

Three examples differ, and no difference is a failure to finish. In `python.metta` and `python_import.metta`, CeTTa returns Python text as a MeTTa string, by a recorded decision of 27 October that follows Hyperon Experimental, while this SWI-PeTTa checkout returns it as a symbol, the default of its Python bridge: `(.upper "abc")` prints `ABC` on SWI-PeTTa and `"ABC"` on CeTTa, and a `repr` of the returned text shows the quotes. The examples' expected values were written for symbols, so CeTTa fails their assertions. `test_unify_eval_branches.metta` imports `lib/lib_he` by a path relative to the PeTTa checkout: SWI-PeTTa finds it from its working directory, while CeTTa resolves an import against the importing file's directory and stops with an existence error.

	wall (ms)		MiB		
example	SWI	CeTTa	SWI	CeTTa	result
`add_atom_fun_space.metta`	72.5	4.6	18.9	12.3	same
`and_or.metta`	72.8	3.7	18.9	12.2	same
`atomops.metta`	76.3	4.6	19.0	11.9	same
`booleansolver.metta`	73.8	3.6	18.9	12.0	same
`builin_types.metta`	77.6	5.7	19.6	12.3	same
`callquoteevalreduce.metta`	75.6	4.4	19.7	13.0	same
`callquoteevalreduce2.metta`	74.0	4.4	18.9	12.6	same
`case.metta`	72.2	3.7	18.9	12.6	same
`case2.metta`	72.5	4.8	18.9	12.7	same
`caseconstrain.metta`	73.3	3.4	18.9	11.5	same
`caseempty.metta`	74.0	4.7	18.9	12.5	same
`casenew.metta`	73.8	4.5	18.9	12.5	same
`chain.metta`	73.8	4.2	18.8	12.3	same
`collapse.metta`	72.5	3.6	18.9	11.7	same
`comments.metta`	72.5	3.9	18.9	12.0	same
`constanthead.metta`	73.6	3.6	18.9	12.0	same
`curry.metta`	75.3	4.8	19.0	12.5	same
`cut.metta`	77.0	4.0	18.9	12.9	same
`empty.metta`	48.5	3.9	18.9	12.0	same
`eval.metta`	50.2	3.8	19.0	13.0	same
`factorial.metta`	52.4	3.9	18.9	12.2	same
`fib.metta`	665.3	81.7	19.7	12.3	same

example (continued)	SWI	CeTTa	SWI	CeTTa	result
fibadd.metta	515.3	82.4	19.6	12.4	same
fibsmart.metta	49.6	5.4	19.0	12.3	same
fibsmartimport.metta	51.3	4.6	19.0	12.3	same
foldall.metta	76.9	4.9	19.1	12.9	same
foldallmatch.metta	74.8	4.4	18.9	12.7	same
foldallspacecount.metta	74.3	4.0	18.9	13.0	same
forall.metta	78.0	5.4	19.7	12.5	same
functionhead.metta	75.0	4.1	18.9	12.5	same
functionhead2.metta	76.4	4.5	19.0	12.4	same
functionhead3.metta	75.0	4.5	18.9	12.8	same
functionremoval.metta	75.3	5.0	19.0	12.8	same
functionremovalspec.metta	76.9	4.6	19.0	12.7	same
functiontypes.metta	76.8	5.4	19.1	12.8	same
git_import.metta	79.2	23.7	19.0	20.3	same
git_import2.metta	fixture; not timed				same
greedy_chess.metta	fixture; not timed				same
he_assert.metta	78.8	5.3	19.6	13.3	same
he_atomspace.metta	81.0	4.8	19.6	13.6	same
he_equalreduct.metta	83.1	5.2	19.7	13.0	same
he_error.metta	81.1	26.3	19.7	20.6	same
he_evaluation.metta	80.7	5.8	19.6	13.5	same
he_math.metta	77.1	4.3	19.3	12.2	same
he_minimalmetta.metta	1979.5	1491.5	84.3	34.2	same
he_quoting.metta	51.2	4.9	19.6	13.3	same
he_types.metta	49.7	4.0	19.7	12.9	same
holbenchmark.metta	4096.4	711.9	850	627	same
holfunctions.metta	51.4	3.9	19.0	12.8	same
holfunctions_intrinsicop.metta	49.7	3.4	19.0	12.6	same
hyperpose_primes.metta	3467.2	624.2	20.3	65.6	same
identity.metta	49.6	3.4	18.9	11.9	same
if.metta	50.3	3.4	18.9	11.8	same
if2.metta	50.0	2.9	18.9	11.9	same
if3.metta	56.2	5.0	18.9	12.2	same
if4.metta	55.0	3.3	18.9	12.0	same
ifcasenondet.metta	55.5	6.2	18.9	12.6	same
ifsimple.metta	61.2	3.1	18.9	11.7	same
invertfunction.metta	54.6	17.5	19.0	19.9	same
invertpeanoplus.metta	55.0	4.2	19.0	12.5	same
is_alpha_member_test.metta	58.3	4.4	19.7	12.3	same
iter.metta	70.1	3.7	18.9	12.5	same
lambda.metta	70.0	5.6	19.0	13.4	same
lambda_fresh_names.metta	68.0	4.0	18.9	12.2	same
let_superpose_if_case.metta	50.4	4.0	18.8	12.4	same
letext.metta	48.3	2.6	18.9	12.1	same
letlet.metta	48.4	3.5	18.9	12.1	same
letstar.metta	47.8	2.6	18.8	11.5	same
lib_roman_pair_helpers.metta	52.2	4.4	19.7	13.4	same
library.metta	51.9	3.7	19.7	13.6	same
listhead.metta	48.4	4.0	18.9	12.6	same
llm_cities.metta	fixture; not timed				same
logicprog.metta	49.0	3.7	19.0	12.5	same
logicprogset.metta	48.7	2.8	18.9	12.4	same
matchnested.metta	48.9	3.8	18.9	12.0	same
matchnested2.metta	48.9	3.5	18.9	12.2	same
matchsingle.metta	49.0	3.8	19.0	12.6	same
matchtypes.metta	48.9	3.1	19.0	12.2	same
matespace.metta	4054.7	273.9	626	16.4	same

example (continued)	SWI	CeTTa	SWI	CeTTa	result
matespace2.metta	5819.5	688.0	564	16.3	same
matespacefast.metta	4304.3	1184.6	3341	250	same
math.metta	53.8	3.8	19.3	12.0	same
meta_types.metta	47.8	3.0	18.9	11.7	same
metta4_prog.metta	46.0	2.6	18.9	11.8	same
metta4_streams.metta	48.6	3.8	18.9	13.1	same
mettaset.metta	49.0	2.8	18.9	12.0	same
multicall.metta	50.4	3.4	18.9	11.8	same
multiset_operations.metta	48.6	2.7	18.9	11.6	same
mutex_and_transaction.metta	54.1	3.9	19.8	12.6	same
myinterpreter.metta	48.6	3.2	19.0	12.7	same
nars_direct.metta	63.6	6.0	19.9	15.2	same
nars_tuffy.metta	302.0	112.9	19.9	21.3	same
nestedcons.metta	47.5	3.2	18.9	12.4	same
newtons_method.metta	53.4	2.9	19.3	12.4	same
nilbc.metta	1023.0	27.8	21.5	14.0	same
once.metta	47.5	3.5	18.9	12.8	same
outputtype.metta	46.8	3.4	19.0	12.9	same
parametric_types.metta	47.3	3.3	18.9	12.4	same
parse.metta	49.0	3.2	19.0	11.6	same
partialdef.metta	47.5	3.0	19.1	12.5	same
patrick_iterate_fib.metta	70.8	5.2	19.2	13.3	same
patrick_iterate_quad.metta	647.7	1306.7	19.7	23.2	same
patrick_test.metta	52.6	4.2	19.1	13.1	same
peano.metta	1571.4	15.2	23.4	14.3	same
peanofast.metta	421.5	5.9	342	15.2	same
permutations.metta	639.2	515.0	144	12.2	same
pln_direct.metta	64.2	6.6	23.2	15.0	same
pln_roman.metta	110.3	30.1	23.3	20.9	same
pln_tuffy.metta	65.5	8.5	23.2	15.7	same
plntest.metta	50.1	3.6	19.7	13.0	same
plntestdirect.metta	50.2	4.2	19.6	13.0	same
prologimport.metta	55.2	24.0	19.8	21.9	same
python.metta	65.2	12.0	27.1	19.6	differs
python_import.metta	63.8	12.0	27.1	18.5	differs
recursive_types.metta	52.8	3.6	19.0	11.9	same
recursive_types2.metta	50.5	3.6	19.0	12.9	same
repl.metta	fixture; not timed				same
repr.metta	52.0	3.9	18.9	11.9	same
roman_test.metta	57.5	7.4	19.8	14.0	same
scale.metta	879.4	455.7	399	126	same
selfprog.metta	49.3	3.7	18.9	12.1	same
smartdispatch.metta	49.2	3.2	19.0	12.8	same
spacefunction.metta	48.9	3.9	19.1	12.5	same
spaces.metta	47.9	3.1	18.9	12.7	same
spaces2.metta	49.1	3.6	19.0	12.4	same
spaces3.metta	48.5	3.0	19.0	12.2	same
spaces_find.metta	50.5	4.3	19.1	13.1	same
spaces_removeallatoms.metta	60.0	3.7	19.1	13.0	same
spaces_succeedspredicate.metta	51.5	3.9	19.1	12.8	same
specialize.metta	52.1	4.1	19.7	13.6	same
specialize_recursive_wrap.metta	49.6	3.6	19.1	12.4	same
specializecyclic.metta	49.5	3.9	19.0	12.8	same
specializefunctiontypes.metta	48.1	3.7	18.9	12.8	same
state.metta	47.7	2.7	19.0	11.7	same
streamops.metta	48.5	3.6	19.0	11.8	same
string.metta	51.0	2.6	18.9	11.5	same

example (continued)	SWI	CeTTa	SWI	CeTTa	result
supercollapse.metta	51.3	4.1	19.0	12.5	same
superpose_nested.metta	50.8	3.5	19.1	12.5	same
superpose_primes.metta	55.7	4.8	19.7	13.4	same
swrite_variable_identity.metta	55.8	5.5	21.1	14.1	same
tabling_fib.metta	49.9	4.3	19.1	13.4	same
test_alpha_unique_atom.metta	51.5	3.4	19.7	12.2	same
test_datetime.metta	fixture; not timed				same
test_memo_aggregate.metta	53.8	3.7	19.2	12.4	same
test_memo_dependency_invalidation.metta	54.5	3.7	19.2	12.1	same
test_memo_multi_answer.metta	53.1	2.9	19.2	12.3	same
test_memo_per_arity.metta	54.1	4.0	19.2	12.4	same
test_memo_same_name_multi_arity.metta	54.1	3.0	19.2	12.2	same
test_memo_stats.metta	53.8	3.8	19.2	12.3	same
test_memo_variant_nonground.metta	53.4	3.0	19.2	12.2	same
test_string_comments.metta	49.3	3.6	19.0	11.9	same
test_unify_eval_branches.metta	50.8	2.4	19.7	10.7	differs
tests.metta	49.6	3.9	19.0	12.7	same
tilepuzzle.metta	1553.5	954.0	122	162	same
torch.metta	fixture; not timed				same
translatepredicate.metta	51.5	3.6	18.9	11.8	same
translatorrule.metta	50.1	3.6	19.1	12.0	same
translatorrule_fib.metta	49.5	3.2	18.9	12.0	same
translatorrule_for.metta	50.6	3.8	19.1	12.7	same
twostage.metta	50.8	3.5	18.9	12.1	same
types.metta	50.7	4.2	19.0	12.8	same
types_dependent.metta	49.6	17.6	19.1	20.3	same
types_nondet.metta	49.1	4.2	19.1	12.6	same
xor.metta	47.9	4.0	18.9	12.7	same

A.7 Inductive learning: nuPLN

nuPLN, Nil Geisweiller's proof-producing learner, induces a predicate from three labelled examples by backward chaining: it constructs the conjunction $F \wedge G$ together with the typing and evaluation proofs that explain every label. Its reference implementation is the same PeTTa file on SWI-PeTTa. The benchmark keeps the file's eight inference equations and its knowledge base unchanged, drops the random-sampling helper (which consults a Prolog file and is not used by the query) and the self-tests, and repeats the final learning query 1,000 times as top-level queries; the generator is in `benchmarks/nupln/`. Both engines print the same 1,000 answers, byte for byte.

1,000 learning queries	wall (s)	CPU (s)	MiB
SWI-PeTTa	0.60	0.59	52.9
CeTTa	0.11	0.11	19.9

CeTTa is about five times faster, with less than half the peak memory. This is one small learning problem repeated, a throughput measurement, not a thousand different problems; a single query is too short to time apart from start-up.

A.8 The ai-mathematician

The ai-mathematician, a theorem-proving pipeline over a Megalodon corpus, has a PeTTa port that runs on CeTTa; its original is in Python. The stage that runs without external provers is premise selection: for each of the 966 theorems of the 1,158-declaration corpus that have recorded

dependencies, rank the earlier facts and report the recall of the true dependencies among the first 8, 16, 32 and 64. The port's matched harness runs the frozen Python v4 code and the port on the same corpus; both report the same recall summary to the last printed digit. The preview binary does not run this stage: the port's search preparation, `(ai-math:search $decls)`, returns no answer on it, although loading the corpus works. The table therefore measures the NIK line's binary of 26 October on which the port was qualified (SHA-256 prefix `f11e8311279f`), not the preview.

966 goals	load (s)	prepare (s)	rank (s)	total (s)	CPU (s)	MiB
Python v4	2.56	0.005	16.28	18.74	18.54	247
PeTTa port on CeTTa	2.83	0.392	13.20	16.63	16.30	402

The port ranks about a fifth faster than the Python original (13.2 against 16.3 s; on the loaded machine its three runs ranged from 11.8 to 14.5 s), loads the corpus slightly more slowly and needs 1.6 times the peak memory. Python's total covers loading, preparation and ranking inside an interpreter that has already started; the port's total is its whole process.

A.9 What was not measured

- **The ai-mathematician beyond premise selection.** Its proof stages call Vampire and Lean. The port's Lean worker requires Linux user namespaces for its isolation, which the measuring session did not have, and a full discovery cohort takes about a day on two workers; the preview binary also fails the port's search stage (above).
- **Other inductive-learning programs.** The Popper, Hopper and IGGP programs in the tree replay the reference systems' published programs over every example rather than learn them, and the Metagol-style learners have no matching reference task here; Hopper's own learner is not runnable on this machine. nuPLN's distractor ladder was not timed: with one always-true distractor predicate, CeTTa's depth-first search needed about seven minutes in an untimed run under load, and the faster first-witness strategy reported in September was never committed.
- **Upstream Hyperon Experimental.** Only the fork build named above was measured.
- **Three rungs of the scale ladder**: Hexlife with 200 cells and 30 generations, the counter machine at register value 1,000 and the process calculus at 500 + 500, each of whose MORK runs takes minutes at this load; and SWI-PeTTa and MORK's default build at the ladder's sizes.
- **The largest graph rungs** of Section A.10 on the preview.
- **Portable wheels, physical AArch64 and recovery from allocation failure inside GMP** are not requalified by this appendix.

A.10 Earlier experiments

These are dated observations of earlier binaries and are not measurements of the November preview. On 29 October a NumPy PCG64 input series (seed 0; `benchmarks/opt_suite/generate_large_graph.py`) took the two-hop and triangle task to hundreds of millions of derived pairs, in single runs on a loaded machine against MORK's default build:

	Opt, 29 October		MORK default, 29 October	
derived two-hop pairs	wall (s)	peak GiB	wall (s)	peak GiB
149,892,361	47.87	3.16	176.85	12.60
599,911,956	323.81	12.54	stopped by a memory guard at 15.6 GiB	

The input series is reproduced by the generator: an independent hash-set computation on the regenerated 7,499,796-edge input gives 149,892,361 pairs and 8,897 triangles, the counts both engines recorded then. The values agree with the raw records of those runs. No completed-reference time is assigned to the stopped MORK run, and these runs establish neither a capacity limit nor the preview's footprint.

Reproduction. The generators and runners are in the CeTTa repository: `benchmarks/opt_suite/` (the application suite, its scale ladder, the large graph series and the independent oracles), `benchmarks/mork_leapfrog/`, `benchmarks/petta_examples/`, `benchmarks/nupln/`, and `benchmarks/mam_families/` for the HE cases. Large inputs are generated, not stored.

B How the Listings Were Run

Every listing and probe in Section 2, and every probe quoted in Sections 4, 7 and 8, was run on 27 October 2026 with the binaries below, each from the working state of its tree (which includes uncommitted changes) and each run in place, because a CeTTa binary finds its libraries relative to its own location. A binary is named by the first twelve hexadecimal digits of the SHA-256 digest of its file; the digests were taken before the runs and again after them, and did not change.

used for	binary (version, build date)	SHA-256, tree head
HE and PeTTa lanes; rho calculus	`cetta` of the pre-main worktree (1.5.0-dev, 26 Oct)	`1db746ecc1ec`, feba78e3
Prime lane	the Prime draft's `cetta-interaction-basis-m5-20261027` (1.5.0-dev, 27 Oct)	`7b25fbd48cc0`, a0f1077a
Prime lane, second binary	the Prime draft's `cetta-draft-merges-20261027` (1.5.0-dev, 27 Oct)	`ee205ed7c764`, a0f1077a
runtime counters	the pre-main worktree's `cetta-main-runtime-stats` (1.5.0-dev, 25 Oct)	`54278a7df178`
the published `main`	`cetta` of the `main` checkout (1.4.0-dev, 21 Sep)	`a45d87417931`, 12da03b4
SWI-PeTTa	the PeTTa reference checkout (1.0.6) on SWI-Prolog 10.1.9	b74af1b

The HE lane runs as `cetta --lang he --pretty-vars file`, the PeTTa lane as `cetta --lang petta --pretty-vars file`, the Prime lane as `cetta --lang prime --pretty-vars file`, and SWI-PeTTa through its `run.sh` with `--silent`. The `transform` listing was run on SWI-PeTTa with the same library file imported by a relative path, because `(library lib_transform)` names a library shipped with CeTTa; its native and authored firings on CeTTa differ only by the environment switch `CETTA_TRANSFORM_REFERENCE=1`. Timings quoted from these runs (startup, `fib`, the `variant` table) are single whole-process runs on a shared sixteen-core workstation, measured with `/usr/bin/time`, and are indicative only.

Probes beyond the listings. The text also quotes these runs:

- the one-step reflection layer `lts:he` and the rule table (`lts:he:step-rules`), on the HE lane with the extended profile;
- untabled and tabled `fib 25` and `fib 30` on the HE, PeTTa and Prime lanes, and on `main`;
- the Prime outcome, variable, held-data and `delay`/`resample` behaviour, and the draft deviations of Section 8.4;
- the closed $\lambda\Pi$ conformance files and the explicit need-control fixture of the Prime draft, compared with their expected outputs;
- the rho cost examples of Section 10.7, compared with their expected outputs.

C LeaTTa Cross-Check Appendix

LeaTTa is a Lean 4 MeTTa runtime and proof development that is useful as an independent semantic cross-check for the HE-facing core of CeTTa. The local checkout used for the measurements in this appendix was a clean LeaTTa 1.0.6 tree at `github.com/MesTTo/LeaTTa`, commit `51b4b52` (`2026-06-22`, "prepare 1.0.6 release"). Upstream release notes and the checked-claims book are published at `mestto.github.io/LeaTTa/`.

This appendix records a local scope-and-spot-check run on 2026-06-24; it is a coverage map, not a new benchmark ladder replacing the main paper. The CeTTa side used `--profile he-extended --lang he`. The LeaTTa side used `--oracle` for assertion-bearing files and `--file` for pure loaders. A bulk pass ran every `.metta` file under the CeTTa tree (1255 files) through LeaTTa.

Coverage, in three tiers. The raw aggregate—168 of 951 assertion-bearing files passing in full—is misleading on its own, because most of the CeTTa test corpus exercises extension builtins that LeaTTa, a deliberately minimal HE interpreter, does not implement and is not meant to. The honest decomposition is:

Tier	What LeaTTa does on it
HE core	**Strong.** Of the 21 `he_*` conformance files, 18 pass in full and 2 partially (one loader carries no assertions), for 203/206 assertions. Full passes span symbols, equal-chaining, backchaining, GADTs, dependent types, higher-order functions, grounded basics, type propagation, states, documentation, and PLN truth values. This is the fragment that overlaps the HE evaluation semantics.
CeTTa extensions	**Out of scope by design.** The bulk of the 783 partial failures call builtins LeaTTa does not provide: MORK-backed spaces (84 files), PathMap (44), `hyperpose` (27), typed `new-space hash`/`queue` (12), `import!` (10), `space-set-match-backend`, and the textual Metamath parser. Failing these reflects LeaTTa's minimalism, not a fidelity gap.
Genuine gaps	**In-scope divergences.** A float-only numeric model: `(min-atom (5 4 6))` returns `4.000000` rather than an integer, and rationals such as `1/2` raise "only numbers allowed," so the exact integer/rational tower does not correspond. Typed error contours (e.g. `(+ 5 "S")` → `BadArgType`, empty/wrong-arity `car-atom`/`decons-atom`/`min-atom`) are not reproduced; we treat these as accepted divergences. Two `he_b4` assertions touching inert-symbol and empty-match semantics, and the higher-order WM-PLN bridge (218/221 on this checkout), remain to be diagnosed.

Spec repair triggered by the cross-check. The LeaTTa comparison was useful not only as an oracle but as a debugger for our own HE Lean formalization. During this pass we found that the public equation-query path had been routed through the simplified `simpleMatch` helper even though the faithful two-sided `matchAtoms`/`mergeBindings` semantics already existed. The repair is now written down in three Lean files: `DeclMatchSpec.lean` gives a declarative specification of `match_atoms` and proves `matchAtoms_sound`/`_complete`; `DeclMergeSpec.lean` does the same for `merge_bindings`, proving `mergeBindings_sound`/`_complete`; and `QueryInterfaceAgreement.lean` proves the staged ground-fragment adequacy theorems `queryEquations_ground_two_way_adequate` and `queryEquationsAgainstVisible_ground_two_way_adequate`. So the cross-check is now anchored on the faithful ground-query API rather than on the older simplified one. The remaining variable-bearing query case is deliberately left explicit: it requires the next repair, namely threading equalities through `Bindings.resolve`/`Bindings.apply`, and is the right next theorem rather than a hidden assumption.

Performance. On matched runs LeaTTa is correct where it is in scope but slower and markedly more memory-hungry than CeTTa's C runtime—operational overhead expected of an executable Lean interpreter, not a semantic limitation.

Benchmark	CeTTa	LeaTTa	Note
tail_recursion_deep_regression	0.30 s	0.80 s	both pass 3/3 assertions
tail_recursion_deep	0.30 s	0.80 s	both pass 3/3 assertions
he_c3_pln_stv	0.10 s	0.10 s	both pass 5/5 assertions
he_d2_higherfunc	0.10 s	0.10 s	both pass 25/25 assertions
test_wmpln_ho_similarity_bridge	0.10 s	0.10 s	CeTTa passes; LeaTTa passes 218/221 assertions
bio_refseq_723k	0.40 s	3.30 s	both run; LeaTTa ≈1.16 GB RSS vs. CeTTa ≈196 MB
bio_tfbs_415k	0.30 s	2.10 s	both run; LeaTTa ≈917 MB RSS vs. CeTTa ≈136 MB

Tilepuzzle: semantics vs. performance. The raw CeTTa tilepuzzle benchmark depends on CeTTa's typed `queue` and `hash` spaces, so it does not run unchanged on LeaTTa (a Tier-2 builtin mismatch). A bounded adaptation replacing those spaces with a functional queue and plain named-space membership does run: a 100-step witness reaches 171 visited states in 18.47 s, while the 500-step version did not finish under a 30 s cap. For reference, the clarified native contract counts 181,440 reachable states including the start board, while the older PeTTa example's 181,441 self-check is a counter-convention difference, not evidence for an extra reachable board. A fresh full native CeTTa run now completes with the tail-safe evaluation-arena garbage collector: 181,440 states in 8.80 s at 233 MiB (Section 12.3). So this bounded LeaTTa adaptation is only evidence that the algorithmic idea transfers, while the full native CeTTa row is settled capacity evidence.

Bottom line. LeaTTa is a credible independent oracle and a candidate verified engine-floor for the HE evaluation core of CeTTa: it agrees on the conformance set that matters, diverges only on extension builtins it deliberately omits and on a float-only numeric model, and pays an expected interpreter-speed cost. Linking LeaTTa to the HE Lean specification is therefore well-founded on the core, provided the numeric tower and error-contour divergences are encoded as known boundaries. The specification side is now in a better state than when this appendix was first

drafted: the ground-query fragment is connected to the faithful HE matcher/merge semantics by proved two-way adequacy, and the remaining variable-bearing query case has been isolated cleanly enough to be attacked next rather than smudged into the oracle story.

D Experimental — MAM: The MeTTa Abstract Machine

This appendix is explicitly experimental. It captures design exploration from ongoing discussions (April 2026) between the authors and external reviewers, concerning the post-phase-1f abstract-machine layer for CeTTa*. The original proposal is retained because it records the design pressure, but the blanket statement that nothing is implemented is no longer true: several machine components landed during August 2026. They do not yet constitute a final universal MAM, and proposed names or categorical readings below remain revisable.*

D.1 Motivation

The phase-1f-era extraction seam (`src/search_machine.{h,c}`) pulls policy parsing, stream combinators, and a `TableStore` plugin out of `eval.c`, and wraps them in a thin callback struct `CettaSearchMachine = {stream_source, eval_bound_single, eval_ctx, table_plugin}`, stack-allocated at each call site. This is a useful refactor, but it is a *plug interface* — not an abstract machine in the WAM / ZAM / SLG-WAM sense. Real abstract machines (XSB's SLG-WAM, SWI's tabling runtime, Soufflé's compiled Datalog, miniKanren's CPS-transformed search) have persistent machine state, explicit visitor protocols, capability advertisement, and episode lifetimes.

The proposal here is a *principled* next layer that retains the current code as `SearchAdapter` (transitional helper) while naming the real abstraction from first principles.

D.2 Current categorical reading

The MAM's present semantic reading, under active revision, models it as a **quantale-enriched traced profunctor**, fibrated for reflection. This reading is provisional — it captures the current best understanding of the structure, not a definitional claim that binds future architecture. Unpacked:

- **Profunctor** $P : \mathcal{Q}^{\mathrm{op}} \times \mathcal{A} \to \mathcal{V}$: *search.* Queries in $\mathcal{Q}$ relate to answer streams in $\mathcal{A}$, with intensity weighted by elements of $\mathcal{V}$.
- **Quantale enrichment** $\mathcal{V} = (L, \otimes, \leq, \bigvee)$: *multiplicity.* Answer weights live in a quantale: $\mathcal{V} = \mathbf{2}$ for classical boolean search; $\mathcal{V} = \mathbb{N}$ for counted answers; $\mathcal{V} = [0,1]^2$ (strength $\times$ confidence) for PLN. PathMap's trie-algebra is natively a quantale.
- **Trace** $\mathrm{Tr}_X^{A,B} : \mathrm{Hom}(A \otimes X, B \otimes X) \to \mathrm{Hom}(A, B)$: *fixpoint.* Feedback / saturation / tabling / cyclic queries are all instances of categorical trace (Joyal–Street–Verity, Hasegawa).
- **Kleisli structure over an effect monad** (or algebraic effect handlers *à la* Plotkin–Pretnar): *control.* Demand, cut, suspension, resumption, and schedule are composable effect operations.
- **Fibration** $p : \mathcal{T} \to \mathcal{B}$ (optional, for reflective MeTTa): rules-over-machine-state.

A trace operator is provably equivalent to taking the least fixpoint of a parameterised endofunction (Hasegawa 1997); this is why tabling, PLN-convergence, and saturation all fit one uniform operator.

D.3 The proposed machine family: MAM

We name the machine family **MAM: MeTTa Abstract Machine**. The name is chosen by identity rather than by current structure or scope: MAM is the abstract machine for the MeTTa language family, and that identification remains accurate as the internal architecture evolves. The naming tradition (WAM, ZAM, STG, CEK, CAM) uses either author identity or component decomposition; MAM uses the language identity it serves, which is stable across architectural iteration. Specifically, MAM does *not* hardcode a categorical claim (as "QAM = Quantale Abstract Machine" would have done), and does *not* hardcode a functional scope (as "SCAM = Search & Control Abstract Machine" would have done). The categorical reading and the current feature set are both free to evolve without renaming.

Concept	**Name**
Machine family	`MAM` (MeTTa Abstract Machine)
Formal description	Quantale-enriched traced profunctor
Engine 1 (goal-directed, Kleisli/effect)	`Chainer`
first realization (native)	`native_chainer`
Engine 2 (fixpoint, trace)	`Saturator`
first realization (pathmap)	`pathmap_saturator`
Per-session state	`Episode`
Dispatch vtable	`MAMOps`
Answer	`Answer` / `WeightedAnswer`
Schedule axis	`dfs` / `bfs` / `iddfs` / `interleave` / `native`
Demand axis	`unbounded` / `once` / `limit-k` / `fold`
Visitor control	`CONTINUE` / `STOP` / `ERROR` / `SUSPEND`

D.4 The two-engine pressure strategy

Rather than over-specifying a first concrete machine, MAM is pressured from day 1 by *two* independent realizations that stress complementary axes of the seam:

`Chainer` (goal-directed). WAM-lineage control: explicit environments/frames, choicepoints, trail/undo, deterministic tail-path handling, bounded-demand response, direct conjunction when advertised. Design-guide workload: **metamath backward chaining**. Before the evaluation-arena garbage collector this workload could exhaust memory on CeTTa; with the collector CeTTa now completes the Metamath proof-search rows (Section 11), so the remaining motivation here is WAM-lineage control, not a memory-exhaustion gap. First realization: `native_chainer`.

`Saturator` (fixpoint). SLG / Datalog / saturation lineage: snapshots, delta-sets, conjunction planning, exact membership, counted answers, eventual path toward PLN and ATP-style saturation. Design-guide workload: **PLN multi-pattern deduction**. First realization: `pathmap_saturator` (because PathMap's quantale structure makes weighted answer aggregation natural).

This pair creates the right kind of pressure on the seam: Chainer forces frames, choicepoints, `EnvRef`, demand handling, and explicit scheduling; Saturator forces snapshots, delta sets, conjunction planning, exact membership, counted answers, and the eventual path to PLN and ATP.

Compiled / bytecode (ZAM-lineage) and ATP saturation engines are explicit *later* lanes — they risk freezing the protocol in the wrong shape if pursued before Chainer and Saturator coexist cleanly under one MAM.

D.5 Axes that must stay separate

A core architectural commitment is that the following axes are **distinct data** in `SearchRequest`, not tangled behind source-level operators:

Schedule. `dfs` / `bfs` / `iddfs` /`interleave` are *fairness policies*, not engines. Korf's classic result motivates `iddfs` as a completeness-friendly default over raw `bfs` (BFS completeness with DFS memory); miniKanren's interleaving exists precisely to avoid starvation on infinite branches.

Demand. `once` / `limit-k` / `unbounded` /`fold` describes how many answers are requested, independently of schedule. `once` and `select 1` must build `demand = FIRST`; they must *not* silently hardcode DFS.

Table mode. `none` / `variant` / `subsumptive` /`incremental` mirrors the XSB / SWI ladder; each is an explicit engine-level choice.

Backend. `native` / PathMap / MORK chosen by the actual `Space`, not by a global preference.

Lane. `recursive-dependent-proof` / `atp-saturation` /`solver-oracle` / ... chosen in source syntax.

The request builder (one per `--lang`) normalizes concrete syntax into `SearchRequest + SearchLangContract`; the MAM consumes the normalized object, never raw source atoms.

D.6 The seven-type vocabulary

The original proposal used the following seven-type vocabulary. The implementation did not land this table literally: narrower, falsifiable objects such as `PettaSearchMachine`, `MatchDecision`, `PreparedPureProgram`, and `RuleMachineCoreV1` emerged first. The table remains a useful comparison vocabulary, not a frozen C ABI.

Type	Categorical role
`SearchRequest`	Object of $\mathcal{Q}$
`SearchAnswer`	Object of $\mathcal{A}$ (quantale-weighted)
`SearchLangContract`	Subcategory inclusion $\mathcal{C} \hookrightarrow$ MAM
`SearchCaps`	Capabilities = presence of morphisms/natural transformations
`VisitCtl`	Algebra of the effect operation visit
`MAMOps`	Kleisli dispatch vtable
`Episode`	One trace-iteration instance

General implementation-level `SUSPEND/RESUME`, cancellation tokens, backend-independent snapshot handles, and a single per-`--lang` MAM registration hooks remain deferred; individual machines already carry narrower instances of several of these concerns.

D.7 Implemented machine substrate (August 2026)

The landed components answer several of the proposal's original questions without pretending that one universal machine has been completed:

Choice and continuation ownership. `PettaSearchMachine` owns an explicit heap of clause activations, environments, continuations, choice frontiers, binding rollback, and answer projection. Its host callback API supplies language services but does not own the PeTTa choice point. `SearchContext` retains the small shared bindings/scratch save-and-rollback protocol.

Revision-pinned candidate programs. `MatchDecision` compiles sparse weak-head clause discrimination and callable caches under language, profile, language definition, compiler, and space identity. It may omit only structurally impossible clauses and returns a candidate superset; selected open equations are standardized apart under a fresh epoch and rechecked by the authoritative matcher. Indexing therefore cannot become binding authority.

Prepared and compiled fragments. `PreparedPureProgram` admits a deterministic, effect-free closed equation fragment with explicit heap control and conservative fallback. `RuleMachineCoreV1` executes bounded admitted rule packages and inspectable bytecode blocks under revision-pinned publication. These are compiled fragments with mutation/staleness gates, not a claim that arbitrary MeTTa has been compiled.

Observer-safe substitution fusion. Ordinary clause activation previously traversed a result to apply its local substitution and then traversed it again for the enclosing environment. The current matcher can compose these substitutions and traverse once, but only when the host reports that the equation's result payload is not observed (today `equation_result_payload_observed`). If an observer may inspect the intermediate payload, the two-stage path remains authoritative. Observer visibility, rather than benchmark identity, is the optimization license.

Analysis-provider isolation. The PeTTa `typecheck-v2` profile enters through `PettaAnalysisService`, whose identity, ABI, policy, mutable authority, judgments, reasons, and ready-call validation are explicit. The search machine no longer names the Roman-derived checker; a genuinely generic type and elaboration machine remains open.

The result is a family of interoperating machine components over shared term, space, and observation contracts. Compact activation environments, mode/relation-indexed clause dispatch, generic tabling, and a unified episode protocol remain structural work; the implementation evidence no longer supports describing the whole layer as a mere callback proposal.

D.8 Reference base

The proposal draws on the following literature, all available in the local library:

- *Aït-Kaci 1991, WAM Tutorial Reconstruction* – foundational WAM semantics; drives `Chainer` design.
- *Swift & Warren, XSB System Manual; Sagonas et al. 1994* – SLG-WAM tabling; drives `Saturator` / table-mode ladder.
- *Scholz, Jordan, Subotić, Westmann 2016, Soufflé: On Fast Large-Scale Program Analysis in Datalog (CC)* – staged compilation of fixpoint computation; compiled `Saturator` path.

- *Keoliya & Byrd 2020, microKanren* – interleaving search as a distinct schedule axis.
- *Ager, Biernacki, Danvy, Midtgaard 2003, A Functional Correspondence Between Evaluators and Abstract Machines (BRICS)* – derivation methodology for machines from evaluator semantics.
- *Leroy 1990, The ZINC Experiment* – machine-description vs. run-instance separation (maps to `MAMOps` / `MAM` / `Episode`).
- *Russell & Norvig, AIMA Ch. 3* – IDDFS completeness; uniform treatment of uninformed search.

D.9 Status

Since this appendix was first drafted, the derivation methodology it cites has acquired kernel-checked foundations in the Mettapedia Lean tree (`Mettapedia/Machines/`): a functional-correspondence module deriving a CEK-shaped eager machine and a Krivine-shaped demand-driven machine from fueled evaluators over one shared term substrate, with the evaluator-to-machine correspondence and machine determinism proved; a cone-duality module (forward/production and backward/demand reachability cones, their Galois adjunctions, wavefront exhaustion, and the demand-diamond restriction theorem); and a substrate module packaging the two machines as control cores over one term type with a kernel-checked strategy-separation witness (a single term on which the eager core returns nothing at every fuel while the demand core answers). Axioms for every keystone are within {propext, Classical.choice, Quot.sound}. These results turn "the machine is derived from the semantics" from a methodological citation into project theorems: the eager and lazy language lines require distinct control cores over a shared substrate, and demand-restricted evaluation is sound and complete for goal-reaching by theorem rather than folklore.

The August runtime now supplies concrete local-search, candidate-program, prepared-pure, bounded-rule-machine, and analysis-provider seams, and the counted PathMap backend supplies a working indexed-space pressure case. The two-engine strategy therefore survives as an architectural direction, while the proposed seven-record API and the claim that implementation had not begun are retired. What remains genuinely experimental is the universal composition: tabled suspension/resumption, direct PathMap product-cursor planning, versioned operator/delta/receipt exchange, and one open refinement interface spanning HE, Prime, PeTTa, Zero, and future engines.

This text remains a living design record. Implemented behavior is named explicitly; provisional categorical readings and future interfaces do not bind Prime or CeTTa *development.*

E The Prime Living Specification, Distilled

This appendix distills the Prime living specification and decision ledger as of 23 July 2026, and is kept as that record; the decisions recorded since, and the draft's current behaviour, are in Section 8. The living document separates selected design, implemented behavior, verified evidence, proposals, and open questions; every consequential clause carries a counts-primal evidence annotation (independent supporting items n^+ and unresolved counterexamples n^- are primal; a strength/confidence pair is derived from them, and retired refutations are kept on the record rather than deleted).

E.1 Constitutional boundary

Prime is a distinct language: branch-local call-by-need with call-time choice is its ordinary application semantics, while the HE-family profiles keep their existing evaluation semantics; shared backend refactoring is permitted, silent semantic bleed is not. Execution produces answers, effects, coverage, and receipts — never theoremhood; claim-bearing results lower to an external generic checker for replay.

E.2 Design principles (P0–P11)

P0 (meta). Prime aims to be as good as theoretically possible as a cognitive language: specification details must follow from best-possible elegance, ergonomics, performance, and coherence — and from the principles below — rather than from incident, imitation, or implementation convenience. A clause that derives from no principle is either evidence of a missing principle or a candidate for removal.

P1. Checking-first authority: engines, machines, indexes, and learned guidance are producers or schedules; a generic checker replays claim-bearing derivations. A hit is not a proof.

P2. Canonical meaning is distinct from execution schedule; an optimized schedule must refine the declared relation.

P3. Identity is semantic when sharing is observable: suspension, answer-occurrence, branch, effect, and capability identities are not interchangeable, and serialization may not erase aliasing or correlation.

P4. Choice preserves occurrences and correlation: results form an occurrence bag; one call-time suspension is shared by every occurrence denoting the same source argument; explicit resampling creates fresh choice identity.

P5. Effects are never hidden by backtracking vocabulary; sibling branches do not share mutable state by default; commit and merge are explicit policies with receipts.

P6. Completion is data: *incomplete* is not *false*; query polarity determines what one witness or one counterexample can settle.

P7. Scope and names are explicit layers: canonical binding is signature-driven de Bruijn; matcher metavariables are a separate layer; quoted structural names present identity without conferring resolution.

P8. Quotation does not grant authority: quoting makes syntax inert and nameable; privacy, liveness, reachability, and authorization are resolver and capability properties.

P9. Unknown syntax remains data: a form is evaluated only when an admitted operation, equation, or judgment determines how; otherwise it is residual, and an error requires positive evidence of a violated contract.

P10. Open extension is package-indexed, with formation rules, capabilities, and correspondence status; dynamic changes invalidate caches by digest.

P11. Theoretical analyzability is an implementation requirement: state-changing operations correspond to named transitions; tests carry positive, negative, and mutation witnesses per semantic boundary.

E.3 The Need core

A cell separates an immutable origin (term, lexical environment, package revision, provenance) from a checked mutable cache (`Empty` → `Evaluating(owner, generation)` → cached value or stable fault), with same-owner recursive demand reported as a blackhole (receipted, not cached, cell restored) and other-owner demand routed to an explicit wait/help protocol (selection open). Only faults stable under re-execution in the same world are cached; resource exhaustion and cancellation retain a resumable frontier. `delay`, `force`, and `resample` are the source constructs (lower-case only); `resample` deliberately breaks call-time correlation.

E.4 Ratified in the 2026-07 sprint

- Bare `$` as a fresh anonymous variable per occurrence (parser 25/25, reference model 29/29, grammar tournament 16/16, evaluator/quotation matrix 99/99 with sanitizers, a mechanized unique-selection theorem, and a corpus check showing no existing program depended on the old reading); the rejected root-binder alternative is retired on the record.
- Producer identity is the Need cell (store-lineage allocation identity), not application or source position — a native alias probe shows one shared cell observed from four application sites remaining correlated; the four static frontier policies fail a common fifteen-law audit at 9, 10, 11, and 12 laws respectively.
- A policy-neutral certified control layer: controllers reorder certified frontier nodes and can neither create nor destroy answers; pruning is explicit. Exhaustive policy invariance is proved; finite-budget fairness is a recorded obligation, not assumed.
- The source Prime LanguageDef as executable syntax authority through the generated reader (Section 16).
- An equality vocabulary pin: directional rewriting, computed equality, unreduced syntactic comparison, alpha-equivalence, and registered/conversion equality are five different relations, and the candidate frontier is scheduling, not equality.

E.5 Open

The public suspension-observation API (constraints and invariants selected; the concrete observed form is a narrowed choice); the worlds carrier representation (event structures lead); registered-equality concrete syntax; the other-owner forcing protocol; cyclic graph persistence; typed-laziness coherence as a mechanized refinement; the cell-causal native contender behind its confidence gate; and finite-budget fairness under adversarial ranking.

F Core Design, Implementation, and Specification Principles

Distilled from the project's design discussions (June–July 2026) and the implementation campaigns they governed. Principles are grouped; each group opens with its load-bearing head. They overlap deliberately with Appendix E: the Prime principles are the language-level refinement of these project-wide commitments.

F.1 Authority: who may mint truth

Checking-first. Producers — search procedures, elaborators, learned models, external provers, the runtime itself — are untrusted proposers; theoremhood is conferred only when a generic checker replays an explicit certificate against an admitted `LanguageDef`. Corollaries: authority flows through replayable proof objects, never execution traces of another engine; certificates carry their substitutions (the checker applies and compares, never unifies) and name their exact subject; a capability is enabled only by a machine-checked certificate of the property that makes it safe (checker conversion requires a confluence/canonical-section certificate; deterministic parser specialization requires a determinism proof); passing test suites are evidence, never theorems; and checking is search with the answers pinned — one kernel, two guidance policies, heuristics never deciding truth.

F.2 Inertness and honest verdicts

Unknown stays inert. The evaluator does not evaluate what it cannot interpret: unmatched forms are their own normal form, and no profile emits "feature unavailable." Externally this becomes a layered vocabulary: unknown syntax is residual; a recognized protocol with an unresolved endpoint is an explicit pending value; transport failure is a fault receipt; service rejection is a rejection value. Judgments return proven, rejected-with-reason, unknown, or resource-incomplete — unknown is never rejection, exhaustion is never quiescence; no verdict may flip as fuel increases; "must" judgments require nonemptiness plus completeness; and fuel is never a semantic premise — the semantics is pure, with resource outcomes layered outside it.

F.3 Specification as source

Syntax is a GSLT; one root per language. Concrete syntax is itself a graph-structured lambda theory — streams as terms, productions as rewrites — and the parser is that theory's evaluator (print-parse as a section–retraction law; unambiguity as a confluence theorem). Each hosted language has exactly one mathematical root; the engine supplies generic mechanism only. The genericity gate: hosting language $N{+}1$ must be a zero-line diff in generic code — only new data. Roots change by governed correction (impossibility witness plus sign-off), never by silent drift; hand-maintained artifacts styled as generated are forbidden. Generated artifacts pass falsifier ladders (acceptance, AST differentials against an independent reference, near-miss rejection, print-parse stabilization, per-production coverage), and failures repair the declarative specification, never the engine. The specification itself is self-hosted, queryable data — properties, decisions, and support edges with explicit status — in which refuted claims persist as refuted nodes so they cannot silently resurrect. Generic machinery is verified against stable semantic authorities first; a rapidly evolving language rides as a canary, not an anchor. Before accepting an obstruction, ask whether it survives in the canonical mathematical object or is an artifact of the encoding.

F.4 Minimality

Mechanism in the kernel, policy in the library. The core owns rewriting, the checking judgment, and one resumable fair scheduling primitive; strategies — including proof search as type inhabitation — are compiled library code, inspectable and overridable. Match, unify, communication, comprehension, and inhabitant selection are one primitive. Inference rules are semantic; direction, agenda, indexing, tabling, and saturation are schedules over the same rules. A candidate core construct must serve at least two of practical programming, native dependent typing, and

guest-logic adequacy, or it is an extension package; each core principle carries an independence witness (something breaks without it, nothing else forces it).

F.5 Pluralism

Profiles are conservative extensions; no sovereign object theory. A profile is legitimate while existing programs run unchanged; the first non-gateable conservativity violation is the dialect trigger, and minting a dialect is a normal event — the language is a family. The generic waist standardizes how rules are replayed, never what guest judgments mean; at least two object logics are maintained at all times so no package is both the definition and the validation target. "The checker accepted a derivation relative to package P" is categorically different from "P is sound"; bridges between logics are theorems or they are nothing.

F.6 Homoiconicity, reflection, binding

No second grammar, and quotation confers no authority. Types, judgments, packages, and claims are ordinary atoms the reasoner can inspect and rewrite; one sigil-to-expansion table drives the reader and both printers, so compact and expanded notation are one language. Binding is "both, by kind": matcher metavariables keep identifier machinery while object binders are canonically de Bruijn, meeting at elaboration, printing, and replay seams; stored checked terms are closed and alpha-equality is structural equality. Reflective observation is taxed at the dangerous act, not the epistemic one: looking is cheap and pure, forcing carries the ceremony, and receipts attach where claims are made.

F.7 Typing and graduality

Graduality is a same-program theorem. A validly typed working program with annotations erased computes the same results — always typed-versus-untyped of one program, never a cross-profile comparison. Semantic value types, syntactic shapes, and staging are three vocabularies; the gradual dynamic type has deliberately non-transitive consistency (no laundering a type through "?"); permissive success typing is never advertised as sound gradual typing without cast-and-blame enforcement; and there is no subtyping without coercion coherence — honest poverty over dishonest wealth.

F.8 Equality

Stratified, weakest-in-core. Identity and typed equality answer different questions (a cross-type comparison is ill-posed, not false); commitments form a lattice (same-atom, conversion, equal-at-type, identity, observational equivalence) with the core adopting the weakest adequate one. Program equations are directional rewrites; checked conversion is admitted only from certified fragments; view equations are pure projections; semantic equivalences are separately proven — a directional rule never silently becomes checker-authoritative symmetric conversion, and no registered equality may identify a suspension with its origin or two cells by origin equality, because observation is a projection and identifying across it destroys sharing.

F.9 Laziness, sharing, capability

Sharing is semantics. Branch-local call-by-need with call-time choice makes cell identity observable: a shared argument forces once per branch and every consumer sees the same value. A

suspension has three strictly ordered observables — cell identity, origin, eventual result — and the strictness *is* call-time choice. Origin is an allocation-time snapshot, so optimization cannot change what inspection sees; observing never forces and never leaks the cell capability; re-delaying an observed origin allocates fresh. "May force" and "may inspect" are separate authorities; names are addresses while capabilities are authority (knowing a descriptor authorizes nothing); foreign services are one typed request/reply protocol with receipts, one suspension being one request with a memoized response and resampling the explicit freshness boundary — which also gives sample-once stochastic choice for free.

F.10 Occurrences, cost, and enrichment

The non-conflation chain. Legal transition ≠ occurrence identity ≠ probability/rate ≠ scheduling priority ≠ resource cost ≠ utility ≠ provenance/authority; every enrichment is a readout of one relational process. Results are occurrence bags with completion status; answer identity is defined before any deduplication; rates attach to occurrences, never branch counts, so refactoring-invariance of the measure is a theorem obligation. Cost receipts are measured pomsets whose algebraic laws are derived, never posited — and when a desired law met a kernel-proved obstruction it was withdrawn by counterexample and repaired by indexing, not weakened by assumption. One law appears three times: erasure recovers the base (gradual typing's erasure theorem, profile noninterference, cost observer-transparency) — "the definition of 'this is still MeTTa'." Control fields route compute, never truth; every legal transition keeps a protected baseline rate (no starvation, anytime behavior); reversibility is an effect property, not a global mode; and a system that modifies its own laws accepts a delta only with a proof that the delta preserves what it claims.

F.11 Effects, errors, resources

Unmetered by default; errors are values. Budgets and ledgers are opt-in and zero-cost when absent; effectful power is capability-gated and never hidden inside conversion; errors are inspectable, matchable atoms in the result bag — branch-local, never process-killing, and richer than a boolean; default judgment forms carry no fuel argument, and exhaustion surfaces only when it changes the answer.

F.12 Evidence and confidence

Counts-primal, ladder-honest. Hard pruning requires proven legality or impossibility with recall preservation; everything uncertain is soft ranking, demonstrated by a counterexample of an uncertified mask eating a real solution. Evidence accumulates as counts with explicit independence accounting (correlated descendants never count twice); the confidence ladder distinguishes kernel-factive "program meets specification" from the graded "specification captures intent," whose independent review is the second source; specification authors are kept blind to candidate artifacts; and a reviewed specification plus a kernel-checked divergent witness is a factive, *generative* refutation — first mismatch, certified continuations, repair targets.

F.13 From semantics to machines; method and naming

Machines are derived, not designed. Abstract machines arise from evaluators by the functional correspondence, so the eager and lazy lines share a substrate while requiring distinct control cores — now a mechanized fact — and demand-driven execution restricted to the production-cone/demand-cone diamond is sound and complete for goal-reaching, making magic-set restric-

tion, premise pruning, and tabling instances of a theorem. Method: contested semantic laws are settled by feature-ablation tournaments (every candidate behind a flag, one corpus, discriminating programs choose); falsification ladders run theory-decidable gates before compute is spent ("no conservativity, no experiment"); independently written engines share defect classes, so the executable-independent declarative specification is the productive referee. Naming: mathematical vocabulary is used only after its preconditions are theorems ("earn the word"); the algebra is the specification, stated in interface headers and enforced as gates; one name, one meaning, no aliases after renames; anti-goals name integrity violations only, with scope stated positively; ergonomics is a first-class late phase, redone against collected real confusion once the semantics settles.

F.14 Supersession, on the record

The principles themselves are governed by B.3: superseded forms stay recorded rather than deleted. Notable retirements: notation as a side-record superseded by syntax-as-GSLT; early list-shaped cost receipts superseded by measured pomsets, whose desired law was withdrawn by kernel counterexample and repaired by parameterized-monad indexing; blanket trace-refusal refined to generator-based replay of standard certificate formats; "freeze" vocabulary replaced by governed change with impossibility witnesses; and a draft rule promoting every program equation to checker conversion refused in favor of the stratification of B.8.

G Concrete Syntax

The authoritative syntax of every profile is its `LanguageDef`: the generated readers of Section 16 *are* the grammar, and the rendering here is descriptive. This appendix gives the human-readable account a newcomer needs before consulting the definitions.

G.1 The shared s-expression core

```
program    ::= form*
form       ::= bang | atom            ; top level
bang       ::= "!" "(" atom* ")"      ; evaluated directive
atom       ::= symbol | variable | literal | expr
expr       ::= "(" atom* ")"
variable   ::= "$" name               ; named, co-referring
literal    ::= integer | float | "string"
comment    ::= ";" ... end-of-line
```

Distinguished expression heads shared by the family: `(= lhs rhs)` declares an equation (an unordered member of the enclosing space, never an ordered clause); `(: term type)` declares a type fact; `(match space pattern template)` queries; `(add-atom space a)` and `(remove-atom space a)` effect the store; `(pragma! key value)` configures. Results of a bang print as a bag — ordering is non-semantic throughout the family.

G.2 Profile deltas

HE lane (`--lang he`, profiles `he`, `he-compat`, `extended`, `he-prime`): eager evaluation; an argument whose declared type is `Expression` or `Atom` is passed unevaluated; grounded operators

receive unevaluated arguments unless their contract demands values; unknown symbols remain uninterpreted (never an error); `&self` denotes the ambient space and `(new-space kind)` introduces backend-selected spaces.

Prime (`--lang prime`): identical concrete syntax with these visible deltas — bare `$` is a *fresh anonymous* variable per occurrence; a parameter typed `(Data Atom)` holds its argument as written; standard definitions are editable, with originals under `std:`; and evaluation is branch-local call-by-need with call-time choice, so program meaning coincides with the eager reading exactly on the terminating, effect-in-demanded-position fragment (Appendix H).

PeTTa (`--lang petta`, the default; `petta@1.1` is typed): the same core grammar compiled to Prolog clauses; a call that no clause of a defined function accepts has no answer; shipped libraries are imported as `!(import! &self (library name))`; test harness forms (`!(test ...)`) and library namespaces (`lib:op`) are conventions of the distribution rather than language deltas.

G.3 Operational semantics, in brief

In the style of the MeTTa formal-specification lineage, evaluation is a relation from a space and a term to a *multiset* of results, written $\mathrm{eval}(s,\, e) \Downarrow B$; the full small-step rule table and its Lean certification are Section 4.8 material, and this digest fixes the reading.

Eager lane. The generating rules, informally:

$$\begin{aligned}
&(\textsc{Value}) && \mathrm{eval}(s,\, v) \Downarrow \{v\} \quad v \text{ irreducible} \\
&(\textsc{Dispatch}) && \mathrm{eval}(s,\, (f\,\bar{a})) \Downarrow \biguplus_{(=\ (f\,\bar{p})\ r) \in s,\ \sigma = \mathrm{match}(\bar{p},\bar{a})} \mathrm{eval}(s,\, r\sigma) \\
&(\textsc{Ground}) && \mathrm{eval}(s,\, (g\,\bar{a})) \Downarrow \mathrm{apply}(g, \bar{a}) \quad g \text{ grounded; arguments per } g\text{'s contract} \\
&(\textsc{Match}) && \mathrm{eval}(s,\, (\mathrm{match}\ s'\ p\ t)) \Downarrow \biguplus_{\sigma \in \mathrm{query}(s',p)} \{\, t\sigma \,\} \\
&(\textsc{Effect}) && \mathrm{eval}(s,\, (\mathrm{add\text{-}atom}\ s'\ a)) \Downarrow \{()\} \text{ bumping } s'\text{'s revision}
\end{aligned}$$

Multiset union $\uplus$ is the whole story of nondeterminism: no rule orders branches, and no observation may depend on branch order. Equations are members of s, so Dispatch consults the store at its current revision — rules are data.

Prime lane. Application allocates rather than recurses: $\mathrm{eval}(s,\, (f\,\bar{a}))$ binds each argument to a *need cell* (allocated once, shared by every candidate equation — call-time choice), and evaluation proceeds by *forcing* cells the taken path demands. Forcing a completed cell replays its cached bag; forcing a fresh cell evaluates under the cell's captured lexical environment and memoizes values and stable faults. A budgeted observation of this process reads:

$$\frac{\mathrm{force}^{\le k}(\mathrm{eval}(s,\, e)) \ni v}{s \vdash \mathrm{May}\ (\mathrm{PrimeEvaluate}\ e)\ v \quad \mathrm{Established}\ (k)}$$

read: within budget k, some demand-driven run of e reaches v. Budget-indexed observations of one run are monotone for deterministic tails and eventually constant in general (the fuel-convergence discipline); the judgment therefore asserts a property of the run, not an artifact of its truncation.

H Moving from HE and PeTTa to Prime

Prime is a sibling, not an extension: most programs written against the shared fragment run unchanged, and the differences that matter are exactly the ones laziness makes visible. A migration checklist:

1. **Unneeded arguments may never run.** Under call-by-need an argument the taken path never demands is never evaluated: `(const 5 (loop))` returns `5` in Prime where an eager engine diverges. Programs that relied on argument evaluation *for its side effects* must put the effect in demanded position.

2. **Effects fire on demand.** An `add-atom` buried in a never-demanded subterm does not fire. Sequence effects through demanded positions (e.g. a `let` whose body uses the bound result) rather than relying on evaluation order.

3. **Sharing is observable (call-time choice).** One delayed nondeterministic choice used twice yields *correlated* outcomes: `(pair x x)` with `x` a shared coin has two outcomes, not four. This is the semantics that makes the shared-cause probability discriminator exact (1/2, not 3/4) — migrating probabilistic code should treat it as a feature, and code that wanted independent draws must introduce two sources.

4. **Strict equality forces.** `==` demands the structure of both sides (a `STRUCTURE`-demand rule), so idioms like `(filter-atom xs $a (== $a v))` behave as in HE.

5. **Bare `$` is anonymous.** Rename any HE program that used bare `$` as a named variable; in Prime each occurrence is fresh and binds nothing referenceable.

6. **Hold syntax with `(Data Atom)`.** An HE parameter typed `Atom` receives its argument unevaluated; in the Prime draft such a parameter is evaluated, and `(Data Atom)` is the type that holds syntax as written (Section 8).

7. **Import into a fresh name.** `import!` into a space bound earlier is refused on the HE and Prime lanes; import into `&self` or a name not yet bound (Section 2.7).

8. **The standard library is editable.** An equation for a library name adds an alternative in Prime, where HE ignores it and PeTTa refuses it; `std:` names reach the originals (Section 2.8).

9. **Memoization is opt-in.** `(pragma! search-table-mode variant)` enables the variant table for the session. Nothing is memoized by default.

10. **Faults are data at a boundary.** Ordinary evaluation propagates faults; use the structural observers to inspect them deliberately instead of pattern-matching on error shapes mid-flow.

11. **What does not change.** Bag semantics and non-semantic ordering; `match` and space effects on the shared fragment; equations as unordered space members; the profile discipline (silent fall-through for builtins a profile does not provide).

From PeTTa specifically, the additional step is dropping reliance on clause order and cut-like control: Prime's dispatch explores an unordered candidate frontier under a policy-neutral certified control layer, so programs encoding priority through textual order must encode it in data instead.

I Implementation Algorithms and Asymptotics

This appendix records the algorithmic decisions behind the engine's current asymptotics, in the order an atom meets them. Design principle throughout: identity is assigned once at the store boundary, inner loops speak identifiers rather than bytes, and every optimization must be a homomorphism of the observable algebra (bag union, conjunctive query, oriented rewrite, reify, sharing) — enforced by differential and planted-mutation gates rather than review alone.

I.1 Interning and the slot-index boundary

All terms intern into a term universe: structural bytes are hashed incrementally per piece, and equality on the hot path is identifier comparison. The July campaign exposed a subtle obligation: the *stored* record hash and the *slot index* derived from it are different roles. An incremental `(h*33)^piece` hash keeps a name family's entropy in the low bits, so families like `(friend sam $k)` collapsed into one open-addressing cluster — each new sibling's probe walked the whole family, $O(N^2)$ in aggregate. The repair applies an avalanche finalizer (fmix-style) at exactly the four sites where a stored hash becomes a slot index, through one shared function so lookup and insert cannot disagree; the stored hash — and every contract downstream of it — is untouched. Bulk ingestion is $O(N)$; the harness fell from tens of seconds to fractions of a second at 100k facts. The general lesson is recorded as a store obligation: constant carving reach in *shape* and in *constant* — an access path whose cost grows with sibling count is a defect even when memory stays linear.

The avalanche boundary repairs the average distribution but does not by itself rule out primary clustering. A later adversarial structural family formed a 71-slot chain under linear probing despite a healthy mean. The current table derives a second hash from the finalized key, forces its stride odd, and advances by that stride modulo the power-of-two capacity. Because an odd stride is coprime to the capacity, lookup, insertion, and rehash share a full-cycle probe rather than a fixed 64-slot escape hatch. The regression checks both exact lookup/rehash equivalence and a tail bound scaled to the workload; table capacity, structural equality, and stored hashes are unchanged.

I.2 Match indexing and selectivity

Pattern queries route through a backend seam (candidates, query, and a reserved whole-conjunction operation). The native discrimination index stores per-position keys; the campaign's design correction is *selectivity-gated discrimination*: a position discriminates only where it has selectivity, and high-cardinality leaf positions collapse to flat lists under their prefix (preserving $O(1)$ ground lookup and the $O(N)$ variable walk while removing the per-leaf node overhead that dominated index memory at scale).

I.3 Arena discipline

Evaluation scratch lives in arenas. Two proven refinements: branch arenas — an all-solutions engine gains nothing from backtracking-style undo, so each nondeterministic branch marks and releases its scratch, with survivors copied out — and per-iteration reset on the deterministic-tail loop lane, where the no-escape argument is strongest (effect-only, unit-result branches). Together they inverted the paired memory ladders: insertion workloads now peak below the pre-Prime baseline.

I.4 The deterministic-tail loop lane

A single-branch tail self-call executes as a loop: constant frames, accumulator-threaded effects, per-iteration arena reset, with any nondeterministic tail falling back to the general evaluator (the call-time-choice guard). The lane's semantic license is kernel-checked (`Machines/DeterministicTail`): iterative execution equals fuel-indexed recursive evaluation — answers and effect *multisets* alike; halting deterministic-tail runs are precisely the fuel-convergent class; and per-unfolding invariants transfer to whole-run results. Resource behaviour is deliberately outside the theorem: the differential and ladder gates check the implementation against it.

I.5 Tables and views

Memo structures share the revision discipline: a table entry is valid only at the exact space revision that produced it. The session's variant table (`(pragma! search-table-mode variant)`) is the memoization a program can request; an earlier ground-call table with whole-bag replay was retired on 11 August 2026. The revision-keyed *compiled view* direction starts from per-(head, revision) admissibility bits and value-specific type-fact gates, and grows toward loop-body views derived from the specification by partial evaluation rather than bolted beside the evaluator as caches.

I.6 Receipts under the valve

Histories are recorded at seams, on demand, under budget. Prime's per-occurrence bookkeeping (world, need-cell identity, causal receipt, multiplicity, fault status, pinned revision) lives behind one lazy extension pointer, `NULL` on the HE lane, so provenance-bearing evaluation and provenance-free evaluation share code without sharing cost. Authority never flows from traces (the one-way evidence valve), which is what licenses throttling them arbitrarily for performance.

I.7 Asymptotic summary

operation	cost	note
intern / equality	$O(1)$ amortized	avalanche-finalized slots
bulk ingest (N adds)	$O(N)$	was $O(N^2)$ pre-repair
ground exact-hit query	$O(1)$ expected	index probe
variable-pattern match	$O(N)$ output-linear	seam visitors
counting a match set	O(matches)	streamed, no tuple
deterministic tail loop	$O(1)$ frames, flat RSS	lane + kernel
judge-path recording	$O(1)$ per step	was super-linear

The remaining engineering frontier is a constant factor against the reference Prolog engine's assert path, under active reduction via the compiled-view work; no super-linear behaviour is currently known on the covered workload families.

References

[1] B. Goertzel et al., "MeTTa: A Reflective Language for AGI," OpenCog Foundation, 2024.

[2] V. Saraswat, D. Weinbaum, K. Kahn, and E. Shapiro, "Detecting Stable Properties of Networks in Concurrent Logic Programming Languages," in *Proc. 7th ACM Symposium on Principles of Distributed Computing (PODC)*, pp. 210–222, 1988.

[3] Z. Goertzel and Oruži, "MeTTaIL: Contract-First Boot Compilation for Direct C MeTTa Runtimes," Working paper, 2026.

[4] L. G. Meredith and M. Radestock, "A Reflective Higher-Order Calculus," *Electronic Notes in Theoretical Computer Science*, vol. 141, no. 5, pp. 49–67, 2005.

[5] L. G. Meredith, "Continued Interactive GSLTs and the Cost Endofunctor: Wrapping by Construction," Working paper, 2026.

[6] Mettapedia, "Pure Rho and Cost: One Authored Calculus, Named Extensions, and Corrected Hypotheses," Working paper, 2026.

[7] R. Atkey, "Parameterised Notions of Computation," *Journal of Functional Programming*, vol. 19, no. 3–4, pp. 335–376, 2009.

[8] S. Katsumata, "Parametric Effect Monads and Semantics of Effect Systems," in *Proc. POPL*, 2014.

[9] D. Orchard, P. Wadler, and H. Eades III, "Unifying Graded and Parameterised Monads," in *Proc. MSFP*, 2020.

[10] V. Khudobakhshov, "JeTTa — Performance: Compiling MeTTa to JVM Bytecode," Slide deck, July 2026.

[11] Oruži and Z. Goertzel, "JeTTa: MeTTa Compiled to JVM Bytecode — A Source-Level Review, Cross-Engine Benchmark Study, and Defect Catalog," Working paper, 2026.

[12] K. H. Knuth and J. Skilling, "Foundations of Inference," *Axioms*, vol. 1, no. 1, pp. 38–73, 2012.

[13] SWI-Prolog Project, "Update View," SWI-Prolog Reference Manual, `https://www.swi-prolog.org/pldoc/man?section=update`.

[14] Salsa Project, "The Red–Green Algorithm," Salsa Reference, `https://salsa-rs.github.io/salsa/reference/algorithm.html`.

[15] Rust Project Developers, "Incremental Compilation," *Rust Compiler Development Guide*, `https://rustc-dev-guide.rust-lang.org/queries/incremental-compilation.html`.

[16] A. Mokhov, N. Mitchell, and S. Peyton Jones, "Build Systems à la Carte," *Proceedings of the ACM on Programming Languages*, vol. 2, ICFP, Article 79, 2018. `https://doi.org/10.1145/3236774`.

[17] M. A. Hammer, Khoo Yit Phang, M. Hicks, and J. S. Foster, "Adapton: Composable, Demand-Driven Incremental Computation," in *Proc. PLDI*, pp. 156–166, 2014. `https://doi.org/10.1145/2594291.2594324`.

[18] F. McSherry, D. G. Murray, R. Isaacs, and M. Isard, "Differential Dataflow," in *Proc. CIDR*, 2013; see also "Arrangements," Differential Dataflow Book, `https://timelydataflow.github.io/differential-dataflow/chapter_5/chapter_5.html`.

[19] G. Sutcliffe, "The TPTP Language and Syntax BNF," TPTP World user documentation, `https://tptp.org/UserDocs/TPTPLanguage/TPTPLanguage.shtml`.

[20] G. Sutcliffe, "The TPTP Format for Derivations," TPTP World quick guide, `https://tptp.org/UserDocs/QuickGuide/Derivations.html`.

[21] Oruži and Z. Goertzel, "The Interaction Basis: Spaces, Channels and Concurrency for MeTTa Prime," Working paper, 27 October 2026.

[22] H. Ehrig, K. Ehrig, U. Prange, and G. Taentzer, *Fundamentals of Algebraic Graph Transformation*, Monographs in Theoretical Computer Science, Springer, 2006.

[23] D. Gelernter, "Generative Communication in Linda," *ACM Transactions on Programming Languages and Systems*, vol. 7, no. 1, pp. 80–112, 1985.

[24] T. Frühwirth, "Theory and Practice of Constraint Handling Rules," *Journal of Logic Programming*, vol. 37, no. 1–3, pp. 95–138, 1998.

[25] C. Fournet and G. Gonthier, "The Reflexive CHAM and the Join-Calculus," in *Proc. POPL*, pp. 372–385, 1996.

[26] T. Harris, S. Marlow, S. Peyton Jones, and M. Herlihy, "Composable Memory Transactions," in *Proc. PPoPP*, pp. 48–60, 2005.

[27] K. P. Eswaran, J. N. Gray, R. A. Lorie, and I. L. Traiger, "The Notions of Consistency and Predicate Locks in a Database System," *Communications of the ACM*, 19(11):624–633, 1976.